期中复习题库（第 1—6 章）

目录

第 1 章 概论 **3**
一、填空题 3
二、选择题 4
三、辨析与简答 5
四、算法设计与分析 6
五、数学证明 8

第 2 章 线性表 **11**
一、填空题 11
二、选择题 11
三、辨析与简答 13
四、算法设计与分析 14
五、数学证明 19

第 3 章 栈与队列 **21**
一、填空题 21
二、选择题 22
三、辨析与简答 23
四、算法设计与分析 25
五、数学证明 34

第 4 章 字符串 **39**
一、填空题 39
二、选择题 40
三、辨析与简答 41
四、算法设计与分析 44
五、数学证明 48

第 5 章 二叉树 **51**
一、填空题 51
二、选择题 53
三、辨析与简答 54
四、算法设计与分析 56
五、数学证明 68

第 6 章 树 **71**
一、填空题 71
二、选择题 72
三、辨析与简答 74

四、算法设计与分析 77
五、数学证明 85

模拟卷（120 分钟，100 分） **89**
一、填空题（每题 2 分，第 1 题两空各 1 分，共 20 分） 89
二、选择题（每题 2 分，共 16 分） 89
三、辨析与简答题（每题 6 分，共 24 分） 90
四、算法设计与分析题（每题 10 分，共 30 分） 91
五、证明题（10 分） 93

考前速查表 **95**
渐进分析 95
线性表 95
栈与队列 96
字符串 97
二叉树 97
树与森林 98
答题格式提醒 99

本篇按《数据结构与算法》期中复习提要逐条配题，范围是第 1 章到第 6 章。提要里标 ★ 的条目 是重中之重，题号后同样标 ★；没标的也配了题，但分量轻一些。**提要原件逐字存在仓库里**：`ref_ 复习大纲/期中复习提要.md`。

怎么用：分章题答案跟在题目下面的引用块里，模拟卷答案集中在各组之后，先把引用块遮住自己做一遍，再对答案。每章按考试题型分五组：填空、选择、辨析与简答、数据结构或算法的设计与分析、数学证明，分章题共 184 道；之后是一套完整的模拟卷（26 道，按真实考试的分值和时间配比出的），最后一页是速查表。分章题已对照 2021—2024 年的期中试卷逐题核过考点，试卷里有而这里缺的都补了题（题面与数据另换）。

几个贯穿全篇的约定（考试写答案时最好也先声明一句，避免歧义）：

- 数组、字符串、树的层次编号一律**从 0 开始**；根结点在**第 0 层**。凡是与此不同的题，题面会单独说明。
- 「深度为k 的二叉树」指最大层号为k，此时结点数至多$2^{k+1}-1$；树的「高度」默认指**层数**（若按边数计会单独说明）；「n 个结点的完全二叉树高度」按提要口径同样指**层数**，即$\lceil \log_2(n+1) \rceil$。
- 算法设计题按提要的要求：**写明基本思想、无歧义即可**，必要时加注释。用到栈、队列等现成结构时，先写出 ADT（函数名、入口参数、出口参数、功能一句话），再在算法里调用。本篇提供了常用 ADT 的写法，后续题目可引用相应接口。
- 本篇的「满二叉树／满K 叉树」采用严格（正则）树的含义：每个内部结点恰有 2 ／ K 个孩子，不要求叶结点同层；所有层都填满时另称「各层全满」。K、k 出现在除以$K-1$、$k-1$ 的公式中时均要求至少为 2。
- 除另有说明，结点数据的比较、赋值和交换按常数时间计；树的遍历序列用于还原结构时，结点标识互不相同。
- 空指针计数公式均针对非空树，且只计结点内的孩子指针域。
- 伪代码用缩进表示层次，← 是赋值，= 是比较；逻辑运算采用短路求值。

第 1 章 概论

一、填空题

1-1. 数据结构的研究内容包括三个方面：____、____ 和 ____。

答：数据的**逻辑结构**、数据的**存储结构**（物理结构）、**在此结构上定义的运算**（算法）。三者的关系是：逻辑结构描述「数据之间是什么关系」，存储结构描述「这些关系在内存里怎么落实」，运算的代价则由前两者共同决定。见第 1.2 节。

1-2. 抽象数据类型（ADT）由 ____ 和 ____ 两部分组成，它**不包括** ____。

答：由**数据对象及其上的关系**（数据模型）和**定义在其上的一组操作**组成；它**不包括数据的存储结构**。这一条是历年反复考的辨析点：ADT 只回答「能做什么」，把「怎么存、怎么做」留给实现。一旦把存储方式写进 ADT，它就不再「抽象」了。也常把前一部分拆开，说成「数据对象、数据关系、基本操作」三部分（模拟卷一·1 即按此口径），内容相同。

1-3. 数据的四种基本存储方式是 ____、____、____、____。

答：**顺序**存储、**链接**存储、**索引**存储、**散列**存储。顺序靠下标算地址，随机访问$O(1)$、插删要搬元素；链接靠指针相连，已知所需结点及前驱时插删$O(1)$，但不能随机访问；索引另建索引表，借助索引定位记录；散列由关键码直接算地址，平均$O(1)$ 但要处理冲突。

1-4. 大O 表示法的定义：称$f(n)=O(g(n))$，当且仅当存在正常数 ____ 和 ____，使得对所有 ____ 都有 ____ 成立。

答：存在正常数c 和n_0，使得对所有$n \ge n_0$ 都有$f(n) \le c \cdot g(n)$。三个要点缺一不可：c 与n 无关、只要求n 充分大之后成立、是**上界**而不是确界。

1-5. 把下列函数按渐进增长率从小到大排序：$n\log n$，2^n，n^2，$\log n$，$n!$，n，n^3，1，$\sqrt{n}$，2^{2^n}。

答：$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < 2^n < n! < 2^{2^n}$。常错的两处：$\sqrt{n}$ 在$\log n$ 之后（任何正数次幂都最终超过对数）；$n!$ 在2^n 之后（由$n! \ge (n/2)^{n/2}$ 可得）。

1-6. 本书列出的算法基本性质有 ____、____、____、____。

答：**通用性**（对符合输入类型的任意输入都能正确求解）、**有效性**（每条指令都能被人或机器确切执行）、**确定性**（每步之后下一步怎么走有明确指示）、**有穷性**（有限步内结束）。见第 1.3.1 节。少了有穷性的就只是「计算过程」（如操作系统主循环），不叫算法。另一种常见口径是 Knuth 的「五特性」：有穷性、确定性、可行性、输入、输出——有效性大致对应可行性，输入输出则体现在通用性之中。题面若明说「五个特性」就按这一套答。

1-7. 数据的逻辑结构用二元组(K, R)表示时，K 是 ___，R 是 ___；画结构图时，R 中的序偶$\langle x, y\rangle$ 必须画成 ___。

> **答**：K 是**结点（数据元素）的有穷集合**，R 是**定义在**K **上的关系的集合**；$\langle x, y\rangle$ 必须画成**从**x **指向**y **的有向边**（箭头方向不能省，$\langle x, y\rangle$ 与$\langle y, x\rangle$ 是两条不同的边）。只有明确声明关系对称时才画成无向边。

1-8. 三个算法的时间代价分别为：$T_1(n) = 2T_1(n-1) + 3,\ T_1(1) = 1$；$T_2(n) = n^2 + n\sqrt{n}\log_2 n$；$T_3(n) = 4^{n/2} + n^{10}$。它们的渐进紧确界依次是 ___、___、___；按增长从高到低排列为 ___。

> **答**：$\Theta(2^n)$、$\Theta(n^2)$、$\Theta(2^n)$；T_1 与T_3 **同阶**，都高于T_2。
>
> - T_1：两边同加 3 得$T_1(n)+3 = 2\,(T_1(n-1)+3)$，于是$T_1(n)+3 = 2^{n-1}(T_1(1)+3) = 2^{n+1}$，即$T_1(n) = 2^{n+1} - 3$。
> - T_2：$n\sqrt{n}\log n = n^{1.5}\log n$，而$\log n$ 比任何正数次幂（这里是$n^{0.5}$）长得慢，被n^2 吸收。
> - T_3：**陷阱**——$4^{n/2} = (2^2)^{n/2} = 2^n$，并不比$T_1$ 高；n^{10} 再大，也终究被2^n 超过。
>
> 排序题碰到同阶要明写出来（如「T_1 与T_3 同阶，高于T_2」），硬排出先后会被扣分。

二、选择题

1-9. 下列关于抽象数据类型的叙述，**错误**的是（　）。A. ADT 给出了数据对象、数据关系和基本运算 B. ADT 的定义中包含了数据的存储结构 C. 同一个 ADT 可以有多种不同的实现 D. ADT 把「做什么」和「怎么做」分开

> **答：B**。ADT 与具体存储无关。栈这个 ADT 既可以用数组实现（顺序栈），也可以用链表实现（链式栈），两者的 ADT 完全相同，接口一字不差，这正是 C 和 D 说的事情。

1-10. 以下说法正确的是（　）。A. 算法的时间代价必定随问题规模单调增长 B. 若$f(n) = O(n^2)$，则$f(n)$ 一定不是$O(n)$ C. 大O 给出的是渐进上界，同一个函数可以有多个正确的大O 表达式 D. 最坏情况分析和大O 分析是同一件事

> **答：C**。$f(n) = n$ 既是$O(n)$ 也是$O(n^2)$，所以 B 错、C 对。D 是最常见的混淆：「**最好/最坏/平均」说的是输入的挑法，「**$O/\Omega/\Theta$**」说的是函数的界法**，两者是正交的两个维度——完全可以说「最好情况是$O(n)$」。A 错：代价未必对规模单调，例如插入排序对已有序的大输入只需$O(n)$，可能比更小的逆序输入还快。

1-11. 下面程序段中循环体执行次数的渐进紧确界是（　）。

```
s ← 0;  i ← 1
do
    s ← s + 5 * i
    i ← i + 1
```

```
while (s < n) and (i < n)
```

A. $\Theta(n)$　B. $\Theta(\sqrt{n})$　C. $\Theta(\log n)$　D. $\Theta(n^2)$

答：B。循环i次后$s = 5(1+2+\cdots+i) = \frac{5i(i+1)}{2}$，当$n$充分大时，退出条件由$s \geq n$先触发，于是$i = \Theta(\sqrt{n})$。这题的陷阱是 and 右边那个 `i < n` ——它只能给出较松的$O(n)$上界，不能据此认定为$\Theta(n)$，**两个条件取先满足的那个**。

1-12. 对同一问题，算法 A 的时间代价为$1000n$，算法 B 为n^2。下列说法正确的是（　）。A. A 总比 B 快　B. B 总比 A 快　C. $n > 1000$ 时 A 比 B 快　D. 两者渐进同阶

答：C。渐进分析丢掉常数，是因为**规模足够大时常数必然被增长率淹没**；但「足够大」是多大要具体算：这里的分界点正是$n = 1000$。实际工程里n若始终只有几十，反而该选 B——这也是第 1.4.4 节说的「选择和评价」。

1-13. 下列各式中**不成立**的是（　）。A. $3n^2 + 10n = O(n^2)$　B. $n^2/(n+1) = O(n)$　C. $2^{n+1} = O(2^n)$　D. $2^{2n} = O(2^n)$

答：D。$2^{2n} = (2^n)^2$，比值2^n无界。C 成立是因为$2^{n+1} = 2 \cdot 2^n$，只差常数因子 2。B 成立是因为$n^2/(n+1) < n$。

1-14. 下面两段程序中 s++ 执行次数的渐进紧确界分别是（　）。

```
// 程序甲
for (i = 1; i <= n; i++)
    for (j = i; j >= 1; j /= 3)
        s++;
```

```
// 程序乙
for (i = 1; i <= n; i *= 2)
    for (j = 1; j <= i; j++)
        s++;
```

A. $\Theta(n \log n)$，$\Theta(n \log n)$　B. $\Theta(n \log n)$，$\Theta(n)$　C. $\Theta(n)$，$\Theta(n \log n)$　D. $\Theta(n^2)$，$\Theta(n)$

答：B。**甲**：对固定的i，j 从i起每次整除 3，到 0 为止，内层执行$\lfloor \log_3 i \rfloor + 1$次；总数$\sum_{i=1}^{n} (\lfloor \log_3 i \rfloor + 1) = \Theta(\log n!) = \Theta(n \log n)$（1-28）。**乙**：外层$i = 1, 2, 4, \cdots, 2^k$（$2^k \leq n < 2^{k+1}$），内层执行$i$次，总数$1 + 2 + \cdots + 2^k = 2^{k+1} - 1$，介于$n$与$2n$之间，是$\Theta(n)$。乙最容易错答成$\Theta(n \log n)$：外层确实只有$\log n$轮，但内层次数是**几何级数**，被最后一轮主导，不能拿「轮数 × 最大一轮」当紧确界。（$n = 1000$时脚本实数：甲 5914 次，乙 1023 次。）

三、辨析与简答

1-15. 辨析：「数据的逻辑结构一旦确定，存储结构也就确定了。」

答：错。同一逻辑结构可以有多种存储结构，这正是本课程的核心。线性表既可以顺序存储（顺序表）也可以链接存储（链表）；树既可以用

左子/右兄二叉链存，也可以用父指针数组存，还可以按先根次序排进一维数组。**逻辑结构决定「能做什么」，存储结构决定「做得多快」**。

1-16. ★ 简答：最坏情况分析与大O表示法有什么区别和联系？为什么说「某算法的时间复杂度是$O(n^2)$」这句话本身并不说明它是最坏情况？

答：1. **它们回答的是不同的问题**。「最好/最坏/平均情况」是在**所有规模为n的输入**里挑一个 来分析：最坏是取代价最大的那个输入，平均是按输入分布取期望。而$O/\Omega/\Theta$ 是在**确定了某个输入类别之后**，给这条代价曲线定界：O 是上界、Ω 是下界、Θ 是同阶。2. **所以两个维度可以自由组合**，「最好情况是$\Theta(n)$」「最坏情况是$O(n^2)$」都是合法句子。3. 因此单说「复杂度是$O(n^2)$」只承诺了「不会比n^2 更糟」，**并没有说清是对哪一类输入说的**，严谨的写法要把两个维度都点明。考试答题时建议写成「最坏情况下为$O(n^2)$」。

1-17. 简答：时间与空间的折衷，举一个例子说明。

答：多用空间往往能省时间，反之亦然。例：判断一个字符是否属于某集合，逐个比较是$O(|S|)$ 时间、$O(1)$ 额外空间；预先建一张 256 项的布尔表，则是$O(1)$ 时间、$O(256)$ 空间。又如第 4 章 KMP 的 next 数组：多花$O(|P|)$ 空间，把匹配从$O(|P|\cdot|T|)$ 降到$O(|P|+|T|)$。反向的例子是原地排序：不借助辅助数组，用时间换空间。

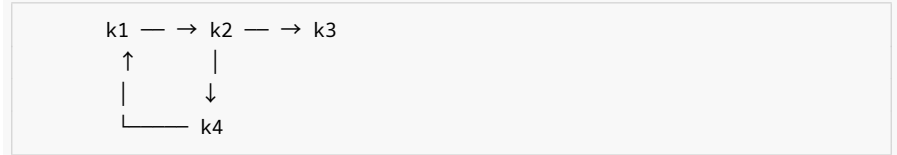
1-18. 简答：为什么渐进分析要「保留增长最快的那一项」并丢掉常数因子？什么时候这样做会误导人？

答：因为当$n\to\infty$ 时，低阶项相对最高阶项的比值趋于 0；固定的正倍数则不改变渐进阶。省略它们能突出不同算法随规模增长的主要差别。会误导人的情形有三类：(1) n **始终很小**——此时常数是主角（见 1-12）；(2) **常数因子大得离谱**——某些理论上更优的算法常数是天文数字，实际从不使用；(3) **关心的不是渐进而是确切次数**——例如比较次数下界这类精确结论。

四、算法设计与分析

1-19. 设$K=\{k_1,k_2,k_3,k_4\}$，$R=\{\langle k_1,k_2\rangle,\langle k_2,k_3\rangle,\langle k_2,k_4\rangle,\langle k_4,k_1\rangle\}$。画出该逻辑结构的示意图，并说明它属于哪一类结构。

答：四个结点，画四条**有向**边：$k_1\to k_2$、$k_2\to k_3$、$k_2\to k_4$、$k_4\to k_1$。



它**不是**线性结构（k_2 有两个后继），也**不是**树（存在回路$k_1\to k_2\to k_4\to k_1$，且k_1 有前驱），因此属于**图**结构（有向图）。评分要点：箭

头方向必须画对；结论要给出判断依据，不能只写「是图」。

1-20. ★ 设计：某系统要维护一批学生成绩，要求支持三种操作—— (a) 按学号查一个学生 (b) 插入一个新学生 (c) 按成绩从高到低输出全部学生。若三种操作的频率相当，请给出一种数据结构设计并分析代价；若 (c) 只在学期末执行一次，设计又该怎么改?

> **答**：**情形一（三者频率相当）**：按学号组织一棵平衡二叉搜索树（平衡树可作为第 5 章 BST 的改进），另在每个结点里存成绩。- (a) 查找$O(\log n)$（树平衡时）；- (b) 插入$O(\log n)$；- (c) 中序遍历得到按学号有序的序列，再排序，$O(n \log n)$。
>
> 若 (c) 也频繁，可**同时维护两个索引**：学号平衡 BST 与成绩平衡 BST，以（成绩，学号）作为成绩索引的关键码以处理同分，两棵树的结点指向同一份记录，插入时各插一次，仍是$O(\log n)$；(c) 变成一次逆中序遍历（右、根、左）$O(n)$。代价是空间翻倍、且两个索引必须同步更新——**这就是典型的时空折衷**。
>
> **情形二（(c) 只在期末一次）**：不必为 (c) 付日常代价。只保留学号索引（BST 或散列表，散列时 (a)(b) 平均$O(1)$），期末一次性取出全部记录排序，$O(n \log n)$ 只发生一次。
>
> 评分要点：**先说清每种操作的频率如何影响选择**，再给结构，最后逐条给代价。只写「用二叉搜索树」而不分析，拿不到设计题的分。

1-21. ★ 分析下列程序段中 `x ← x + 1` 的执行次数m，并给出时间复杂度。

```
for i ← 1 to n
    for j ← 1 to n - 2 * i + 1
        x ← x + 1
```

> **答**：内层对每个i 执行$\max(0,\, n-2i+1)$ 次，只有$i \le n/2$ 时才进入循环，于是
>
> $$m = \sum_{i=1}^{\lfloor n/2 \rfloor} (n-2i+1) = \frac{n}{2}(n+1) - 2\cdot\frac{(n/2)(n/2+1)}{2} = \frac{n^2}{4}$$
>
> （取n 为偶数；$n=10$ 时$m=25$，$n=100$ 时$m=2500$，与公式相符。n 为奇数时求和上限是$(n-1)/2$，同法算得$m=\frac{n^2-1}{4}$，如$n=11$ 时$m=30$。）时间复杂度$\Theta(n^2)$。评分要点：**先写出求和式再化简**，直接目测「两重循环所以是n^2」在本题恰好对，但遇到内层次数与i 相关且可能为负的情形就会错。

1-22. ★ 分析下面程序段中内层循环体的执行次数，给出时间复杂度。

```
for i ← 1 to n
    j ← i
    while j ⩽ n
        操作
        j ← j + i
```

答：内层对每个i，j 取遍$i, 2i, 3i, \cdots$，执行$\lfloor n/i \rfloor$ 次，于是总次数

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor \;\approx\; n \sum_{i=1}^{n} \frac{1}{i} \;=\; n \cdot H_n$$

而**调和数** $H_n = \ln n + \gamma + O(1/n) = \Theta(\log n)$，故总次数为$\Theta(n \log n)$。（$n = 10^4$ 时实算是$93\,668$，$n \ln n \approx 92\,103$，量级吻合。）

这道题的价值在于「内层次数既不是常数也不是n」。看见两重循环就写$\Theta(n^2)$ 是错的——那要求内层每次都跑$\Theta(n)$ 次，而这里只有$i = 1$ 那一轮如此，i 一大内层就很短。做法仍是 1-21 的老套路：**先写出求和式，再化简**；区别只在这次求和的是n/i 而不是常数。

顺便把这个界记住：$1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\log n)$。它在「把每个数的倍数都划一遍」的筛法的代价、快速排序的平均情况、散列表的平均探查次数里都会再出现。（真正的埃拉托色尼筛法只划**素数**的倍数，代价更低，是$\Theta(n \log \log n)$，别把两者混为一谈。）

1-23. ★ 求下列递推式的渐进解：(1) $T(n) = T(n/2) + c$；(2) $T(n) = 2T(n/2) + c$；(3) $T(n) = 2T(n/2) + n$；(4) $T(n) = T(n-1) + n$。

答：(1) $\Theta(\log n)$——每层只花常数，共$\log n$ 层（二分检索）。(2) $\Theta(n)$——递归树有n 个叶结点，每个结点花常数，总代价与结点数同阶。(3) $\Theta(n \log n)$——**每一层的总代价都是**n，共$\Theta(\log n)$ 层（归并排序、快排最好情况）。(4) $\Theta(n^2)$——$T(n) = n + (n-1) + \cdots = n(n+1)/2$（快排最坏情况）。

通用做法：画递归树，先算「每层总代价」，再乘「层数」；不确定时用代入法验证。

五、数学证明

1-24. 用大O 的定义证明$3n^2 + 10n + 7 = O(n^2)$，并给出一组具体的c 与n_0。

证明. 当$n \geq 1$ 时，$10n \leq 10n^2$、$7 \leq 7n^2$，故$3n^2 + 10n + 7 \leq 3n^2 + 10n^2 + 7n^2 = 20n^2$。取$c = 20$、$n_0 = 1$，对一切$n \geq n_0$ 有$3n^2 + 10n + 7 \leq c\,n^2$，由定义即得。∎

说明：c 不唯一。取$n_0 = 10$ 时$c = 5$ 也够用（$10n \leq n^2$、$7 \leq n^2$，得$\leq 5n^2$）。只要**给出一对具体的**(c, n_0) **并验证不等式**即可，不必求

最优。

1-25. 证明：对任意非负函数f, g，有$\max\{f(n), g(n)\} = \Theta(f(n) + g(n))$。

证明. 一方面$\max\{f, g\} \le f + g$（另一个非负），取$c_2 = 1$得上界。另一方面$f + g \le 2\max\{f, g\}$，即$\max\{f, g\} \ge \frac{1}{2}(f + g)$，取$c_1 = \frac{1}{2}$得下界。两侧对一切$n$成立（$n_0 = 1$），故$\frac{1}{2}(f + g) \le \max\{f, g\} \le f + g$，即$\Theta$关系成立。∎

这条结论的用处：程序里「先做 A 再做 B」的总代价与「两者中较慢的那个」同阶——这正是渐进分析里「取最慢的一项」的严格依据。

1-26. ★ 证明$T(n) = 2T(\lfloor n/2 \rfloor) + n$（$T(1) = 1$）的解是$\Theta(n \log n)$。

证明. 上界（递归树）：第i层有2^i个规模为$\lfloor n/2^i \rfloor$的子问题，该层总代价$2^i \lfloor n/2^i \rfloor \le n$（叶结点代价也是其规模 1）。最大层号为$\lfloor \log_2 n \rfloor$（共$\lfloor \log_2 n \rfloor + 1$层），故总代价$\le n(\log_2 n + 1) = O(n \log n)$。

代入法验证：设$T(k) \le ck \log_2 k$对一切$2 \le k < n$成立，则$T(n) \le 2c\frac{n}{2}\log_2 \frac{n}{2} + n = cn \log_2 n - cn + n \le cn \log_2 n$（只要$c \ge 1$）。**基础情形要单独核**：$n = 1$时右边$c \cdot 1 \cdot \log_2 1 = 0 < T(1) = 1$，不等式本身不成立，所以归纳只能从$n = 2, 3$起步（它们的子问题规模$\lfloor n/2 \rfloor = 1$直接用$T(1) = 1$代入）：$T(2) = 4 \le 2c$、$T(3) = 5 \le 3c \log_2 3 \approx 4.75c$，取$c = 2$都成立，此后$n \ge 4$的子问题规模都$\ge 2$，上面的归纳步就能接着用。大$O$只要求$n \ge n_0$，这样做完全合法。

下界：令$h = \lfloor \log_2 n \rfloor$。对$0 \le i < h$，第$i$层的子问题规模至少为 2，且该层代价$2^i \lfloor n/2^i \rfloor \ge n - 2^i \ge n/2$。因此$T(n) \ge (n/2)h = \Omega(n \log n)$。上下界同阶，得$T(n) = \Theta(n \log n)$。∎

1-27. 设f, g为非负函数（(1) 中取对数时要求$f, g > 0$），判断下列命题真假并证明或举反例：(1) 若$f(n) = O(g(n))$，则$\log f(n) = O(\log g(n))$；(2) 若$f(n) = O(g(n))$，则$2^{f(n)} = O(2^{g(n)})$；(3) 若$f(n) = O(g(n))$，则$f(n)^2 = O(g(n)^2)$。

答：(1) **不成立**。取$f(n) = 2$、$g(n) = 1$：$f = O(g)$成立，但$\log_2 f = 1$、$\log_2 g = 0$，而$1 = O(0)$不成立。（若补充前提$f, g \ge 2$则命题成立。）(2) **不成立**。取$f(n) = 2n$、$g(n) = n$：$f = O(g)$，但$2^{2n} = (2^n)^2$与2^n之比为2^n，无界。**教训：大O关系不能直接放进指数**。(3) **成立**。由定义存在c, n_0使$n \ge n_0$时$f \le cg$，两边非负，平方得$f^2 \le c^2 g^2$，取常数c^2即可。∎

1-28. 证明$\log_2(n!) = \Theta(n \log n)$。

证明. 上界：$n! \le n^n$，故$\log_2(n!) \le n \log_2 n = O(n \log n)$。**下界**：$n! \ge \left(\frac{n}{2}\right)^{n/2}$（后一半的$n/2$个因子每个都$\ge n/2$），故$\log_2(n!) \ge \frac{n}{2}\log_2 \frac{n}{2} = \frac{n}{2}\log_2 n - \frac{n}{2} = \Omega(n \log n)$。两界同阶。∎

用处：这正是「基于比较的排序至少要$\Omega(n \log n)$次比较」那条下界

的核心一步。

第 2 章 线性表

一、填空题

2-1. 线性表是n（$n \geq 0$）个数据元素的 ____ 序列。表中除第一个元素外，每个元素有且仅有 一个 ____；除最后一个元素外，每个元素有且仅有一个 ____。

答：**有限**序列；**直接前驱**；**直接后继**。注意「有限」和「有且仅有一个」两处都是得分点；其中把线性表和树、图区分开的是「有且仅有一个」直接前驱／后继（树、图的结点集同样是有穷集合，见第 1.2.1 节）。

2-2. 长度为n 的顺序表，在等概率前提下，插入一个元素平均需要移动 ____ 个元素，删除一个元素平均需要移动 ____ 个元素。

答：插入$\frac{n}{2}$（由$\frac{1}{n+1}\sum_{i=0}^{n}(n-i) = \frac{1}{n+1}\cdot\frac{n(n+1)}{2}$ 得）；删除$\frac{n-1}{2}$（由$\frac{1}{n}\sum_{i=0}^{n-1}(n-1-i)$ 得）。两者相差半个元素，原因是**插入有**$n+1$**个合法位置，删除只有**n**个**。

2-3. 在带头结点的单链表 `L` 中，空表的判定条件是 ____；在不带头结点的单链表中，空表的判定条件是 ____。

答：`L->next == nullptr`；`L == nullptr`。这正是头结点的好处之一：**表头有没有元素，链表对象本身始终存在**，插入删除不必单独处理「改的是表头指针」这种特例。

2-4. 在双链表中删除指针 `p` 所指的结点（`p` 不是首尾结点），需要执行的两条指针操作是 ____ 和 ____。

答：`p->prev->next = p->next;` 和 `p->next->prev = p->prev;`（顺序无所谓，之后再释放 `p`）。若 `p` 可能是首/尾结点，则要各自判空；若用的是**带头结点的循环双链表**，就一条特例都不用判—— 这是循环表的主要优点。

2-5. 设每个数据域大小为s、指针大小为p（忽略头结点、内存对齐及分配开销）。当表的装填程度（实际元素数 / 预分配容量）小于 ____ 时，链表比顺序表省空间。

答：小于$\frac{s}{s+p}$。推导见 2-27。直观地说：顺序表浪费在**没用上的空槽**，链表浪费在**每个结点都要带的指针**，谁更亏取决于表满到什么程度。

2-6. 在循环单链表中，判断指针 `p` 指向尾结点的条件是 ____。

答：`p->next == head`（带头结点时）或 `p->next == first`（不带头结点、`first` 为首结点指针）。对比：普通单链表的判据是 `p->next == nullptr`。

二、选择题

2-7. 在一个长度为n 的顺序表中，在第i 个元素之前插入一个新元素（$1 \leq i \leq n+1$），需要向后移动 ____ 个元素。A. $n-i$　B. $n-i+1$　C. $n-i-1$　D. i

答：B。原来的第i 个到第n 个，共$n-i+1$ 个元素都要后移一格。特

例检查：$i = n + 1$（插在末尾）时移动 0 个，符合；$i = 1$（插在最前）时移动n 个，符合。**做这类题一定要用两个端点验证公式**，凭印象写$n - i$ 是最常见的错法。

2-8. ★ 已知 p 指向带头结点单链表中的某个结点（非尾结点），要在 p 之后插入结点 s，下列语句序列正确的是（ ）。A. `s->next = p->next; p->next = s;` B. `p->next = s; s->next = p->next;` C. `s->next = p; p->next = s;` D. `p->next = s->next; s->next = p;`

答：A。B 的两句顺序颠倒了：先让 `p->next` 指向 s，原来的后继就丢了，第二句 `s->next = p->next` 等于 `s->next = s`，链表当场成环。**口诀：先接新结点的后继，再改前驱的指针**——永远先连「还没人指着的那一端」。

2-9. ★ 设 p 指向单链表中的某结点，下列操作**不能**在$O(1)$ 时间内完成的是（ ）。A. 删除 p 的后继结点 B. 在 p 之后插入一个结点 C. 删除 p 本身（单链表，仅给 p） D. 访问 p 的数据域

答：C。删除 p 本身需要修改其**前驱**的指针，而单链表无法由 p 反查前驱，只能从头找，$O(n)$。补充一个常考的「例外」：若 p **不是尾结点**，可以用「偷天换日」在$O(1)$ 内做到—— 把后继的数据复制到 p，再删掉后继（见 2-21）。但 p 是尾结点时此法失效。

2-10. 下列关于顺序表与链表的比较，**正确**的是（ ）。A. 顺序表查找任意元素都比链表快 B. 链表的插入删除一定比顺序表快 C. 顺序表适合表长稳定、以随机访问为主的场合 D. 链表不需要预先估计表长，所以任何情况下都更优

答：C。A 错：按值查找两者都是$O(n)$，顺序表只在**按位置**访问上有$O(1)$ 的优势。B 错：链表插入删除本身是$O(1)$，但**前提是已经拿到了目标位置的指针**；若还要先定位，定位那一步仍是$O(n)$，整体并不快。D 错：链表的指针开销与不连续存储对缓存不友好，都是代价。

2-11. 带头结点的循环双链表 L 为空的判定条件是（ ）。A. `L == nullptr` B. `L->next == nullptr` C. `L->next == L` D. `L->prev == nullptr`

答：C。循环表里指针永不为空，头结点自己指自己即为空表。

2-12. ★ 在带头结点的循环双链表中，p 指向某个结点，要在 p 之后插入结点 s。下列语句序列中**错误**的是（ ）。A. `s->prev = p; s->next = p->next; p->next->prev = s; p->next = s;` B. `s->next = p->next; p->next->prev = s; s->prev = p; p->next = s;` C. `s->prev = p; s->next = p->next; p->next = s; p->next->prev = s;` D. `s->next = p->next; s->prev = p; p->next = s; s->next->prev = s;`

答：C。四个序列改的是同样四个指针域，s 自己的两个域怎么排都无害，要紧的是另外两句的先后。C 先执行了 `p->next = s`，第四句的 `p->next->prev = s` 就成了 `s->prev = s`：s 的前驱指向它自己，原后继的 prev 却仍指着 p，从后往前走会跳过 s。A、B 都是**先借 `p->next` 修好原后继的 prev，最后才改 `p->next`**；D 的第四句改从 `s->next` 找原后继（第一句已把它存进去），不再依赖 `p->next`，所以放在最后也对。**记**

法：p->next 是通往原后继的唯一一条路，**最后一个改它**；或者先把原后继存进 s->next，之后都从 s->next 走。这和单链表那条口诀（2-8）是同一个道理。循环表的好处是 p 为尾结点时 p->next 是头结点而不是空指针，四句话不需要特判。

2-13. 链表在使用中丢失了头指针，手里只剩表中**某一个**结点的指针 p（不知道它是第几个）。下列哪些结构仍能从 p 出发访问到表中全部结点？（多选）（ ）A. 不带头结点的单链表　B. 循环单链表　C. 双向链表（非循环）　D. 循环双链表

答：B、C、D。A 只能沿 next 往后走，p 之前的结点再也找不回来。B 沿 next 绕一圈回到 p 即走遍全表；D 同理，两个方向都行。**C 最容易漏选**：非循环双链表虽然不能绕圈，但可以先沿 prev 退到表首（prev 为空处），再沿 next 走到表尾，同样访问到全部结点，代价仍是$O(n)$。「只有循环链表才行」这个印象来自单链表的对比；有了 prev，表首就不再是不可回头的起点。

三、辨析与简答

2-14. 辨析：「链表比顺序表节省存储空间。」

答：不一定。分两块看：- **顺序表**要预分配一块连续空间，浪费在**没用上的空槽**上；用满时利用率 100%。- **链表**按需分配，没有空槽，但**每个结点都要额外存一个（双链是两个）指针**。

所以谁省取决于装填程度：当装填程度低于$s/(s+p)$时链表省（s 为数据大小，p 为指针大小），高于它则顺序表省。数据元素越大（s 越大），链表的相对开销越小。此外顺序表还有**缓存局部性**的隐性优势，这一条在渐进分析里看不出来。

2-15. 简答：头结点（附加表头结点）有什么作用？不带头结点会多出哪些特例？

答：头结点是一个不存数据、只占位置的结点，作用是**让「表头」和「表中」在代码里长得一样**。不带头结点时，至少多出三类特例：(1) 在表头插入——要改的是链表对象里的 first 指针，而不是某个结点的 next；(2) 删除首结点——同上；(3) 空表——first 为空，很多循链代码要先判一次空。带了头结点，这三类都退化成普通情形，代价只是一个结点的空间。

2-16. ★ 简答：从**查找、插入删除、空间、适用场合**四个方面比较顺序表和链表。

答：

维度	顺序表	链表
按位置访问	$O(1)$（下标直接算地址）	$O(n)$（必须循链）
按值查找	$O(n)$；有序时可二分$O(\log n)$	$O(n)$；**有序时二分仍需$O(n)$ 时间**

维度	顺序表	链表
插入/删除	$O(n)$（平均移动一半元素）	已知位置指针时$O(1)$，否则定位$O(n)$
空间	需预估容量，有空槽浪费；无指针开销	按需分配，无空槽；每结点多一个指针
适用	表长稳定、以读为主、需要随机访问	表长变化大、频繁在已知位置插删

> 一句话结论：**顺序表把代价付在「改」上，链表把代价付在「找」上。**有序链表可以按折半思路减少比较次数，但循链定位中点的总时间仍是$O(n)$，无法获得数组二分的$O(\log n)$ 时间。

四、算法设计与分析

本组题统一使用如下单链表 ADT（考试时也建议先写这一段，再写算法）：

```
结点 Node
    data    元素值
    next    指向下一结点的指针，尾结点为 nullptr

链表 List（带头结点，head 指向头结点）
    Head()              出口：头结点指针
    Empty()             出口：表空返回真
    MakeNode(x)         入口：元素值 x；出口：新结点指针，data 为 x、next 为空
    FreeNode(p)         入口：结点指针 p；功能：释放该结点
```

2-17. ★ 设计算法把带头结点的单链表**就地逆置**（不申请新结点），并分析时空代价。

> **答**：**基本思想**——把所有结点从原链上摘下来，逐个「头插」回去。头插天然产生逆序。
>
> ```
> Reverse(L)
>     p ← L.head->next        // 摘下整条链，原表先变空
>     L.head->next ← nullptr
>     while p ≠ nullptr
>         q ← p->next         // 先存后继，否则下一句一改就找不回来了
>         p->next ← L.head->next
>         L.head->next ← p
>         p ← q
> ```
>
> **正确性关键**：循环不变式是「`L.head` 之后是已处理结点的逆序，`p` 指向未处理部分的首结点」。每轮都必须**先用 `q` 存好 `p->next`**——这是本题唯一的指针陷阱，漏掉它链表就断在中间。

代价：时间$\Theta(n)$，每个结点只被摘挂一次；空间$\Theta(1)$，只用了 p、q 两个指针。如果允许新建结点或用数组暂存，空间就退化成$\Theta(n)$，本题的「就地」二字要的正是$\Theta(1)$。

2-18. ★ 已知两个带头结点的**递增有序**单链表 A、B，设计算法把它们归并成一个递增有序表 C，要求**不申请新结点**，且结果中相同的值只保留一个。

答：**基本思想**——双指针同步扫描，每次把较小者摘下接到 C 的尾部；相等时只接一个、另一个释放。因为两表都已有序，谁也不必回退。

```
Merge(A, B)
    C ← A.head              // 复用 A 的头结点
    r ← C                   // r 始终指向 C 的当前尾结点
    pa ← A.head->next
    pb ← B.head->next
    C->next ← nullptr
    while pa ≠ nullptr or pb ≠ nullptr
        if pb = nullptr or (pa ≠ nullptr and pa->data ⩽ pb->data)
            t ← pa;  pa ← pa->next
        else
            t ← pb;  pb ← pb->next
        if r ≠ C and r->data = t->data
            FreeNode(t)                 // 也去掉同一输入表内部的重复值
        else
            r->next ← t;  r ← t
            r->next ← nullptr
    FreeNode(B.head)
    return C
```

代价：时间$\Theta(m+n)$，空间$\Theta(1)$。**评分要点**：(1) 尾指针 r 必须有，否则每次接尾都要走一遍，退化成$O(n^2)$；(2) 剩余部分也要逐个检查重复值，不能未经检查就整体接上；(3) 释放重复结点前，必须先保存它的后继并推进相应输入指针。

变式：若要求归并成**递减**有序表，可先按上述算法归并去重，再按 2-17 就地逆置，渐进代价不变——这和 2-17 是同一个技巧。

2-19. 设计算法在带头结点的单链表中查找**倒数第k个**结点，要求只遍历一趟、空间$O(1)$。

答：**基本思想**——双指针拉开固定间距。先让 fast 从首结点走k步，再让 slow 与 fast 同步前进；fast 走到表尾时，两者间距恰为k，slow 即为所求。

```
FindKthFromEnd(L, k)
    if k < 1 then return nullptr
    fast ← L.head->next
    for i ← 1 to k
```

```
        if fast = nullptr then return nullptr      // 表长不足 k
        fast ← fast->next
    slow ← L.head->next
    while fast ≠ nullptr
        fast ← fast->next
        slow ← slow->next
    return slow
```

代价：时间$\Theta(n)$（fast 只走一趟），空间$\Theta(1)$。**边界**：k 大于表长时第一个循环里就返回空；$k = 1$ 时返回尾结点，可代入验证。

2-20. 设计算法判断一个单链表中是否存在环，若存在则返回入环的第一个结点。要求空间$O(1)$。

答：**基本思想**（快慢指针）——slow 每次一步、fast 每次两步。若无环，fast 必先到达表尾；若有环，fast 会在环内追上 slow。

找入环点用如下结论：设表头到入环点距离为a、入环点到相遇点为b、环长为c。相遇时 slow 走了$a + b$，fast 走了$a + b + kc$，又 fast 的路程是 slow 的两倍，故$a + b = kc$，即$a = kc - b$——**从表头和从相遇点同时每次走一步，必在入环点相遇**。

```
FindCycleEntry(L)
    slow ← fast ← L.head->next
    while fast ≠ nullptr and fast->next ≠ nullptr
        slow ← slow->next
        fast ← fast->next->next
        if slow = fast                      // 有环
            p ← L.head->next
            while p ≠ slow
                p ← p->next;  slow ← slow->next
            return p
    return nullptr                           // 无环
```

代价：时间$O(n)$，空间$O(1)$。若允许$O(n)$空间，用散列表记下走过的结点地址也可以，但那就不是本题要考的东西了。

2-21. ★ 已知 p 指向单链表中的某个结点（不是头结点也不是尾结点），要求在$O(1)$时间内 删除 p。给出算法，并说明它为什么对尾结点无效。

答：**基本思想**——既然拿不到前驱，就**让 p 变成它的后继**：把后继的数据抄进 p，再把后继摘掉。链表在逻辑上等价于删掉了 p。

```
DeleteNodeInPlace(p)
    q ← p->next
    p->data ← q->data
    p->next ← q->next
```

```
    FreeNode(q)
```

为什么尾结点不行：尾结点没有后继可供顶替，而删它必须修改前驱的 next，单链表无法在$O(1)$ 内取得前驱，只能从头扫描，$O(n)$。**另一个隐患**：如果外部还有别的指针指着 q，此法会让那个指针变成野指针—— 这是「偷天换日」的真实代价，答题时值得写上一句。

2-22. ★ 双链表中结点 p 的前驱和后继都存在，且它的后继不是尾结点。设计算法把 p 与它的**后继**交换位置，要求只改指针、不动数据域；并说明为什么这些语句的先后次序不能随意调换。

答：**基本思想**——不要去想「一口气把六根指针改对」，写成两步：**先把 p 从链上摘下来，再把它插到原后继之后**。每一步都是已经会的基本操作（摘见 2-4，插见 2-8）。

```
SwapWithNext(p)
    q ← p->next                    // 第一句就得把后继记下来
    // 第一步：摘掉 p，让 q 顶上 p 原来的位置
    p->prev->next ← q
    q->prev ← p->prev
    // 第二步：把 p 插到 q 之后
    p->next ← q->next
    q->next->prev ← p
    q->next ← p
    p->prev ← q
```

正确性：设原来依次是A（p 的前驱）、B（就是 p）、C（p 的后继）、D，目标是$A \leftrightarrow C \leftrightarrow B \leftrightarrow D$。第一步做完是$A \leftrightarrow C$；第二步把$B$ 接在C 与D 之间。六根指针一根不漏。

为什么次序要紧，三条理由：1. `q ← p->next` **必须是第一句**。后面 `p->next` 会被改写，改完就再也找不回原来的后继；2. `q->next->prev ← p` **必须在** `q->next ← p` **之前**。一旦 `q->next` 改成了B，`q->next->prev` 指的就成了B 自己的 prev 域，这句会把 `p->prev` 改成 p 本身，而D 的 prev 根本没被改到；3. 每条语句读到的 `p->prev` / `p->next` 都是**执行到那一刻的值**—— 这类题只能一句一句代进去核，不能整体看。**先接还没人指着的那一端**（2-8 的口诀）在这里同样管用。

代价：时间$O(1)$，空间$O(1)$。

另一种合法做法是直接交换数据域（`swap(p->data, q->data)`），$O(1)$ 且不可能接错。但外部若还有指针指着某个结点并认准它的值，这种做法会骗过它们—— 与 2-21 的「偷天换日」是同一类代价。题目要求「只改指针」时不能用它。

2-23. 设计算法删除顺序表中所有值等于x 的元素，要求时间$O(n)$、空间$O(1)$。

答：**基本思想**——用一个写指针 k 记录「已保留的元素个数」，读指针

扫一遍，不等于x 的就写到 k 处。这是顺序表上的「原地压缩」通用套路。

```
RemoveAll(A, n, x)
    k ← 0
    for i ← 0 to n - 1
        if A[i] ≠ x
            A[k] ← A[i]
            k ← k + 1
    return k                  // 新表长
```

代价：时间$\Theta(n)$（每个元素只读写一次），空间$\Theta(1)$。**对照错法**：每遇到一个x 就把后面的整体前移一格，最坏$O(n^2)$—— 这正是本题要避开的。

2-24. 两个不带头结点的单链表 ha、hb 存放单词，后缀相同的部分**共享结点**，例如 loading 与 being 共享 i → n → g 三个结点（整个结构中无环）。设计算法返回两表共享部分的第一个结点（不相交时返回空），要求时间$O(m+n)$、额外空间$O(1)$，m、n 为两表长度。

答：**基本思想**——共享部分是两表的公共**后缀**，所以要从表尾对齐：先求两表长度，让长表的指针先走$|m-n|$ 步，此后两指针到各自表尾的距离相等，同步前进，第一次**地址**相等的结点就是所求；一直走到空都没相等，说明不相交。

```
Length(h)
    len ← 0
    while h ≠ null do
        len ← len + 1;  h ← h.next
    return len

FirstShared(ha, hb)
    m ← Length(ha);  n ← Length(hb)
    while m > n do                    // 长表先走 |m-n| 步
        ha ← ha.next;  m ← m - 1
    while n > m do
        hb ← hb.next;  n ← n - 1
    while ha ≠ hb do                   // 比地址，不比字母
        ha ← ha.next;  hb ← hb.next
    return ha                         // 不相交时两者同时变成 null
```

正确性：设共享部分长c。对齐后两指针离各自表尾都还有同样多的结点，因此会**同时**走到共享部分的第一个结点；在这之前二者地址不可能相等—— 一旦某处地址相等，其后的结点就完全相同，那一处本身就属于共享部分。**代价**：三趟线性扫描，时间$O(m+n)$，额外空间$O(1)$。**评分要点**：循环条件必须是 ha ≠ hb（结点地址），写成 ha.data ≠

`hb.data` 就错了—— `ab` 与 `cb` 两表末尾都是字母 `b`，却是两个不同的结点。**变式**（不求长度）：两指针各自走完本表后转到对方表头继续走，两人走过的总路程都是$m+n-c$，必在第一个共享结点相遇（不相交时同时到达空）。

五、数学证明

2-25. 证明：在长度为n 的顺序表中等概率地插入一个元素，平均移动元素数为$n/2$。

证明. 插入位置有$n+1$ 个（在下标 0 的元素之前直到下标$n-1$ 的元素之后），等概率即每个为$\frac{1}{n+1}$。插在位置i（$0 \le i \le n$）时需要后移$n-i$ 个元素，故平均移动数

$$E = \frac{1}{n+1}\sum_{i=0}^{n}(n-i) = \frac{1}{n+1}\cdot\frac{n(n+1)}{2} = \frac{n}{2}$$

∎ 同理删除时位置有n 个，平均移动$\frac{1}{n}\sum_{i=0}^{n-1}(n-1-i) = \frac{n-1}{2}$。

2-26. 证明：对有序的单链表，二分检索无法达到$O(\log n)$。

证明. 二分检索的每一步都要求在$O(1)$ 时间内取到当前区间的中间元素。单链表只能从表头沿 `next` 逐个前进，取第k 个元素需要$\Theta(k)$ 时间。因此即使每次都正确地把区间折半，第一次取中点就已经花了$\Theta(n/2)$，总代价至少是$\Theta(n)$，即$\Omega(n)$，不可能是$O(\log n)$。∎

推论：**要获得通常的$O(\log n)$ 二分检索时间，需要高效的随机访问，普通单链表不具备这一条件**。这也是第 10 章讲检索时必须先讨论存储结构的原因。

2-27. 设每个数据元素占s 个存储单元、每个指针占p 个单元，顺序表预分配容量为m、实际存n 个元素。忽略头结点、内存对齐及分配开销，求链表比顺序表省空间的条件。

证明. 顺序表占用$m\cdot s$；单链表占用$n(s+p)$。链表更省当且仅当

$$n(s+p) < m\,s \quad\leftrightarrow\quad \frac{n}{m} < \frac{s}{s+p}$$

即**装填程度n/m 小于$\frac{s}{s+p}$** 时链表省空间。∎

代入感受一下：$s=p$（元素和指针一样大，如 32 位机上存一个 `int`）时阈值是$1/2$，即顺序表填过一半 链表就吃亏了；$s=10p$（元素是个大记录）时阈值约0.91，装填程度低于约 0.91 时链表更省空间。**结论：元素越大，链表的指针开销越不值一提**。

第 3 章 栈与队列

一、填空题

3-1. 栈是限定只能在 ____ 进行插入和删除的线性表，其特点是 ____；队列是限定在 ____ 插入、在 ____ 删除的线性表，其特点是 ____。

> **答**：**表的一端（栈顶）**；**后进先出（LIFO）**；**表的一端（队尾）**；**另一端（队头）**；**先进先出（FIFO）**。两者都是**限制存取点的线性表**（第 3.3.3 节把它们与双栈、超队列、超栈一并看作受限的双端队列）—— 把线性表的存取点限制得越死，能提供的运算越少，但实现可以越高效。

3-2. ★ 设循环队列的数组容量为M（牺牲一格，最多存$M-1$ 个元素），`front` 指向队头元素、`rear` 指向队尾元素的下一个位置。则：队空的条件是 ____，队满的条件是 ____，队中元素个数为 ____。

> **答**：队空 `front == rear`；队满 `(rear + 1) % M == front`；元素个数 `(rear - front + M) % M`。元素个数公式里的 `+ M` 是为了让 `rear < front`（已绕回）时余数非负，**不能省**。

3-3. ★ 设有n 个元素按$1, 2, \cdots, n$ 的顺序依次入栈（中途可任意出栈），则可能得到的不同出栈序列共有 ____ 种。

> **答**：$\frac{1}{n+1}C_{2n}^{n}$，即第n 个 **Catalan 数**。$n = 1, 2, 3, 4, 5$ 时依次是$1, 2, 5, 14, 42$。考试里最常出的是$n = 3$（5 种）和$n = 4$（14 种），建议把这两个数背下来。推导见 3-30。

3-4. 顺序栈用数组 `data[0..m-1]` 存储，若约定 `top` 指向**栈顶元素**，则空栈时 `top` 为 ____，入栈的两步是 ____；若约定 `top` 指向**栈顶元素的上一个空位**，则空栈时 `top` 为 ____。

> **答**：`-1`；先 `++top` 再 `data[top] = x`（先加后存）；`0`（此时入栈是先存后加：`data[top++] = x`）。两种约定都对，**但一份代码里必须自始至终只用一种**；混用是顺序栈最常见的越界来源。

3-5. ★ 递归调用时，系统运行栈中每个活动记录（栈帧）主要保存三样东西：____、____、____。

> **答**：**返回地址**（这一层返回后从哪继续执行）、**实在参数、本层的局部变量**（以及需要保护的寄存器现场）。手工用栈消除递归时要压进栈的，就是这三样里**本层还要再用到**的那部分—— 不再用到的可以不存，这正是优化的空间（见第 3.1.5 节背包问题的「优化」一段）。

3-6. 中缀表达式 `A + B * (C - D) / E` 的后缀形式是 ____。

> **答**：`A B C D - * E / +`。检验办法：后缀式里运算符出现的**先后顺序**必须与实际求值顺序一致—— 这里是先 `-`，再 `*`，再 `/`，最后 `+`，与中缀式的优先级和括号完全吻合。

3-7. 双端队列允许在两端插入和删除。把$1, 2, 3, 4, 5$ 依次插入一个初始为空的双端队列（每次可以插到队头或队尾，**中途不删除**），全部插完后从队头到队尾

读出，可能得到 ___ 种不同的序列；对一般的n（$n \geq 1$），答案是 ___。

答：**16**；2^{n-1}。第 1 个元素插到哪一端结果都一样（队列原本为空），之后每个元素各有「队头/队尾」两种选择，共2^{n-1} 种插法。不同插法得到的序列也一定不同：最后插入的n 必在某一端，看它在队头还是队尾就能反推最后一次的选择；剥掉它再看$n-1$..... 逐个还原，插法与序列一一对应。别答成2^n（把第 1 个元素的两种插法算成了两个结果），也别和出栈序列的 Catalan 数（3-3）混淆—— 那里允许中途出栈，这里只进不出。

二、选择题

3-8. ★ 设入栈序列为 `1,2,3,4`，下列**不可能**是出栈序列的是（　）。A. `1,2,3,4` B. `4,3,2,1`　C. `3,1,2,4`　D. `2,4,3,1`

答：C。用判定法则最快：**合法出栈序列中不存在下标**$i<j<k$ **使**$p_j<p_k<p_i$（即不含「312」型）。C 中取$p_1=3,p_2=1,p_3=2$，满足$1<2<3$，故非法。手工验证：要先输出 3，则 1、2 必定还压在栈里且 2 在 1 之上，于是它们只能按 `2,1` 的顺序出来，绝不可能是 `1,2`。证明见 3-31。

3-9. 容量为M 的循环队列（牺牲一格），`front = 3`、`rear = 0`、$M=8$，则队中元素个数是（　）。A. 3　B. 5　C. 0　D. 8

答：B。$(0-3+8) \bmod 8 = 5$。若误用 `rear - front` 会得到-3，这正是公式里 `+ M` 的作用。

3-10. 下列关于循环队列的叙述正确的是（　）。A. 循环队列解决的是队列「真溢出」问题 B. 循环队列必须牺牲一个存储单元，没有别的办法 C. 循环队列解决的是「假溢出」问题，即队尾到达数组末端而前端仍有空闲 D. 循环队列中 `front == rear` 一定表示队满

答：C。A 错：数组真的满了照样溢出，循环队列只是不再浪费前端空闲。B 错：另设一个计数器 `count` 或一个「上次操作是入队还是出队」的布尔标志同样可以区分空与满，牺牲一格只是最常用的做法（见 3-15）。D 错：牺牲一格的约定下 `front == rear` 表示**队空**。

3-11. 后缀表达式 `3 4 + 5 2 - *` 的值是（　）。A. 21　B. 17　C. 35　D. 9

答：A。求值过程：`3`、`4` 入栈；遇 `+` 弹出两个算$3+4=7$ 入栈；`5`、`2` 入栈；遇 `-` 算$5-2=3$ 入栈；遇 `*` 算$7\times 3=21$。**注意减法、除法的操作数顺序**：先弹出的是右操作数，后弹出的是左操作数，写成 `右 = pop(); 左 = pop(); 结果 = 左 - 右`，弄反就会算成$2-5$。

3-12. 用两个栈S_1、S_2 模拟一个队列，下列说法正确的是（　）。A. 每次出队都要把S_1 全部倒进S_2，故出队一定是$O(n)$ B. 入队$O(1)$，出队的**摊还**代价是$O(1)$ C. 无法用两个栈模拟队列 D. 需要三个栈才能模拟

答：B。入队压S_1；出队时若S_2 非空直接弹，空了才把S_1 整个倒进S_2。每个元素**一生只被倒一次**，n 次操作总代价$O(n)$，故摊还$O(1)$。A 的

错误在于把「单次最坏」当成了「每次都这样」。

三、辨析与简答

3-13. 辨析：「栈和队列都是线性表，所以它们的存储结构也只能和线性表一样。」

答：前半句对，后半句说法不准。栈和队列确实是**运算受限的线性表**，顺序和链接两大类 存储方式也都适用。但**受限本身带来了线性表没有的实现手法**：顺序队列必须把数组看成环（循环队列），否则出队后前端的空间收不回来；链式队列为了$O(1)$ 入队必须额外保存队尾指针。这些都是限制存取点之后才出现的设计。

3-14. ★ 辨析：「链式栈没有容量上限、不会上溢，所以总是优于顺序栈。」从时间和空间两方面比较二者。

答：错。两者的入栈、出栈、取栈顶都是$O(1)$，差别在常数和空间上：

	顺序栈	链式栈
入栈、出栈	$O(1)$；栈满扩容时要搬移一次$O(n)$，按倍增扩容（算法 3.3）摊还仍是$O(1)$	$O(1)$，但每次都要申请或释放一个结点，常数较大
空间	预分配一整块，**没用上的空槽**是浪费；元素本身没有额外开销	按需分配，没有空槽；**每个元素多一个指针**，另有分配器的管理开销
容量	定长时可能上溢，须预估容量或扩容	只受可用内存总量限制
其他	连续存储，缓存友好；两个栈可共享一个数组，栈底分设两端、相向增长	栈顶放在链表**表头**，入栈出栈都只动表头，不需要尾指针

结论：元素个数大致能预估时选顺序栈（省指针、速度快）；个数波动大、难以预估，或多个栈要共用空闲内存时选链式栈。「不会上溢」确实是链式栈的长处，但它是用每个元素一个指针、每次操作一次内存分配换来的。

3-15. ★ 简答：为什么循环队列中 `front == rear` 既可能表示空、也可能表示满？列举至少两种区分办法并比较。

答：**根本原因是状态数对不上。**M 个位置的数组，队列的元素个数有$0, 1, \cdots, M$ 共$M+1$ 种状态；而固定 `front` 后 `rear` 只有M 种取值，鸽笼原理保证必有两种状态撞在一起—— 撞上的恰是「空」和「满」。

三种区分办法：

办法	判空	判满	代价
牺牲一格	`front == rear`	`(rear+1)%M == front`	少存一个元素，判据最简
另设计数器 `count`	`count == 0`	`count == M`	能存满M个，每次入出队都要维护计数
另设标志位 `tag`	`front==rear && tag==0`	`front==rear && tag==1`	能存满，判据稍绕

教材与多数教学实现取第一种。

3-16. ★ 简答：什么是递归？用栈消除递归的一般方法是什么？哪些递归可以不用栈就改成循环？

答：**递归**是函数（直接或间接）调用自身的求解方式，靠系统运行栈自动保存每一层的活动记录。

用栈消除递归的一般方法（见第 3.1.5 节）：1. 设计一个「活动记录」结构，存放本层**返回后还要用到**的信息：参数、局部变量、以及「返回后该从哪一步继续」的**返回点编号**；2. 开一个该结构的栈，把初始调用的参数压栈；3. 主循环：弹出栈顶记录，按返回点编号跳到相应分支执行；遇到「递归调用」的地方，先把**本层剩下要做的事**压回栈（改好返回点编号），再把**子问题的参数**压栈，然后回到主循环；4. 栈空时算法结束。

可以不用栈的情形：- **尾递归**：递归调用是函数的最后一个动作，返回后无事可做，直接改成「更新参数 + 循环」即可，空间从$O(\text{深度})$降到$O(1)$；- **单向线性递归**（如求阶乘、求链表长度）：本质上就是尾递归的变形，可直接写成循环；- **（部分消除）两处调用互不依赖、其后无事可做的双分支递归**（如快排）：不能完全去掉栈，但可把其中一侧改成循环——先递归处理**较小的那一侧**、对较大的一侧用循环迭代，栈深由$O(n)$降到$O(\log n)$。

按一般方法模拟多分支递归时：像汉诺塔、二叉树遍历，返回之后还有后续动作，通常用显式栈保存续接状态；这不排除利用问题特性设计其他非递归算法（如 Morris 遍历）。

3-17. 简答：递归改成非递归以后，代价一定变小吗？

答：不一定。时间上，消除了函数调用的开销（保存现场、跳转），常数变小；但**渐进阶不变**，该指数还是指数（斐波那契的朴素递归改成显式栈仍是指数级，真正救它的是记忆化，不是消递归）。空间上，递归吃的是**运行栈**（通常只有几 MB，深了就崩），显式栈吃的是**堆**（可大得多），所以改写的主要收益是**把「栈溢出直接崩溃」换成「可控的容量检查」**，而不是省空间。

四、算法设计与分析

3-18. ★ 写出栈和队列的 ADT，并给出顺序栈 Push、Pop 的实现（含溢出处理）。

答：

```
ADT Stack（元素类型 T）
    Push(x)      入口：元素 x；功能：x 成为新栈顶；栈满时报上溢
    Pop()        出口：原栈顶元素；功能：删除栈顶；栈空时报下溢
    Top()        出口：栈顶元素（不删除）；栈空时报错
    Empty()      出口：栈空返回真
    Clear()      功能：置为空栈

ADT Queue（元素类型 T）
    EnQueue(x)   入口：元素 x；功能：x 接到队尾；队满时报上溢
    DeQueue()    出口：原队头元素；功能：删除队头；队空时报下溢
    Front()      出口：队头元素（不删除）
    Empty()      出口：队空返回真
```

顺序栈（`data[0..m-1]`，`top` 指向栈顶元素，空栈 `top = -1`）：

```
Push(x)
    if top = m - 1 then 报上溢；返回          // 必须先查，再写
    top ← top + 1
    data[top] ← x

Pop()
    if top = -1 then 报下溢；返回
    x ← data[top]
    top ← top - 1
    return x
```

评分要点：溢出检查必须在写数组**之前**；Pop 要先取值再减 top。原书代码 3.2 这两处的次序是对的；它的毛病在别处（数据成员 `top` 与成员函数 `top()` 重名、缺拷贝控制导致二次释放、容器内输出），见本书第 3.1.2 节与「与原书的对照」。

3-19. ★ 写出循环队列的完整实现（判空、判满、入队、出队、求长度），采用「牺牲一格」方案。

答：设数组 `data[0..M-1]`，`front` 指向队头元素，`rear` 指向队尾元素的**下一个**位置。初始 `front = rear = 0`，最多存$M-1$ 个元素。

```
Empty()          return front = rear
Full()           return (rear + 1) mod M = front
Length()         return (rear - front + M) mod M

EnQueue(x)
```

```
    if Full() then 报上溢; 返回
    data[rear] ← x
    rear ← (rear + 1) mod M        // 关键: 一切下标推进都要取模

DeQueue()
    if Empty() then 报下溢; 返回
    x ← data[front]
    front ← (front + 1) mod M
    return x
```

代价：五个运算全是$O(1)$，空间$O(M)$。**评分要点**：(1) `mod M` 一处都不能漏；(2) 判满判空的条件要与 `rear` 的含义一致—— 若改成「`rear` 指向队尾元素」，则空为 `front == (rear+1) % M`，两套约定不能混；(3) 要说明牺牲一格的理由（见 3-15）。

变式（用计数器，可存满M 个）：另设 `count`，`Empty()` 为 `count == 0`、`Full()` 为 `count == M`，入队 `count++`、出队 `count--`，`Length()` 直接返回 `count`。

3-20. ★ 设入栈序列固定为$1, 2, \cdots, n$，给定一个$1..n$ 的排列p，设计算法判断p 是否为一个合法的出栈序列。分析代价。

答：**基本思想**——直接用一个栈**模拟**：按$1..n$ 的顺序依次入栈，每次入栈后，只要栈顶等于p 中下一个待匹配的元素就一直弹出。全部入栈完毕后若p 也匹配完，则合法。

```
IsValidPopSequence(p, n)
    S ← 空栈
    j ← 0                                  // p 中下一个待匹配的下标
    for i ← 1 to n
        S.Push(i)
        while not S.Empty() and S.Top() = p[j]
            S.Pop()
            j ← j + 1
    return S.Empty()
```

正确性：模拟过程中，每个时刻「什么时候必须弹」是唯一确定的——若栈顶恰是当前需要输出的元素，继续压入新元素会挡住它；要维持指定的输出次序，必须先弹出它。因此贪心地「能弹就弹」不会错过任何解。

代价：时间$\Theta(n)$（每个元素入栈一次、出栈至多一次），空间$\Theta(n)$。

另一种写法（判定法则）：检查是否存在$i < j < k$ 使$p_j < p_k < p_i$，存在即非法。朴素实现$O(n^3)$，用单调栈可优化到$O(n)$——但考试里直接写模拟更稳妥。

3-21. ★ 写出中缀表达式转后缀表达式的算法，并用它把 `A + B * C - (D / E +`

F) * G 转成后缀式。

答：**基本思想**——操作数直接输出；运算符进一个栈，进栈前先把栈里**优先级不低于它**的 运算符全部弹出并输出；左括号无条件进栈、并在栈内视为最低优先级；右括号则一直弹到左括号为止。

```
InfixToPostfix(E)
    S ← 空栈（存运算符）
    for 每个记号 t in E
        if t 是操作数         then 输出 t
        else if t = '('       then S.Push(t)
        else if t = ')'       then
            while S.Top() ≠ '('  输出 S.Pop()
            S.Pop()                          // 丢掉那个左括号
        else                                 // t 是双目运算符
            while not S.Empty() and S.Top() ≠ '(' and 优先级 (S.Top()) ≥
↪  优先级 (t)
                输出 S.Pop()
            S.Push(t)
    while not S.Empty()  输出 S.Pop()
```

优先级：*、/ 高于 +、-；括号内的运算符先算，靠「左括号在栈内最低」自动实现。（若要支持右结合的乘方 ^，把条件里的 ≥ 对 ^ 改成 >。）

本题答案：A B C * + D E / F + G * -

推演（左列是当前记号，右列是栈内容，栈顶在右）：

记号	输出	栈
A	A	
+		+
B	B	+
*		+ *
C	C	+ *
-	* +	-
(		- (
D	D	- (
/		- (/
E	E	- (/
+	/	- (+
F	F	- (+
)	+	-
*		- *
G	G	- *

记号	输出	栈
结束		* -

代价：时间$\Theta(n)$，空间$O(n)$。

3-22. 写出后缀表达式求值算法，并求 `A B C * + D E / F + G * -` 在 `A=2, B=3, C=4, D=8, E=2, F=1, G=3` 时的值。

答：**基本思想**——一个操作数栈，扫描后缀式：操作数入栈；遇到双目运算符就弹出两个，**先弹的是右操作数**，算完把结果压回。扫描结束时栈中唯一的元素就是答案。

```
EvalPostfix(E)
    S ← 空栈
    for 每个记号 t in E
        if t 是操作数 then S.Push(值 (t))
        else
            b ← S.Pop()        // 右操作数
            a ← S.Pop()        // 左操作数
            S.Push(a t b)
    return S.Pop()
```

求值：$B*C=12$；$A+12=14$；$D/E=4$；$4+F=5$；$5*G=15$；$14-15=-1$。**答案**-1。

代价：时间$\Theta(n)$，空间$O(n)$。**评分要点**：必须写明两个操作数的弹出顺序，否则减法除法会算反。

3-23. ★ 用栈把下面的递归算法改写成非递归算法，并说明栈中保存了什么。

```
Hanoi(n, A, B, C)          // 把 n 个盘子从 A 经 B 移到 C
    if n = 0 then return
    Hanoi(n - 1, A, C, B)
    输出 "A → C"
    Hanoi(n - 1, B, A, C)
```

答：本题有**两处递归调用**，且第一处返回后还要做事，本题按要求用显式栈模拟。

栈中保存什么：一个活动记录(n, A, B, C, tag)，其中 `tag` 是**返回点编号**——记录「这一层做到哪一步了」：`tag=0` 表示三步都没做，`tag=1` 表示第一处递归已回来、该输出并展开第二处递归。参数n, A, B, C必须存，因为返回后还要用它们组织第二个子问题。

```
HanoiIter(n, A, B, C)
    S ← 空栈
    S.Push((n, A, B, C, 0))
```

```
    while not S.Empty()
        (n, A, B, C, tag) ← S.Pop()
        if n = 0 then continue
        if tag = 0
            S.Push((n, A, B, C, 1))        // 本层还没做完，改好返回点压回去
            S.Push((n - 1, A, C, B, 0))    // 再压子问题，保证它先被处理
        else
            输出 "A → C"（用本记录里的 A、C）
            S.Push((n - 1, B, A, C, 0))
```

关键点：压栈顺序必须是「先压本层的后续、再压子问题」——栈是后进先出，这样才能保证子问题先于本层后续被执行，与递归的次序一致。

代价：时间$\Theta(2^n)$（与递归版相同，消递归不改变渐进阶），栈深$O(n)$，但用的是堆上的显式栈，容量可查可控，不会像运行栈那样直接崩。

对照：尾递归可以不用栈。例如 `Fact(n) = if n ⩽ 1 then 1 else n * Fact(n-1)` 虽然不是严格尾递归，但它是**单向线性递归**，可直接写成 `r ← 1; for i ← 2 to n: r ← r * i`，空间$O(1)$；而 `Gcd(a,b) = if b = 0 then a else Gcd(b, a mod b)` 是标准尾递归，直接改成 `while b ≠ 0: (a, b) ← (b, a mod b)`。**判断标准就一条：递归调用返回之后还有没有事要做**。

3-24. 设计算法用**一个数组**实现两个栈，使得只有在整个数组真正用完时才发生上溢。

答：**基本思想——两个栈从数组的两端相向增长**（共享栈）。栈 0 的栈顶 `top0` 从-1 向右增，栈 1 的栈顶 `top1` 从m 向左减。

```
Push(i, x)                          // i = 0 或 1
    if top0 + 1 = top1 then 报上溢；返回        // 两栈顶相邻即真正满了
    if i = 0 then top0 ← top0 + 1;  data[top0] ← x
    else          top1 ← top1 - 1;  data[top1] ← x

Pop(i)
    if i = 0
        if top0 = -1 then 报下溢；返回
        x ← data[top0];  top0 ← top0 - 1
    else
        if top1 = m then 报下溢；返回
        x ← data[top1];  top1 ← top1 + 1
    return x
```

为什么这样最省：若把数组对半分给两个栈，一个栈满了另一个还空着也得报溢出；相向增长则让两个栈**共享同一片剩余空间**，只有总和用尽才溢出。**注意**：「相向增长」只对**两个**栈有效；三个以上的栈无法

都靠端点，通常改为分区（某区满而数组仍有空闲时整体挪动各区，代价是搬移元素）或用链式栈。

3-25. 设计算法判断一个字符序列是否为回文（正读反读相同），要求用栈和队列各一个。

答：**基本思想**——同一串字符同时进栈和进队列。出栈得到的是**逆序**，出队得到的是**正序**；逐个比较，全部相等即回文。

```
IsPalindrome(s)
    S ← 空栈；  Q ← 空队列
    for 每个字符 c in s
        S.Push(c);  Q.EnQueue(c)
    while not S.Empty()
        if S.Pop() ≠ Q.DeQueue() then return false
    return true
```

代价：时间$\Theta(n)$，空间$\Theta(n)$。只用栈也能做（把前一半压栈，再与后一半比），实际存储的字符约减半（渐进空间仍为$\Theta(n)$），但要处理奇偶长度的中点，本题按题目要求用栈加队列，写法最干净。

3-26. ★ 算法填空：用两个队列实现一个栈。队列只提供 `push(x)` 入队、`pop()` 队首出队、`front()` 取队首、`empty()` 判空（**没有求长度的运算**）。下面这一版让 `push` 保持$O(1)$，把代价放在 `pop` 上；调用 `pop` 的前提是栈非空。补全五个空。

```
class MyStack
    queue q1, q2                        // 元素平时都在 q1 里

    push(x)
        ____(1)____

    pop()                               // 弹出最后压入的那个
        while ____(2)____
            t ← q1.front();  q1.pop()
            if ____(3)____  then  r ← t        // 走到最后一个，它就是答案
            else                  q2.push(t)
        ____(4)____                     // 让 q1 重新成为存元素的那个
        return r

    empty()
        return ____(5)____
```

答：(1) `q1.push(x)`　(2) `not q1.empty()`　(3) `q1.empty()`　(4) `swap(q1, q2)`　(5) `q1.empty()`

思路：栈要后进先出，而队列只能从队首取——最后压入的那个恰好在队尾。于是 `pop` 把 `q1` 里的元素一个个倒进 `q2`，**倒到队列空了的那一次取出的就是队尾元素**，正是要弹的栈顶；其余元素已经按原序进了

q2，交换两个队列的身份即可。第 (3) 空是本题的关键：题目没给求长度的运算，所以「是不是最后一个」只能靠 **取走它之后 q1 是否变空**来判断。

代价：push $O(1)$、empty $O(1)$；pop $O(n)$。top 可以写成「r ← pop(); push(r); return r」——弹出来再压回去，它又成了最后压入的那个，仍在栈顶，代价同样是$O(n)$。

另一版把代价挪到 push：新元素先单独进 q2，再把 q1 里原有的元素按原序全部倒到它后面，最后交换。这样 q1 的队首永远是最后压入的那个，pop/top 都是$O(1)$，而 push 变成$O(n)$。**两版是对称的：总有一端要付**$O(n)$，选哪一版取决于哪个操作更频繁。

和 3-12（两个栈实现队列）对照着记：

	两个栈做队列（3-12）	两个队列做栈（本题）
重的一端	出队要倒栈，但**摊还**$O(1)$	本题两版：总有一端$O(n)$，**摊不掉**
为什么不同	每个元素一**生只被倒一次**	每做一次重操作就要把**全部**元素倒一遍

最后一行才是真正的考点：**摊还分析成不成立，取决于「每个元素被搬动的总次数」是否有界**，不取决于「单次最坏是多少」。

3-27. 字符串用「重复次数 + 方括号」压缩：k[s] 表示 s 重复 k 次（k 为正整数，可以是多位数），允许嵌套，例如 3[d2[c]] 解压为 dccdccdcc。设计算法把压缩串（保证合法，只含小写字母、数字和方括号）解压，写出基本思想与伪代码，给出 2[ab3[c]]d 的解压结果及过程中栈的变化，并分析代价。

答：**基本思想**——方括号的嵌套就是括号匹配，用栈保存「外层现场」。设两个栈：numStk 存每层的重复次数，strStk 存进入这一层之前外层已经拼好的串；cur 是当前层正在拼的串，k 累计多位数字。读到 [就把 (k, cur) 压栈、开一个新层；读到] 就把本层结果重复后接回外层。

```
Decode(s)
    cur ← "";  k ← 0
    for each ch in s do
        if ch 是数字 then k ← k × 10 + ch 的数值
        else if ch = '[' then
            numStk.Push(k);  strStk.Push(cur)     // 保存外层现场
            cur ← "";  k ← 0
        else if ch = ']' then
            t ← numStk.Pop()
            cur ← strStk.Pop() + (cur 重复 t 次)   // 回到外层，接上本层结果
        else cur ← cur + ch
```

```
    return cur
```

`2[ab3[c]]d` 的过程（栈底在左）：

读到	动作	strStk	numStk	cur
`2`	$k=2$	空	空	`""`
`[`	压入$(2,\ \texttt{""})$	`""`	2	`""`
`a b`	接到 cur	`""`	2	ab
`3 [`	压入$(3,\ \texttt{ab})$	`""`, ab	2, 3	`""`
`c`	接到 cur	`""`, ab	2, 3	c
`]`	弹出 3 与 ab	`""`	2	abccc
`]`	弹出 2 与`""`	空	空	abcccabccc
`d`	接到 cur	空	空	abcccabcccd

结果为 **abcccabcccd**。**代价**：扫描$O(|s|)$，每个字符至多入栈出栈一次；但每次遇到 `]` 都要生成一个长度不超过最终长度L 的串，故最坏$O(|s|+bL)$，b 为括号对数；嵌套不深时接近$O(|s|+L)$——输出本身就有L 个字符，这是下界。也可以只用一个字符栈：`]` 时向下弹出字母直到 `[`，再弹出数字，把重复后的串整段压回。

3-28. 设计一个栈，除 `Push`、`Pop`、`Top` 之外再支持 `GetMin()`（返回栈中最小元素），四个操作都是$O(1)$。**(1)** 描述内部数据结构，写出 `Push`、`Pop`、`GetMin`；**(2)** 若辅助栈只在新元素**不大于**当前最小时才压入，说明比较为什么必须用 $\leqslant$ 而不能用 `<`，用 `Push(3)`、`Push(3)`、`Pop()` 这一串操作说明。

答：**(1)** 两个栈：`S` 存全部元素；`M` 存「历史最小值」，自底向上非增，**栈顶就是 `S` 当前的最小值**。

```
Push(x)
    S.Push(x)
    if M.Empty() or x ⩽ M.Top() then M.Push(x)

Pop()
    if S.Empty() then 报下溢；返回
    x ← S.Pop()
    if x = M.Top() then M.Pop()        // 弹走的正是当前最小值的那一份
    return x

GetMin()
    if M.Empty() then 报错；返回
    return M.Top()
```

不变式：`M.Top()` 等于 `S` 中的最小值。`Push` 时新元素若不大于旧最小就成为新最小，压入 `M`；`Pop` 时只有弹出的元素等于当前最小才动 `M`，弹完

后 M 的新栈顶是它入栈之前的最小值—— 比它晚入栈的元素此刻都已出栈，所以这正是剩余元素的最小值。**代价**：四个操作都是$O(1)$；额外空间最坏$O(n)$（输入递减时每个元素都进 M）。

(2) 用 < 时：Push(3) 后 S=[3]、M=[3]；第二个 Push(3) 因$3 < 3$不成立而不进 M；Pop() 弹出的 3 等于 M.Top()，于是 M 被弹空——可 S 里还剩一个 3，GetMin 出错。用 ⩽ 时 M=[3,3]，弹一次还剩 [3]，正确。**重复出现的最小值，每一份都要在 M 里记账**。变式：S 的每一格存二元组（元素, 压入时的最小值），写法最简单，但空间固定翻倍。

3-29. 直方图由n 根宽为 1 的柱子组成, 高度依次为$h_0, \cdots, h_{n-1}$（非负整数）。求直方图中能放下的最大矩形面积。(1) 写出$O(n)$ 算法的基本思想和伪代码；(2) 对$h = [2, 4, 3, 5, 1, 3, 3]$ 写出栈的变化过程和答案。

答：**(1) 基本思想**——最大矩形的高一定等于它覆盖的某根柱子i 的高h_i；以h_i 为高时，矩形向左、向右各自延伸到**第一根比它矮的柱子**为止。用一个**单调栈**存下标，对应高度自底向上递增：扫到i 时，凡栈顶高度$\geq h_i$ 的都出栈—— 对出栈的t 来说，i 就是右边第一根更矮的，出栈后的新栈顶就是左边第一根更矮的，它的矩形宽度就此确定。末尾补一根高度为 0 的哨兵，把栈清空。

```
MaxRect(h[0..n-1])
    h[n] ← 0                                  // 哨兵：保证最后全部出栈
    best ← 0;  S ← 空栈                       // S 存下标，对应高度自底向上递增
    for i ← 0 to n do
        while not S.Empty() and h[S.Top()] ⩾ h[i] do
            t ← S.Pop()
            left ← (S.Empty() ? -1 : S.Top())     // 左边第一根更矮的位置
            best ← max(best, h[t] × (i - left - 1))
        S.Push(i)
    return best
```

代价：每个下标恰好入栈、出栈各一次, 时间$O(n)$, 栈空间$O(n)$。朴素做法枚举左右端点再求区间最小，是$O(n^2)$。

(2) 栈中写下标，括号里是该柱高度：

i	h_i	出栈并结算（高 × 宽 = 面积）	栈（底→顶）	当前最
0	2	—	0(2)	0
1	4	—	0(2), 1(4)	0
2	3	1: $4 \times 1 = 4$	0(2), 2(3)	4
3	5	—	0(2), 2(3), 3(5)	4
4	1	3: $5 \times 1 = 5$; 2: $3 \times 3 = 9$; 0: $2 \times 4 = 8$	4(1)	9
5	3	—	4(1), 5(3)	9
6	3	5: $3 \times 1 = 3$	4(1), 6(3)	9

i	h_i	出栈并结算（高 × 宽 = 面积）	栈（底→顶）	当前最
7（哨兵）	0	6：$3\times2=6$；4：$1\times7=7$	7(0)	9

答案 **9**：以$h_2=3$ 为高、覆盖下标 1～3 的矩形。注意$i=6$ 时等高的 5 号先出栈，它算出的宽度 1 偏小，但真正的宽度会在 6 号出栈时补算回来（3×2），所以 $\geqslant$ 与 $>$ 两种写法都对，只是等高柱子由谁结算不同。

五、数学证明

3-30. ★ 证明：n 个元素按$1,2,\cdots,n$ 顺序入栈，可能的出栈序列共$C_n=\frac{1}{n+1}C_{2n}^{n}$ 种。

证明. 设C_n 为答案，$C_0=1$。考察**元素 1 在输出序列中的位置**：设 1 是第$i+1$ 个被输出的（$0\le i\le n-1$）。1 是最先入栈的，从入栈到出栈它一直压在栈底。1 出栈之前，入栈序列已经走到某个j，即$2,\cdots,j$ 都已入栈（入栈顺序固定，所以是从 2 起的连续一段）；它们入栈时 1 还在栈里，因此都**压在 1 的上面**。1 要出栈必须先到栈顶，所以这批元素**必须全部先出栈**；而 1 出栈前也只有它们入过栈。于是「在 1 前面输出的元素」恰好就是「在 1 出栈前入栈的元素」，个数为i，即$j=i+1$，集合是$\{2,\cdots,i+1\}$。这i 个元素在 1 弹出前自成一局，栈底的 1 不影响它们的进出，所以它们内部的相对次序是一个规模为i 的同类子问题；而排在 1 之后输出的$n-1-i$ 个元素是$\{i+2,\cdots,n\}$，构成规模$n-1-i$ 的同类子问题。两部分互不影响，故

$$C_n=\sum_{i=0}^{n-1}C_i\,C_{n-1-i}$$

这正是 Catalan 数的递推式。下面用**反射法**求出它的闭式。

由 3-33(2)，合法出栈序列与合法 I/O 操作序列一一对应，所以只需数：由n 个 I、n 个 O 组成、且**任何前缀里 I 不少于 O** 的序列有多少个。思路是「全部减去非法的」。从左到右读序列，I 记$+1$、O 记-1 累加成一个**计数器**（即栈中元素个数）：合法 $\Leftrightarrow$ 计数器从不小于 0 且最后回到 0；非法 $\Leftrightarrow$ 中途**跌到过**-1（对空栈出栈）。

① **全部.** 只要求 I、O 各n 个、暂不考虑前缀条件：在$2n$ 个位置中选n 个放 I，共C_{2n}^{n} 个序列。

② **把非法序列变成别的序列.** 计数器每步只变±1，所以它第一次变负时恰好等于-1，设这是第k 步（即「第一个 O 比 I 多 1 的前缀」）。前k 位不动，把第k 位**之后**的字母**逐位取反**—— 每个 I 改成 O、每个 O 改成 I（不是交换位置，也不是倒序）。数一下个数：- 前k 位有a 个 I、$a+1$

个 0（$k=2a+1$）；- 后面原有$n-a$ 个 I、$n-a-1$ 个 0，取反后变成$n-a-1$ 个 I、$n-a$ 个 0；- 合计 I 有$a+(n-a-1)=n-1$ 个，0 有$(a+1)+(n-a)=n+1$ 个。

所以每个非法序列都变成一个「$n-1$ 个 I、$n+1$ 个 0」的序列。画成折线，就是把第k 步之后的折线**沿**$y=-1$ **做镜像**，终点从 0 变成-2——「反射法」由此得名。

③ **这个变换可以倒回去.** 任取「$n-1$ 个 I、$n+1$ 个 0」的序列，它的计数器从 0 出发、终点是-2，每步只走±1，中途必定**经过**-1；在第一次到达-1 的那一步之后再逐位取反，就得到一个「n 个 I、n 个 0」的非法序列。它恰好是原来那个：取反只改第k 步之后的部分，前k 步一字未动，所以两个方向上「第一次到达-1」都在同一个k；同一位置之后取反两次等于没取反。

于是非法序列与「$n-1$ 个 I、$n+1$ 个 0」的序列一一对应，后者是在$2n$ 个位置中选$n-1$ 个放 I，所以非法序列恰有C_{2n}^{n-1} 个。

例（$n=3$）：非法序列 IOOIIO，计数器依次为$1,0,-1$，在第$k=3$ 步第一次到-1。

位置	1	2	3	4	5	6
原序列	I	O	O	I	I	O
计数器	1	0	**−1**			
第 3 位后逐位取反	I	O	O	O	O	I

得到 IOOOOI（2 个 I、4 个 0）；反过来，IOOOOI 也在第 3 步第一次到-1，把后面的 OOI 取反回 IIO，正是 IOOIIO。

因此

$$C_n = C_{2n}^{n} - C_{2n}^{n-1} = C_{2n}^{n}\left(1-\frac{n}{n+1}\right) = \frac{1}{n+1}C_{2n}^{n}$$

其中用到$C_{2n}^{n-1} = \frac{(2n)!}{(n-1)!\,(n+1)!} = \frac{n}{n+1}C_{2n}^{n}$。∎

为什么不用生成函数：由递推式也能得$C(x) = 1 + x\,C(x)^2$，解出$C(x)=\frac{1-\sqrt{1-4x}}{2x}$ 再展开$\sqrt{1-4x}$ 取系数，结果相同，但要用广义二项式定理、还要说明根号取负号，考场上更容易算错。反射法得到的$C_{2n}^{n}-C_{2n}^{n-1}$ 也正是教材第 12 章给出的形式。

验证：$C_1=1$，$C_2=2$，$C_3=5$，$C_4=14$，$C_5=42$。**同一个数还数着另外两样东西**：n 对括号的合法配对方案数、n 个结点的不同二叉树形态数（第 5 章会再用一次）——三者之间是一一对应的。

3-31. ★ 证明：排列$p_1p_2\cdots p_n$ 是入栈序列$1,2,\cdots,n$ 的合法出栈序列，当且仅当**不存在下标**$i<j<k$ **使**$p_j<p_k<p_i$。

证明. 注意入栈序列递增，因此**任何时刻栈内元素自底向上是递增的**，这是全部论证的基础。

必要性（合法⇒ 无此模式）：反设存在$i<j<k$使$p_j<p_k<p_i$。p_j、p_k都小于p_i，而入栈序列递增，所以二者都在p_i **之前入栈**；又二者都在p_i **之后输出**，故p_i出栈的那一刻，p_j与p_k都还压在栈里。栈内自底向上递增，而$p_k>p_j$，所以此刻p_k在p_j **之上**，于是p_k必先于p_j出栈，即$k<j$——与$j<k$矛盾。

充分性（无此模式⇒ 合法）：对n作归纳。$n\le 1$显然。设$p_1=m$。把其余元素分成「小值」$\{1,\cdots,m-1\}$和「大值」$\{m+1,\cdots,n\}$。1. **小值在p中必按递减次序出现**：否则存在$a<b$且a排在b前面，取$i=1$、$j=\mathrm{pos}(a)$、$k=\mathrm{pos}(b)$，即有$p_j=a<p_k=b<m=p_i$，正是被禁的模式。2. **大值构成的子序列仍不含该模式**（模式的定义只看相对次序，子序列自然继承），由归纳假设，它是$m+1,\cdots,n$的一个合法出栈序列。3. **两者可以自由交错**：先把$1,\cdots,m$依次入栈并弹出m，此时栈内自底向上是$1,2,\cdots,m-1$，正好能按递减次序逐个弹出，与小值的要求一致；需要输出某个大值时，把当前已入栈到的位置继续往后压、按第 2 步的方案操作即可—— 大值全部压在小值之上，用完就弹走，不影响下面小值的次序。这里还要说明一点：**轮到输出某个小值s时，栈里不会还压着没输出的大值**。否则设压着的是L，它已入栈说明此前输出过某个比它更大的大值L'（正是为了输出L'才把L压进去），于是p中依次出现L', s, L且$s<L<L'$，又是被禁的模式。所以小值要出栈时它一定在栈顶。

因此整个p可以被实际的入栈出栈过程生成，即合法。∎

使用建议：选择题用判定法则一眼看出答案，大题仍写模拟算法——后者更好写、也更好改错。

3-32. 证明循环队列中「元素个数$=(rear-front+M)\bmod M$」对 `rear` 是否绕回都成立。

证明. 分两种情形。(1) `front` ⩽ `rear`（未绕回）：队列占据$[front,rear)$，元素个数为$rear-front\ge 0$；此时$rear-front+M\in[M,2M)$，对M取模正好还原为$rear-front$。(2) `front` > `rear`（已绕回）：队列占据$[front,M)\cup[0,rear)$，元素个数为$(M-front)+rear=rear-front+M$；此时$rear-front\in(-M,0)$，故$rear-front+M\in(0,M)$，取模后不变，正是元素个数。两种情形都得到同一个表达式。∎

顺带说明牺牲一格的必要性：若允许存满M个，则满时$rear=front$，公式算出0，与空队列无法区分——这就是 3-15 里那个鸽笼论证的算式版本。

3-33. ★ 用 `I` 表示一次入栈、`O` 表示一次出栈。一个从空栈开始、到空栈结束的操作过程可以写成一个由 `I`、`O` 组成的序列（例如 `IOIO` 合法，`IOOI` 非法）。(1) 给出序

列合法的**充要条件**并说明理由；(2) 证明：对元素互不相同的输入，两个**不同的**合法操作序列不可能产生**相同的**输出序列；(3) 长度为$2n$ 的合法序列共有多少个?

> **答**：**(1)充要条件**：I 与 O 的总数相等，且**任何一个前缀里 I 的个数都不少于 O 的个数**。
>
> 理由：把栈中元素个数看成一个从 0 出发的计数器，I 使它加一、O 使它减一。「终态是空栈」等价于全部增减之和为零，即两者总数相等；「中途从不对空栈做出栈」等价于**任何时刻计数器非负**，即任何前缀里 I 不少于 O。反之两条都满足时，整个过程每一步都合法且末尾归零。把 I 看成左括号、O 看成右括号，这正是「n 对括号合法配对」的条件。
>
> **务必与 3-8、3-20、3-31 区分**：那几道判的是**输出序列**合不合法（判据是「不存在$i<j<k$ 使$p_j<p_k<p_i$」），这道判的是**操作序列**合不合法。两者都由 Catalan 数计数，但判据完全不同，别张冠李戴。
>
> **(2) 证明.** 设$T_1\neq T_2$ 是两个合法操作序列（长度均为$2n$，即全部n 个输入都入栈并出栈），取它们**第一个不同的位置** $r+1$。前r 步完全相同，所以此刻栈的内容、已输出的内容、下一个待入栈的元素都相同：整个过程是**确定性**的——I 把下一个输入压栈、O 弹出栈顶并输出，没有选择余地，从同一初始状态执行同一串操作，每一步之后的状态必然一样（对步数归纳即得）。特别地，入栈顺序固定，第m 次 I 压入的一定是m；前r 步里 I 的次数相同，设为m，故两者下一个待入栈的都是$m+1$。不妨设T_1 这一步做 I、T_2 做 O。设此刻栈顶是a、下一个待入栈的是b（T_2 能做 O 说明栈非空，a 存在；T_1 能做 I 说明还有元素没入栈，b 存在；b 尚未入栈故$a\neq b$）。- 在T_2 里：a **此刻就被输出**，而b 还没入栈，故输出中a **在**b **之前**；- 在T_1 里：b 被压到a 上面，要弹a 必须先弹b，故b **在**a **之前**。
>
> 两个合法序列最终都把所有元素输出，故a、b 都出现在各自的输出里，而它们的相对次序相反，输出序列必不相同。∎
>
> **(3)** $C_n=\frac{1}{n+1}C_{2n}^{n}$，即第$n$ 个 Catalan 数（由下文的双射与 3-30 得出；也可直接把操作序列看作n 对括号的合法配对来数）。
>
> 把三问连起来看：(2) 说明「操作序列$\to$ 输出序列」这个映射是**单射**；而合法输出序列按定义就是某个合法操作序列的输出，映射是满射，所以它是**双射**；再由 3-30 知合法输出序列恰有C_n 个，操作序列也就恰有C_n 个。于是**合法操作序列、合法出栈序列、n 个结点的二叉树形态，三者一一对应**，个数都是C_n。

第 4 章 字符串

一、填空题

4-1. 字符串是由 ____ 组成的有限序列，串中字符的个数称为串的 ____。不含任何字符的串称为 ____，它的长度为 ____；由一个或多个空格组成的串称为 ____，它的长度 ____。

> **答**：**零个或多个字符**；**长度**；**空串**；**0**；**空格串**；**不为 0**（等于空格个数）。「空串」和「空格串」是最爱考的一组辨析：空格是一个货真价实的字符，其中每个空格字符的 ASCII 编码是 `0x20`，串长不是 0。

4-2. 串 `S = "abcaabcab"` 的长度是 ____，它的所有**真前缀**中最长且同时也是真后缀的那一个是 ____。

> **答**：**9**；**`"ab"`**（长度 2）。验证：真前缀 `a`、`ab`、`abc`…；真后缀 `b`、`ab`、`cab`…；`"ab"` 两边都有，长度 3 到 8 的前后缀均不相等（尤其前 5 个字符是`"abcaa"`，后 5 个是`"abcab"`）。这个「最长公共真前后缀」正是特征向量的定义核心。

4-3. 设目标串长n、模式串长m。朴素（Brute Force）模式匹配算法的最坏时间代价是 ____，KMP 算法的时间代价是 ____，其中求特征向量需 ____、匹配过程需 ____。

> **答**：$O(n \cdot m)$；$O(n+m)$；$O(m)$；$O(n)$。朴素算法达到最坏的典型输入：T 为n 个 `a`、P 为$m-1$ 个 `a` 再加一个 `b`—— 每个起点都要比到最后一个字符才失配。

4-4. ★ 模式 `P = "abaabcaba"`（下标从 0）的**未优化**特征向量（next 数组，约定 `next[0] = -1`）是 ____。

> **答**：`-1 0 0 1 1 2 0 1 2`。逐位算（`next[i]` = `P[0..i-1]` 的最长公共真前后缀长度）：
>
> | i | `P[i]` | 前缀 `P[0..i-1]` | 最长公共真前后缀 | `next[i]` |
> |---|---|---|---|---|
> | 0 | a | (空) | — | −1 |
> | 1 | b | a | 0 | 0 |
> | 2 | a | ab | 0 | 0 |
> | 3 | a | aba | 1 (`a`) | 1 |
> | 4 | b | abaa | 1 (`a`) | 1 |
> | 5 | c | abaab | 2 (`ab`) | 2 |
> | 6 | a | abaabc | 0 | 0 |
> | 7 | b | abaabca | 1 (`a`) | 1 |
> | 8 | a | abaabcab | 2 (`ab`) | 2 |

4-5. ★ 模式 `P = "ababab"`（下标从 0）的**未优化** next 数组是 ____，**优化后**的 next

数组是 ____。

答：未优化 `-1 0 0 1 2 3`；优化后 `-1 0 -1 0 -1 0`。

未优化逐位算最长公共真前后缀：`a` → 0，`ab` → 0，`aba` → 1，`abab` → 2，`ababa` → 3。优化按 4-14 的规则：`P[2]='a'` 与 `P[next[2]=0]='a'` 相同 → 借 `next[0] = -1`；`P[3]='b'` 与 `P[1]='b'` 相同 → 借 `next[1] = 0`；后面交替下去。

周期串使两种 next 的差别格外明显：未优化是$-1, 0, 0, 1, 2, 3$ 步步升高，优化后只在-1 和0 之间跳。原因是这个模式从下标 2 起每一位都与它的回退目标同字符，退过去必定再失配一次，于是一路被借到底。

顺手做一下 4-21：整串`"ababab"` 的最长公共真前后缀是`"abab"`，长度$f = 4$，最小循环节长$6 - 4 = 2$，而$6 \bmod 2 = 0$——所以`"ababab"` 正是`"ab"` 重复三次。**注意这一步必须用未优化的那一行**，优化版的数值已经不是「最长公共真前后缀长度」了。

4-6. ★ 设非空串S 长n，其**整串**的最长公共真前后缀长度为f。则S 的最小循环节长度是 ____；S 由该循环节至少重复两次构成的充要条件是 ____。

答:最小循环节长度$n-f$;充要条件是$n \bmod (n-f) = 0$(且$f > 0$)。例：`s = "abcabcabc"`，$n = 9$、$f = 6$，循环节长3，$9 \bmod 3 = 0$，故由`"abc"` 重复 3 次构成。反例：`s = "abcab"`，$n = 5$、$f = 2$，循环节长3，$5 \bmod 3 \neq 0$，说明 s 是某个周期为 3 的串的**前缀**，但不是整数次重复。证明见 4-26。

4-7. 把一个串分割成若干子串，使每个子串都是回文。`s = "abacdcabba"` 最少需要分割 ____ 次，一种最优分法是 ____。

答:**2** 次;`aba | cdc | abba`。一刀都不切不行(整串不是回文);只切一刀，要求某个前缀和剩余后缀同时是回文，逐一检查都不成立。**一般做法是动态规划**：设$f[i]$ 为前i 个字符最少能分成几段，$f[0] = 0$，$f[i] = \min\{\, f[j] + 1 | 0 \le j < i,\ s[j..i-1] \text{ 是回文}\}$，答案是$f[n] - 1$ 次;先用$O(n^2)$ 的表记下每个区间是否回文，总代价$O(n^2)$。**贪心「每次切下最长的回文前缀」是错的**：`abaab` 贪心得 `aba | a | b`（2 次），最优却是 `a | baab`（1 次）。

二、选择题

4-8. 下列关于串的叙述**错误**的是（　）。A. 空串是任意串的子串 B. 空格串的长度等于其中空格的个数 C. 长度为n 的串，按起止位置计数（相同内容重复计）的非空子串个数为$\frac{n(n+1)}{2}$ D. 两个串相等，只要求它们包含的字符集合相同

答：D。串相等的定义是**长度相等且对应位置的字符逐一相同**，`"abc"` 与`"cab"` 字符集合相同却不相等。A 对（空串是任何串的子串，也是任何串的前缀和后缀);B 对(空格是真字符);C 对：以下标i ($0 \le i < n$)

处字符开头的非空子串有$n-i$个，求和得$\frac{n(n+1)}{2}$；若相同内容重复计、另把空串统一算作一种，则是$\frac{n(n+1)}{2}+1$个。

4-9. ★ 模式 `P = "aaab"` 的未优化 next 数组是（　）。A. `-1 0 1 2`　B. `-1 0 0 0`　C. `0 1 2 3`　D. `-1 1 2 3`

答：A。`next[1]` 看`"a"` $\to 0$；`next[2]` 看`"aa"` $\to 1$（a）；`next[3]` 看`"aaa"` $\to 2$（aa）。注意**优化版**则是 `-1 -1 -1 2`：因为 `P[1]=P[next[1]]=a`、`P[2]=P[next[2]]=a`，都要继续向前借用。

4-10. ★ 用 KMP 匹配时，若模式在位置i处失配且 `next[i] = -1`，则下一步应当（　）。A. 模式右移一位，目标下标回退到本趟起点加一 B. 模式整体右移$i+1$位，目标下标前进一格，从 `P[0]` 重新比 C. 目标下标回退到0 D. 匹配失败，算法结束

答：B。`next[i] = -1` 的含义是「模式已经退无可退」，实现里写成 `i = next[i]` 后 `i` 变成-1，下一轮的 `i == -1` 分支会让 `i` 与 `j` 同时加一——效果正是**目标下标前进一格、模式从头开始**。**KMP 的灵魂就是目标下标 j 只增不减**，A 和 C 都违背了这一条。

4-11. 用朴素算法在$T=$ `"aaaaaaaaab"`（9 个 `a` 加 1 个 `b`，长 10）中匹配$P=$ `"aaab"`（长 4），字符比较的总次数是（　）。A. 10　B. 24　C. 28　D. 40

答：C。起点只能取$s=0,1,\cdots,6$，共 7 个。$s=0..5$ 这 6 趟都在比到第 4 个字符时发现 `P[3]='b'` 与 `T[s+3]='a'` 失配，各比 4 次，计 24 次；$s=6$ 时 `T[6..9] = "aaab"` 完全匹配，又比 4 次。合计$24+4=28$。这正是朴素算法的最坏输入形态：**每趟都要比到模式的最后一个字符才失配**，代价$\Theta(n\cdot m)$。换成 KMP，目标下标一路不回退，比较次数是$O(n+m)$量级。**答题提醒**：这类题按「起点个数 × 每趟比较次数」老实数，先算清楚起点范围是$0..n-m$。

三、辨析与简答

4-12. 辨析：「串是一种特殊的线性表，所以线性表的所有算法都能直接用在串上。」

答：前半句对，后半句不对。串确实是数据元素限定为字符的线性表，但它的**基本运算的粒度不同**：线性表以「单个元素」为操作对象（插入一个元素、删除一个元素），串则以「**子串**」为操作对象（求子串、串联接、模式匹配、替换）。正因为这个差别，串才值得单独成章——模式匹配这类问题在一般线性表里根本不存在。

4-13. ★ 简答：KMP 算法为什么能做到$O(n+m)$？它省掉的究竟是什么？

答：1. **省掉的是目标串的回退**。朴素算法失配后把目标下标退回本趟起点的下一个位置，同一个字符可能被反复比较m次；KMP 里**目标下标 j 只增不减**，全程至多前进n次。2. **凭什么可以不回退**：失配之前已经匹配上的那一段 `P[0..i-1]` 是已知的，它的最长公共真前后缀告诉我们「模式最少要右移多少位才可能重新对上」，移过去之后开头那一

段**必然相同、不必再比**。3. **代价分析**：见 4-25 的严格证明——让 `i` 减小的只有 `i = next[i]`，而每次至少减 1，所以它的执行次数不超过 `i` 增加的次数，总循环次数$\le 2n+1$。4. 再加上预处理 next 的$O(m)$，总计$O(n+m)$。而且**同一个模式的 next 只需算一次**，可以对多个目标串复用。

4-14. ★ 简答：未优化的 next 与优化后的 next 有什么区别？为什么优化后会出现-1？

答：- **未优化版**：`next[i]` = `P[0..i-1]` 的最长公共真前后缀长度（`next[0] = -1`）。它只回答「退到哪里**有可能**继续匹配」。- **优化版**：若 `P[i] == P[next[i]]`，那么退到 `next[i]` 之后比的还是同一个字符、注定再次失配，不如一**步到位借用** `next[next[i]]`，即 `next[i] ← (P[i] == P[k]) ? next[k] : k`（k 为未优化值）。- -1 **从哪来**：当 `next[i]` 一路借用到 0 号位、而 `P[i] == P[0]` 时，就借用 `next[0] = -1`，含义是「模式整体移过这个位置，目标下标前进一格」。- **两者的等价性**：优化只是跳过了若干必然失败的比较，**失配后的最终落点相同**，匹配结果一字不差；但**两个数组的数值不同**，考试写哪一个要看题目对特征向量的定义。本书正文印的是原书算法 4.7 的优化版（例如 `next("ababd") = -1 0 -1 0 2`）。- **重要提醒**：做「最小循环节」「周期」这类题**必须用未优化版**—— 优化版已经不再表示「最长公共真前后缀长度」，拿去套周期公式会算错。

4-15. 简答：原书的朴素匹配算法返回值有一处差 1 的错误，说说这类错误为什么难被发现。

答：模式匹配成功时，正确的返回值应是**本趟起点**在目标串中的下标；匹配成功时目标下标 `j` 已走到匹配段末尾之后，起点应是 `j - pLen`；原书写成 `return (j - pLen + 1)`，多加了 1，所有成功匹配的返回值一律大 1（朴素匹配【算法 4.6】与 KMP【算法 4.8】是同一处错，见第 4.3.1 节）。难发现的原因有三：(1) 匹配**失败**的路径返回值正确，测试若只测失败用例就发现不了；(2) 匹配成功时返回的位置只差 1，肉眼看输出「像是对的」；(3) 书上演示匹配的例子只逐趟画了比较过程、**没有写出返回值**，错误在书里没有机会暴露。**教训：边界值要单独出测试用例**——本书对这一处专门断言了返回位置与 `std::string::find` 一致。

4-16. 简答：实现 `my_memcpy(void* dst, const void* src, size_t n)` 时，若两块内存**重叠**会发生什么？分 `src < dst < src + n` 与 `dst < src < dst + n` 两种情形说明正确做法。

答：逐字节从前往后拷贝时，**每写一个字节都可能覆盖一个还没读到的源字节**，是否出事取决于两块的相对位置。

- **`src < dst < src + n`（目标落在源的后半段）**：从前往后拷会先写 `dst[0]`，而 `dst[0]` 正是某个尚未读到的 `src[k]`（k = `dst` − `src`）；等读到它时，读到的已经是刚写进去的内容。典型症状是**后半段变**

成前半段的重复。正确做法：从最后一个字节开始，倒着拷——每次写的位置都在「还没读的区域」之后。

- **`dst < src < dst + n`（目标落在源的前半段）**：反过来，**倒着拷会出错，必须从前往后拷**。
- **两块不重叠**：哪个方向都对。

所以处理重叠的通用办法就是**先比较两个地址、再决定拷贝方向**，这正是 `memmove` 做的事。

标准库的分工要记住：`memcpy`、`strcpy` 对重叠区间的行为是**未定义**的（正因为不做这道判断，它们才可以更快）；要允许重叠必须用 `memmove`。「未定义」不等于「一定出错」——短串、特定编译器下它可能碰巧是对的，这类 bug 换个优化档就冒出来，属于最难查的一种。

这一条和本章的 `String` 类直接相关：**自赋值**（`s = s`）、以及「把一个串的后半段拷到它自己的前半段」这类操作，共同点都是源与目标是同一块（或相互重叠的）内存：前者先释放再拷会读到已释放的空间，后者拷错方向会读到刚写进去的内容。

4-17. ★ 设 `S = "abaabab"`（下标从 0）。(1) 按字典序列出 `S` 的全部**不同**非空子串，并说明共有多少个；(2) 求 `S` 的未优化 next 数组和优化后的 next 数组。

答：(1) 共 **19** 个。按字典序（`a < b`；一个串是另一个串的前缀时，短的在前）：

a、aa、aab、aaba、aabab、ab、aba、abaa、abaab、abaaba、abaabab、abab、b、ba、baa、baab、baaba、baabab、bab

不重不漏的办法：先按长度分组把不同子串找齐，再统一排序。

长度	不同子串	个数
1	a b	2
2	aa ab ba（bb 没有出现）	3
3	aab aba baa bab	4
4	aaba abaa abab baab	4
5	aabab abaab baaba	3
6	abaaba baabab	2
7	abaabab	1

合计 19 个；按起止位置计数是 $\frac{7\times 8}{2} = 28$ 个（4-8），其中 9 个是重复的。排序时**从左到右逐位比**：`abaabab` 排在 `abab` 前面（第 4 位 `a < b`），所有以 `b` 开头的串都排在以 `a` 开头的串之后，不管长短。

(2) 未优化 `-1 0 0 1 1 2 3`；优化后 `-1 0 -1 1 0 -1 3`。

i	P[i]	未优化k	P[k]	是否相同	优化后
0	a	-1	—	—	-1
1	b	0	a	否	0
2	a	0	a	是，借 next[0]	-1
3	a	1	b	否	1
4	b	1	b	是，借 next[1]	0
5	a	2	a	是，借 next[2]	-1
6	b	3	a	否	3

易错点在$i=5$：借的是**已经优化过的** next[2]，即-1；误借未优化的 next[2]=0 会写成 0。规则见 4-14，逐位从左往右算，借用时前面各位都已是优化值。

四、算法设计与分析

4-18. ★ 写出求未优化特征向量（next 数组）的算法，并说明其时间代价。

答：**基本思想**——next[i] 的计算复用 next[i-1]：设已知 P[0..i-2] 的最长公共真前后缀长度为k。若 P[i-1] == P[k]，则新的长度是$k+1$；否则把k 退到 next[k] 再比，一直退到匹配或退到-1 为止。

```
GetNext(P, m)
    if m = 0 then return 空数组
    next[0] ← -1
    i ← 0;  k ← -1
    while i < m - 1
        if k = -1 or P[i] = P[k]
            i ← i + 1;  k ← k + 1
            next[i] ← k                  // 未优化版：直接记下长度
        else
            k ← next[k]                  // 沿已算好的特征值回退
```

优化版只改一行：把 next[i] ← k 换成 if P[i] = P[k] then next[i] ← next[k] else next[i] ← k。

时间代价$O(m)$：i 只增不减，至多走m 次；k 每次 k ← next[k] 至少减 1，而 k 总共只增加了至多m 次，故回退的总次数也不超过m。与 4-25 是同一套摊还论证。

4-19. ★ 写出 KMP 匹配算法（利用已求出的 next 向量），并分析其代价。

答：

```
KMP(T, n, P, m, next)
    i ← 0                  // 模式下标，可以退到 -1
    j ← 0                  // 目标下标，只增不减
```

```
while i < m and j < n
    if i = -1 or T[j] = P[i]
        i ← i + 1;  j ← j + 1
    else
        i ← next[i]              // 唯一与朴素算法不同的一句
if i ⩾ m then return j - m       // 匹配成功，返回本趟起点
else return -1                   // 失败
```

代价：匹配$O(n)$、预处理$O(m)$，合计$O(n+m)$；额外空间$O(m)$（next 数组）。**评分要点**：(1) `i = -1` 这个分支不能少，它负责「模式退无可退时目标前进一格」；(2) 成功时返回 `j - m` 而不是 `j`；(3) 要说明 `j` 不回退。

4-20. ★ 对模式 `P = "abaabcac"`，求它的未优化 next 与优化 next；并用 KMP 在T = `"abaabaabcac"` 中匹配，写出每一趟的失配位置与右移位数。

答：

i	0	1	2	3	4	5	6	7
`P[i]`	a	b	a	a	b	c	a	c
未优化 `next[i]`	−1	0	0	1	1	2	0	1
优化 `next[i]`	−1	0	−1	1	0	2	−1	1

（优化版逐位核对：`P[2]='a'` 与 `P[next[2]=0]='a'` 相同 → 借 `next[0] = -1`；`P[4]='b'` 与 `P[1]='b'` 相同 → 借 `next[1] = 0`；`P[6]='a'` 与 `P[0]='a'` 相同 → 借—1。）

匹配过程（用未优化 next，T = `abaabaabcac`，下标从 0）：

趟	对齐位置	比到哪里	结果
1	$P[0]$ 对$T[0]$	`T[5]='a'` 与 `P[5]='c'` 失配	`next[5]=2`，模式右移$5-2=3$位
2	$P[2]$ 对$T[5]$	从 `P[2]` 起逐字符相同，一直比到 `P[7]='c'` 与 `T[10]='c'`	**匹配成功**，此时$j=11$，起点$11-8=3$

全程目标下标j 从 0 单调走到 11，**一次都没有回退**——这正是与朴素算法的差别。朴素算法在同样输入上要退回$T[1]$、$T[2]$ 重来，比较次数明显更多。

4-21. ★ 变种应用：给定串S（长n），设计$O(n)$ 算法求它的**最小循环节**，并判断S 是否由该循环节至少重复两次构成。

答：**基本思想**——把特征向量**算到下标**n（即整串S 的最长公共真前后缀长度f），则最小循环节长度为$n-f$；S 是整数次重复当且仅当$n \bmod (n-f)=0$ 且$f>0$。

```
MinPeriod(S, n)
    if n = 0 then return (无正周期, 不是至少重复两次)
    用 GetNext 求 S 的未优化特征向量，并把它多算一项 f = next[n]
    p ← n - f                          // 最小循环节长度
    if f > 0 and n mod p = 0
        return (p, n / p)              // 循环节长 p，重复 n/p 次
    else
        return (p, 不是整数次重复)
```

两个必须写清的细节：1. **要用未优化版 next**——优化版的数值已不是「最长公共真前后缀长度」，套公式必错；2. 教材实现的特征向量只算到下标$n-1$，本题要的$f=next[n]$ **得单独再推一步**（把 `GetNext` 的数组开到$n+1$ 项、循环条件从 `i < m - 1` 放宽到 `i < m` 即可）。

代价：$O(n)$ 时间、$O(n)$ 空间。**例**：`S = "abcabcabc"`，$f=6$，$p=3$，$9 \bmod 3=0 \to$ 循环节`"abc"`，重复 3 次。`S = "abaabaaba"`，$n=9$、$f=6$、$p=3$，$9 \bmod 3=0 \to$ 循环节`"aba"`，重复 3 次。`S = "abcab"`，$f=2$、$p=3$，$5 \bmod 3 \neq 0 \to$ 不是整数次重复。证明见 4-26。

4-22. 变种应用：设计算法判断串B 是否为串A 的**循环移位**（如`"cdeab"` 是`"abcde"` 的循环移位），要求$O(|A|+|B|)$。

答：**基本思想**——B 是A 的循环移位，当且仅当$|A|=|B|$ 且B **是**$A+A$ **的子串**。因为把A 接成两份之后，A 的每一种循环移位都恰好作为一个长度$|A|$ 的窗口出现在其中。

```
IsRotation(A, B)
    if |A| ≠ |B| then return false
    return KMP(A + A, B) ≠ -1
```

代价：拼接$O(|A|)$，KMP 匹配$O(2|A|+|B|)=O(|A|+|B|)$。**易错点**：长度相等这一条必须先判——否则 `B = "ab"`、`A = "abab"` 会被误判为真。

同类变式（都是「换个目标串或换个特征定义」）：- 统计P 在T 中出现的次数（**允许重叠**）：KMP 匹配成功后不要从头开始，而是令 `i ← next[m]` 继续（next 须按 4-21 的办法多算到下标m），仍是$O(n+m)$；- 求S 的**最长回文前缀**：对 `S + '#' + reverse(S)` 求特征向量，整串的最长公共真前后缀长度即为答案（`#` 是不在字母表中的分隔符，防止跨越中线）；- 求A 的最长的、在B 中作为子串出现过的**前缀**：对 `A + '#' + B` 求未优化特征向量，取 `next[|A|+2]` 到 `next[|A|+1+|B|]` 的最大值（每个值是「A 的某个前缀恰好在B 的此处结束」的最大长度；须多算

整串的那一项；B 为空时返回 0）。注意它不是「两串的最长公共前缀」——后者只要从头逐位比到第一个不同处，$O(\min(|A|,|B|))$，用不着 KMP。

4-23. 变种应用：若某个子串形如AA（非空串A 紧接着再出现一次，如`"bcbc"`、`"aa"`），称它为**平方子串**。设计算法判断串S（长n）是否含平方子串；若含，返回**起点最小**的那个的起点和$|A|$。要求最坏$O(n^2)$。

答：**基本思想**——按半长$L=1,2,\cdots,\lfloor n/2\rfloor$ 分别找。L 固定时，起点为i、长$2L$ 的子串是平方，当且仅当$S[t]=S[t+L]$ 对$t=i,\cdots,i+L-1$ 都成立，也就是「$S[t]$ 与$S[t+L]$ 相等」**连续成立L 次**。扫一趟、记下连续相等的次数 `run` 即可。

```
FirstSquare(S, n)
    best ← 无;  bestL ← 0
    for L ← 1 to ⌊n/2⌋
        run ← 0
        for j ← 0 to n - L - 1
            if S[j] = S[j + L] then run ← run + 1 else run ← 0
            if run = L                    // S[j-L+1..j+L] 是平方子串
                if best = 无 or j - L + 1 < best
                    best ← j - L + 1;  bestL ← L
                跳出内层循环                      // 同一个 L，往后的起点只会更大
    return (best, bestL)                      // best = 无 表示没有平方子串
```

正确性：`run` 是「以j 结尾、连续满足$S[t]=S[t+L]$ 的t 的个数」。对固定的L，`run = L` 第一次成立时j 最小，起点$j-L+1$ 也就最小；再在所有L 之间取最小起点（起点相同时先找到的L 较小，用 `<` 比较不会被替换），就是全局起点最小的平方子串。

代价：外层$\lfloor n/2\rfloor$ 趟，每趟$O(n)$，共$O(n^2)$ 时间、$O(1)$ 额外空间。朴素做法是枚举起点和L 再逐字比较，要$O(n^3)$；本法把「逐字比较L 个字符」摊进了一趟扫描。

易错点：不能在第一个找到平方子串的L 上直接返回。例如 `S = "abcbcaa"`：$L=1$ 先找到`"aa"`，起点 5；但$L=2$ 的`"bcbc"` 起点是 1，答案是起点 1、$|A|=2$。另一个例子 `S = "abacabad"` 不含平方子串，返回「无」。

别和 4-21 混淆：4-21 问的是**整串**是否由某个串重复至少两次构成，起点固定为 0，用 next 一次求出；本题的平方子串起点和长度都不定，所以改为按L 逐趟扫描。

4-24. 对长为n 的串S，记$S[0..L-1]$ 为长L 的前缀，$\mathrm{occ}(L)$ 为它在S 中出现的次数（允许重叠）。设计尽可能快的算法求$\max_{1\le L\le n} L\cdot\mathrm{occ}(L)$，写出思想与代价，并对 `S = "abababab"` 给出各个$L$ 的$\mathrm{occ}(L)$ 和答案。

答：**基本思想**——长L 的前缀在某处结束（即$S[j-L..j-1]=$

$S[0..L-1]$），当且仅当L是「以该处结尾的最长前缀」的某个公共真前后缀长度，或就是它本身。而$S[0..j-1]$的全部公共真前后缀长度恰好是 next 链$\texttt{next}[j]$, $\texttt{next}[\texttt{next}[j]], \cdots$（**未优化** next，4-4 的定义，并且要**算到**$j=n$）。于是先给每个前缀记 1 次（它在下标 0 处的那次出现），再**自长向短**把次数沿 next 链往下传：

```
MaxPrefixProduct(S[0..n-1])
    求未优化的 next[0..n]：next[j] = S[0..j-1] 的最长公共真前后缀长度，
↪   next[0] = -1
    for L ← 1 to n do cnt[L] ← 1
    for j ← n downto 1 do                    // 长的先定稿，再传给它的 next
        if next[j] > 0 then cnt[next[j]] ← cnt[next[j]] + cnt[j]
    return max{ L × cnt[L] | 1 ⩽ L ⩽ n }
```

倒序的理由：$\texttt{next}[j]<j$，所以处理到j时，所有比它长、会往它身上传次数的前缀都已处理完，`cnt[j]` 已是终值。**代价**：求 next $O(n)$，传递$O(n)$，求最大$O(n)$，合计$O(n)$；枚举每个前缀再逐位匹配是$O(n^2)$以上。

`S = "abababab"`：$\texttt{next}[0..8]=-1,0,0,1,2,3,4,5,6$。

L	1	2	3	4	5	6	7	8
$\text{occ}(L)$	4	4	3	3	2	2	1	1
$L\cdot\text{occ}(L)$	4	8	9	**12**	10	**12**	7	8

最大值 **12**，在$L=4$（abab 出现 3 次）和$L=6$（ababab 出现 2 次）同时取到。用优化 next 会把链剪断，次数就传丢了——与 4-20 提醒的求周期是同一个坑。

五、数学证明

4-25. ★ 证明 KMP 匹配过程的时间代价为$O(n)$（n为目标串长度）。

证明. 考察循环体的两个分支。- **分支一**（`i = -1` 或 `T[j] = P[i]`）：`i` 与 `j` 同时加一。由于 `j` 只增不减且$j<n$，这个分支至多执行n次。- **分支二**（`i ← next[i]`）：由 next 的定义有 `next[i] < i`，所以每执行一次 `i` **至少减 1**。

设分支一执行了a次、分支二执行了b次。`i` 的初值为 0，分支一使它总共增加a，分支二使它总共减少至少b，而 `i` 的终值≥ -1，故

$$0+a-b\geq -1\Rightarrow b\leq a+1\leq n+1$$

于是循环体总执行次数$a+b\leq 2n+1=O(n)$，每次循环体是常数工作量。∎

加上求 next 的$O(m)$（同样的摊还论证），KMP 整体为$O(n+m)$。**这条论证的模板值得记住**：「一个量只增不减 + 另一个量每次至少减 1」⇒ 线性摊还。

4-26. ★ 设非空串S 长n，f 是S 的最长公共真前后缀长度。证明：$p=n-f$ 是S 的最小周期，且S 能写成某个串至少重复两次（$S=w^k$，$k\ge 2$）当且仅当$f>0$ 且p 整除n。

证明. 称p 是S 的**周期**，若对一切$0\le i<n-p$ 有$S[i]=S[i+p]$。

(1) p 是周期，当且仅当S 有长为$n-p$ 的公共前后缀。$S[i]=S[i+p]$ 对一切合法i 成立，等价于说$S[0..n-p-1]$（前缀）与$S[p..n-1]$（后缀）逐位相同，即长为$n-p$ 的前缀与后缀相等。

(2) 最长公共真前后缀$\leftrightarrow$ 最小周期。由 (1)，公共前后缀的长度越长，对应的周期$p=n-$（长度）越小。f 是最长的公共**真**前后缀长度，故$p=n-f$ 是最小的**正**周期。

(3) 整数次重复的充要条件。若$f>0$ 且p 整除n，则$p<n$、$n/p\ge 2$，把S 按每p 个字符切成n/p 段，由周期性质相邻两段逐位相同，故每段都等于第一段，S 是第一段重复n/p 次。反之若$S=w^k$（$k\ge 2$，w 的长度为q），则q 是周期，故最小周期$p\le q$，且p 整除q、q 整除n，于是p 整除n；又$p\le q<n$，故$f=n-p>0$。∎

「p 整除q」这一步用到**周期引理**（Fine–Wilf 定理）：若p、q 都是S 的周期且$p+q\le n+\gcd(p,q)$，则$\gcd(p,q)$ 也是周期。这里$p\le q$、$n=kq\ge 2q\ge p+q$，条件满足，于是$\gcd(p,q)$ 是周期；而p 已是最小周期、$\gcd(p,q)\le p$，只能$\gcd(p,q)=p$，即p 整除q。（考试中这一步一般允许直接引用。）

注意p 不整除n 的情形并不意味着没有周期：`"abcab"` 的最小周期是 3，它只是「某个周期为 3 的无限串的前 5 个字符」，不是整数次重复。这正是 4-21 里那个取模判断存在的理由。

4-27. 证明：求 next 数组的算法（4-18）时间代价为$O(m)$。

证明. 与 4-25 同构。变量 `k` 在成功分支里每次加 1，共加至多m 次；在失败分支里执行 `k ← next[k]`，由 `next[k] < k` 知每次至少减 1。`k` 的初值为-1、终值≥ -1，故失败分支的执行次数不超过成功分支的次数加 1，总循环次数$\le 2m+1=O(m)$。∎

直观说法：**`k` 是一个「涨得慢、跌得快」的量，但它涨了多少才能跌多少**，所以尽管内层是个 `while`，整体仍是线性的——这正是初学者最容易误判成$O(m^2)$ 的地方。

第 5 章 二叉树

一、填空题

5-1. 二叉树是结点的有限集合，它或者为空，或者由一个根结点及两棵**互不相交**的、分别称为 ____ 和 ____ 的二叉树组成。含 3 个结点的二叉树共有 ____ 种不同形态。

> **答**：**左子树**、**右子树**；**5** 种。一般地，n 个结点的不同二叉树形态数是 Catalan 数 $\frac{1}{n+1}C_{2n}^{n}$：$n=1,2,3,4$ 依次为 $1,2,5,14$——**和第 3 章出栈序列的个数是同一个数**（见 5-35）。二叉树与度为 2 的树的根本差别就在「左右有别」：只有一个孩子时也要说清是左是右。

5-2. 二叉树的第 i 层（根为第 0 层）最多有 ____ 个结点；深度为 k 的二叉树最多有 ____ 个结点；有 n 个结点的完全二叉树，其高度（层数）为 ____。

> **答**：2^{i}；$2^{k+1}-1$；$\lceil \log_2(n+1) \rceil$。**换约定就换公式**：若根记为第 1 层，则第 i 层最多 2^{i-1} 个、深度 k 最多 $2^{k}-1$ 个。考试时先声明自己用哪套约定，能避免大量无谓的扣分。

5-3. ★ 在任意一棵非空二叉树中，若叶结点数为 n_0、度为 2 的结点数为 n_2，则它们的关系是 ____。若该树共有 n 个结点、度为 1 的结点 n_1 个，则 $n=$ ____。

> **答**：$n_0=n_2+1$；$n=n_0+n_1+n_2$。证明见 5-33。这条性质的用法：题目给出叶子数就能直接得到度为 2 的结点数，反之亦然。

5-4. 完全二叉树按层次顺序从 0 开始编号存入一维数组，则结点 i 的父结点编号是 ____，左孩子是 ____，右孩子是 ____；结点 i 有左孩子的条件是 ____。

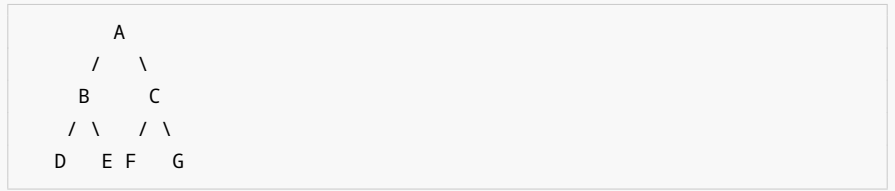
> **答**：$\lfloor (i-1)/2 \rfloor$（$i>0$，根无父结点）；$2i+1$；$2i+2$；$2i+1<n$。（若编号从 1 开始，则分别是 $\lfloor i/2 \rfloor$、$2i$、$2i+1$。）这三个公式是第 5.5 节堆的全部基础——堆之所以不需要指针，靠的就是它们。

5-5. 有 n（$n\geq 2$）个结点的完全二叉树，最后一个分支（非叶）结点的编号是 ____，叶结点共有 ____ 个。

> **答**：$\lfloor n/2 \rfloor -1$（编号从 0）；叶结点 $\lceil n/2 \rceil$ 个。建堆时「从 $i=\lfloor n/2 \rfloor -1$ 开始向前逐个筛选」用的正是前一个公式—— **编号比它大的全是叶结点，本身就是堆，一次都不用碰**。

5-6. 某二叉树的前序序列为 ABDECFG，中序序列为 DBEAFCG，则它的后序序列为 ____，层次序列为 ____。

> **答**：后序 DEBFGCA；层次 ABCDEFG。还原过程：前序首字符 A 是根 → 中序中 A 把序列切成左 DBE、右 FCG；左子树前序 BDE → 根 B，中序 DBE → 左 D、右 E；右子树前序 CFG → 根 C，中序 FCG → 左 F、右 G。树形如下：



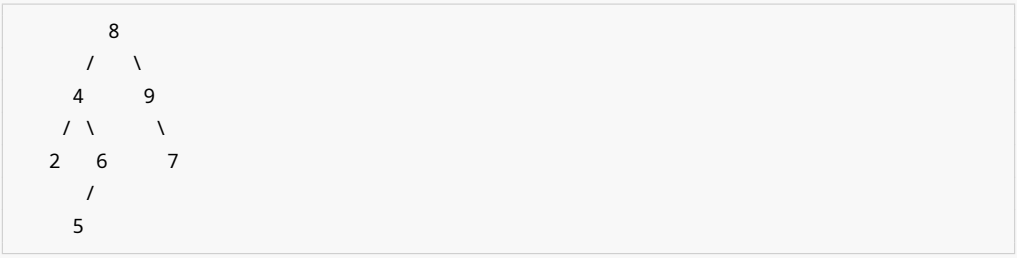
5-7. ★ 含n 个叶结点（权值互不相同）的 Huffman 树共有 ____ 个结点，其中度为 2 的结点 ____ 个，**没有**度为 ____ 的结点。

> **答**：$2n-1$；$n-1$；**1**。理由：Huffman 树是每次合并两棵树得到的，合并$n-1$ 次，每次新增一个内部结点，故内部结点$n-1$ 个、总结点$2n-1$ 个；每个内部结点都恰有两个孩子，所以是**满二叉树**，$n_1=0$，与性质$n_0=n_2+1$ 一致。

5-8. 二叉搜索树的 ____ 遍历序列是一个递增有序序列；在一棵含n 个结点的二叉搜索树中检索一个关键码，最好情况比较 ____ 次，最坏 ____ 次。

> **答**：**中序**；最好 **1 次**（关键码恰在根上）；最坏n **次**（树退化为链且查找最深结点，或沿全链查找失败）。若另问「平衡树中的最坏检索代价」，才是$O(\log n)$，不能与单次检索的最好情况混用。「**二叉搜索树平均$O(\log n)$**」**这句话有前提**——随机插入下才成立，最坏依然是$O(n)$，这正是后面要讲平衡树的原因。

5-9. 把下面这棵二叉树扩充（每棵空子树换成一个外部结点，记为 Φ）后，扩充二叉树的前序序列是 ____，其中外部结点共 ____ 个；反过来，任给一个扩充二叉树的前序序列，____（能／不能）唯一确定原二叉树。



> **答**：8 4 2 Φ Φ 6 5 Φ Φ Φ 9 Φ 7 Φ Φ；**8** 个（$n+1$，见 5-33 推论）；**能**。理由：扩充二叉树里每个内部结点恰有两棵子树，按前序读序列时，读到内部结点就先递归读出它的左子树、再读右子树，读到 Φ 立即返回——每一步怎么走都是唯一的，对序列长度归纳即得结构唯一。对比：**不扩充**的单个前序序列不能确定二叉树（5-12），正是因为丢了「这里是空子树」的信息；Φ 把这份信息补了回来，二叉树的序列化（存盘、传输）常用这种写法。

5-10. 2-3 树满足：(1) 每个内部结点有 2 个或 3 个孩子；(2) 所有叶结点到根的路径长度相同。若一棵 2-3 树有 10 个叶结点，它的内部结点可能有 ____ 个（写出全部可能）。

答：**7 或 8**。设所有叶到根的路径长为h（边数），则叶数在2^h 与3^h 之间：$h=2$ 时至多 9 个，$h=4$ 时至少 16 个，所以$h=3$。**自底向上逐层数**：叶的父结点那一层有m 个结点，需$2m \le 10 \le 3m$，得$m \in \{4,5\}$；再上一层（根的孩子）有k 个，需$2k \le m \le 3k$ 且$2 \le k \le 3$，两种m 都只能$k=2$；最上面是根。内部结点数$= 1+2+m$，即 **7**（$m=4$）或 **8**（$m=5$）。常见错误是只算「全是 2 叉」或「全是 3 叉」两种极端，而 10 两种极端都凑不出来。

二、选择题

5-11. 一棵完全二叉树有 1001 个结点，其中叶结点的个数是（　）。A. 250　B. 500　C. 501　D. 505

答：C。$n=1001$ 为奇数，说明没有度为 1 的结点（度为 1 的结点在完全二叉树中至多一个，且当且仅当n 为偶数时存在）。于是$n_1=0$，由$n_0=n_2+1$ 与$n_0+n_2=1001$ 得$n_0=501$。也可以直接用公式$\lceil n/2 \rceil = \lceil 1001/2 \rceil = 501$。

5-12. 已知二叉树的**前序**和**后序**序列，下列说法正确的是（　）。A. 能唯一确定这棵二叉树 B. 不能唯一确定，但能确定它的中序序列 C. 不能唯一确定这棵二叉树 D. 只有当树中没有度为 1 的结点时才不能确定

答：C。反例：前序 `AB`、后序 `BA` 既可以是「`B` 是 `A` 的左孩子」也可以是「`B` 是 `A` 的右孩子」。**根源在于前序和后序都无法区分「唯一的那个孩子是左是右」**，而中序可以。D 说反了：**恰恰是在没有度为 1 的结点（满二叉树）时，前序加后序才能唯一确定**。

5-13. ★ 在下列关键码序列中，可以构成**最小堆**的是（　）。A. `(12, 70, 33, 65, 24, 56)`　B. `(12, 24, 33, 65, 70, 56)` C. `(70, 65, 56, 33, 24, 12)`　D. `(12, 24, 33, 65, 70, 30)`

答：B。按$K_i \le K_{2i+1}$ 且$K_i \le K_{2i+2}$ 逐个验证（下标从 0）：B 中$K_0=12 \le 24, 33$；$K_1=24 \le 65, 70$；$K_2=33 \le 56$ √。A 错在$K_1=70 > K_3=65$；C 是**最大堆**不是最小堆；D 错在$K_2=33 > K_5=30$。**答题技巧：先把序列按层画成完全二叉树，再只看父子两两比较**——兄弟之间没有任何要求，这是最常被误判的一点。

5-14. 在二叉搜索树中删除一个既有左子树又有右子树的结点，下列做法**能**保持二叉搜索树性质的是（　）。A. 用它的左孩子顶替 B. 用它的右孩子顶替 C. 用它在中序序列中的前驱（左子树中最大者）顶替 D. 直接置空

答：C（用中序**后继**，即右子树中的最小者，同样可行）。理由：中序前驱比左子树其余全部都大、比右子树全部都小，顶上来之后「左小右大」原封不动。A、B 若只是单纯顶替会破坏性质（左孩子顶上来后，它原来的右子树与被删结点的右子树争同一个右指针；要保持性质还得把后者挂到前者最右结点之下，已不是单纯「顶替」；B 与之对称，

争的是左指针）。**实现上还有一个必考的细节**：把前驱顶上去之前，必须先让**前驱的左子树接到前驱原来的位置**，否则会形成环或者丢掉一棵子树（见 5-24）。

5-15. 关于 Huffman 编码，下列说法**错误**的是（　）。A. 它是一种前缀编码，任何字符的编码都不是另一个字符编码的前缀 B. 权值较大的字符，其码长不会比权值较小的字符更长 C. Huffman 树的形态可能不唯一，但带权路径长度 WPL 一定唯一 D. Huffman 编码是定长编码

答：D。Huffman 编码是**不等长（变长）编码**，这正是它能压缩的原因。A 对：字符只放在叶结点上，因此任一编码对应的路径都不会是另一条路径的前缀。B 对。C 对：合并时权值相等会产生不同的选择，形态可以不同，但每一步合并的权值之和都一样，故 WPL 唯一。

5-16. 用筛选法对n 个元素的无序序列建最小堆，时间代价的渐进紧确界是（　）。A. $\Theta(n)$　　B. $\Theta(n\log n)$　　C. $\Theta(\log n)$　　D. $\Theta(n^2)$

答：A。**注意区分两种建堆法**：逐个 `Insert` 建堆是n 次$O(\log n)$ 的上浮，合计$O(n\log n)$；而**筛选法**（从最后一个分支结点起倒着 `SiftDown`）是$O(n)$，因为越靠底层的结点越多、能下沉的距离越短，两者恰好抵消（证明见 5-34）。考题问「建堆」时默认指筛选法。

三、辨析与简答

5-17. 辨析：「二叉树就是度不超过 2 的树。」

答：错。两点差别：1. **左右有别**：二叉树中只有一个孩子时，必须区分它是左孩子还是右孩子，这两棵二叉树是不同的；度为 2 的树中没有这个区分。2. **空树合法**：二叉树允许为空，而树的定义通常要求至少有一个根结点。

一个直接后果：3 个结点的二叉树有 5 种形态，而 3 个结点的度不超过 2 的**树**只有 2 种。

5-18. ★ 简答：为什么「前序 + 中序」或「后序 + 中序」能唯一确定一棵二叉树，而「前序 + 后序」不能？

答：还原一棵二叉树需要两样信息：**谁是根、左右子树各由哪些结点构成**。- 前序的第一个、后序的最后一个都能给出**根**；- **中序的作用是把根两侧切开**，根左边的是左子树、右边的是右子树，于是子问题的规模确定，可以递归下去。- 前序加后序虽然能确定根，却**无法确定左右子树的分界**：当某个结点只有一个孩子时，那个孩子在前序里紧跟父结点、在后序里紧挨父结点之前，无论它是左是右，两个序列都一模一样。

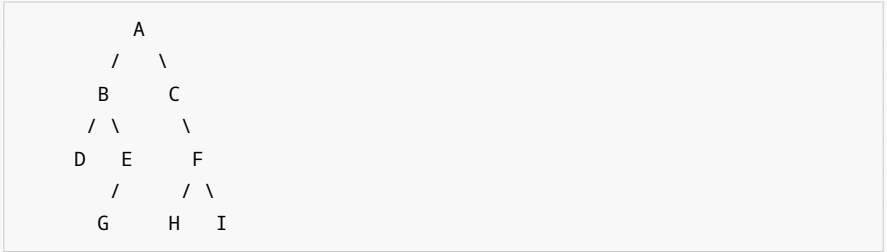
例外：若已知树中**没有度为 1 的结点**（满二叉树），分界就唯一了，此时前序加后序可以确定。

5-19. ★ 某二叉树的 9 个结点标识为 `A`～`I`，三种遍历序列都有残缺（_ 表示缺失的一位）：

```
前序  A B _ E G _ F _ I
中序  D _ G E A C H _ I
后序  _ G E B H _ F C A
```

补全三个序列，并画出这棵二叉树。

答：前序 A B D E G C F H I；中序 D B G E A C H F I；后序 D G E B H I F C A。



思路：始终以中序切分左右，前序、后序只用来认根；缺的那一位由「这一段应有哪些结点」反推。1. **整棵树**：根为 A（前序第一位、后序最后一位一致）。中序中 A 在第 5 位，所以左右子树各 4 个结点：前序第 2～5 位 B _ E G 与后序第 1～4 位 _ G E B 属于左子树，前序第 6～9 位 _ F _ I 与后序第 5～8 位 H _ F C 属于右子树。2. **左子树**：前序首位、后序末位都是 B，根为 B。中序左段 D _ G E 缺的一位必须是左子树的结点，已知左子树含 B，所以是 B；于是左子树结点集是 $\{B, D, E, G\}$，前序、后序缺的都是 D。中序 D B G E 中 B 左边是 D、右边是 G E；右边这段前序为 E G，根是 E，G 在中序里位于 E 左边，是 E 的左孩子。3. **右子树**：后序末位是 C，根为 C，于是前序第 6 位补 C；右子树结点集只剩 $\{C, F, H, I\}$，前序还缺 H，得 C F H I；后序缺 I，得 H I F C；中序右段 C H _ I 缺 F，得 C H F I。中序里 C 排在这一段的第一位，所以 C **没有左子树**；其余 H F I 的前序为 F H I，根 F，左 H、右 I。

为什么三个序列都要用：C、E 都是度为 1 的结点，只看前序和后序分不出它们的孩子在左还是在右（5-18），正是中序给出了这个信息；反过来，中序缺的两位又要靠前序、后序认出的根来补。每补出一位就多一条线索，这类题按「定根 → 按中序定左右规模 → 补本段缺的标识」一层层往下做即可。

5-20. ★ 简答：堆和二叉搜索树都是二叉树，它们的「有序性」有什么本质区别？各适合什么场合？

答：

	最小堆	二叉搜索树
有序性	**局部有序**：只保证父≤子，兄弟之间毫无关系	**全序**：左子树 < 根 < 右子树
形状	必须是**完全二叉树**，可用数组存、无需指针	形状随插入次序而变，最坏退化成链
取最小值	$O(1)$（就是根）	$O(h+1)$，平衡时$O(\log n)$、退化时$O(n)$
查任意值	$O(n)$，只能线性扫	$O(\log n)$（平衡时）
中序遍历	**无意义**，不是有序序列	得到递增有序序列

结论：堆适合「反复取极值」的场合（优先队列、堆排序、Huffman 构造）；二叉搜索树适合「按关键码检索、且要求插删也快」的场合。建堆只要$O(n)$ 而排序要$O(n\log n)$，差别正来自堆只要求局部有序。

5-21. 简答：非递归的二叉树遍历为什么需要栈？层次遍历为什么用队列？

答：- **深度优先遍历需要栈**：访问完一个结点之后要先深入它的一棵子树，等整棵子树处理完再回来处理另一棵——「回来」意味着必须记住**还欠着什么没做**，而欠账的处理次序是「最后欠的最先还」，正是后进先出。递归版本用的是系统运行栈，非递归版本把它换成了显式栈（见第 3.1.5 节的一般方法）。- **广度优先遍历需要队列**：要按离根由近及远访问，先被发现的结点其孩子也应该先被访问，这是先进先出。

一句话：**栈让你走得深，队列让你铺得广**。

四、算法设计与分析

本组题统一使用如下二叉树 ADT：

```
结点 Node
    data          结点数据
    left, right   左右孩子指针，空为 nullptr

BinaryTree
    Root()                出口：根指针
    Left(p) / Right(p)    入口：结点指针；出口：左/右孩子指针
    Value(p)              入口：结点指针；出口：该结点数据
```

5-22. ★ 用**递归**写出下列算法，并分析代价：(1) 求二叉树的高度；(2) 求叶结点个数；(3) 交换所有结点的左右子树（求镜像）；(4) 判断两棵二叉树是否相同。

答：四题是同一个模板——「**空树给出平凡答案，非空树由两棵子树的答案合成**」。

```
Height(p)                                  // (1) 高度按层数计：空树为 0，只有根
↪  时为 1
    if p = nullptr then return 0
    return 1 + max(Height(p->left), Height(p->right))

Leaves(p)                                  // (2)
    if p = nullptr then return 0
    if p->left = nullptr and p->right = nullptr then return 1
    return Leaves(p->left) + Leaves(p->right)

Mirror(p)                                  // (3) 镜面映射
    if p = nullptr then return
    交换 p->left 与 p->right
    Mirror(p->left)
    Mirror(p->right)

Same(p, q)                                 // (4)
    if p = nullptr and q = nullptr then return true
    if p = nullptr or  q = nullptr then return false
    return p->data = q->data and Same(p->left, q->left) and
↪  Same(p->right, q->right)
```

代价：前三个算法的时间为$\Theta(n)$；`Same` 可提前返回，最坏时间为$\Theta(n)$（n 为两树结点数之和）；空间是递归栈深度$O(h)$，h 为树高——**平衡时**$O(\log n)$，**退化成链时**$O(n)$。后一句是评分点：只写「空间$O(\log n)$」是不完整的。

(2) 的常见错法：写成 `if p = nullptr then return 1`。这样只有一个孩子的结点会被算成一片叶子，结果偏大。**叶子的定义是「两个孩子都空」**，必须显式判断。

5-23. ★ 算法填空：下面是二叉树**中序**遍历的非递归框架，补全空缺处。

```
InOrder(root)
    S ← 空栈
    p ← root
    while p ≠ nullptr or not S.Empty()
        if p ≠ nullptr
            S.Push(p)
            p ← ____(①)____
        else
            p ← S.Pop()
            访问 p
            p ← ____(②)____
    结束
```

答：① `p->left`　② `p->right`

框架怎么记：一路向左压栈，压到底之后弹一个出来访问，然后**转向它的右子树**，对右子树重复同样的过程。循环条件里的 `or` 不能写成 `and`——刚弹出一个结点而它没有右子树时 `p` 为空，但栈里还有欠账，不能退出。

同一框架的另外两种次序：

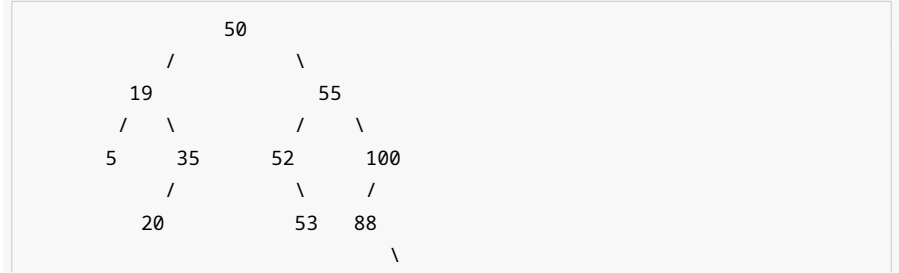
```
PreOrder(root)                              // 前序：弹出即访问
    S ← 空栈；  S.Push(root)
    while not S.Empty()
        p ← S.Pop()
        if p = nullptr then continue
        访问 p
        S.Push(p->right)                   // 先压右、后压左，弹出时才是「左优先」
        S.Push(p->left)

LevelOrder(root)                            // 层次（广度优先）：把栈换成队列
    Q ← 空队列；  Q.EnQueue(root)
    while not Q.Empty()
        p ← Q.DeQueue()
        if p = nullptr then continue
        访问 p
        Q.EnQueue(p->left)
        Q.EnQueue(p->right)
```

前序那段里「先压右后压左」是必考点：栈是后进先出，要让左子树先被处理就得后压它。**后序**最麻烦，因为一个结点要等两棵子树都处理完才能访问，常见写法是给栈里的记录附一个标记位（第一次弹出只是「回来过一次」，压回去再走右子树），这正是第 3 章「用栈消除递归」里那个**返回点编号**的直接应用。

5-24. ★ 给定关键码序列$K = \{50, 19, 35, 55, 20, 5, 100, 52, 88, 53, 92\}$，(1) 按此次序依次插入，画出生成的二叉搜索树；(2) 写出中序序列；(3) 删除结点 50，画出删除后的树并说明步骤。

答：**(1) 建树**（每个新键从根开始比较，小的往左、大的往右，走到空位置挂上去）：



```
                                92
```

完整结构：50 的左子树是 19（左 5，右 35，35 的左是 20）；右子树是 55（左 52，52 的右是 53；右 100，100 的左是 88，88 的右是 92）。

(2) 中序序列：5 19 20 35 50 52 53 55 88 92 100——**递增有序**，可用来自检建树对不对。

(3) 删除 50（它左右子树都在，用**中序前驱**顶替）：1. 在左子树中一路向右找到最后一个结点，即 35（它是左子树里的最大者，也是 50 的中序前驱）；2. **先把 35 从原位置摘走**：35 没有右子树，用它的左子树 20 顶替它的位置，即令 19->right ← 20；3. 再让 35 继承被删结点的两棵子树：35->left ← 19、35->right ← 55，并让原来指向 50 的指针（此处是根）指向 35；4. 释放 50。

删除后：

```
            35
          /    \
        19      55
       /  \    /  \
      5    20 52   100
               \   /
               53 88
                    \
                     92
```

中序验证：5 19 20 35 52 53 55 88 92 100，仍然递增 √。

第 2 步不能省：若直接让 35 顶替而不先安置它的左子树 20，20 就会丢失（或者 35->left ← 19 之后 19->right 仍指着 35，形成环）。本书教学版的测试专门搭了一棵「前驱自己还有左孩子」的树——去掉那一句，树上立刻成环。

5-25. ★ 用值为1 到5 的 5 个结点建二叉搜索树。(1) 若要求树上每个结点的**左、右子树结点数之差不超过 1**，共有多少棵? (2) 若把要求换成每个结点的**左子树结点数不少于右子树**，又有多少棵?

答：(1) **4 棵**；(2) **6 棵**。

关键第一步：给定关键码集合，**二叉搜索树由树的形状唯一决定**——中序必须递增，所以哪个值放哪个位置是定死的。于是问题变成「数满足约束的二叉树**形状**」，用 5-35 的递推、再把求和范围收窄即可。

(1) 设$g(n)$为n个结点、每个结点都满足「左右子树结点数之差≤ 1」的形状数，$g(0)=g(1)=1$。按根的左右子树规模(l,r)，$l+r=n-1$：

n	允许的(l,r)	$g(n)$
2	(1,0)、(0,1)	$1+1=2$
3	只有 (1,1)（(2,0) 差 2，不行）	1
4	(2,1)、(1,2)	$2+2=4$
5	只有 (2,2)	$2\times 2=4$

(2) 设$f(n)$ 为满足「$l\ge r$」的形状数，求和范围换成$l\ge r$：

n	允许的(l,r)	$f(n)$
2	(1,0)	1
3	(2,0)、(1,1)	$1+1=2$
4	(3,0)、(2,1)	$2+1=3$
5	(4,0)、(3,1)、(2,2)	$3+2+1=6$

对照：完全不加约束时就是 Catalan 数$C_5=42$（5-35）。加一条约束，递推式的求和范围从「全部l」缩成一段，数量就掉到 4 或 6。

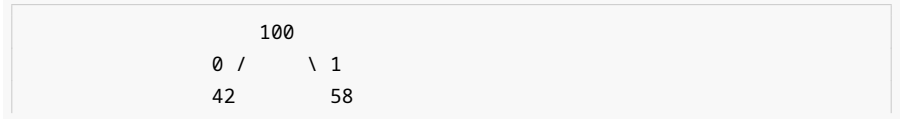
带约束的计数题一律这么做：先写出无约束的递推，再收窄求和范围。不要试图直接画图穷举——n 稍大就画不完，而且极易重复或漏掉。

5-26. ★ 已知字符集$\{a,b,c,d,e,f,g,h\}$ 的使用频率依次为 5、29、7、8、14、23、3、11（合计 100）。(1) 构造 Huffman 树；(2) 给出各字符的 Huffman 编码；(3) 求带权路径长度 WPL 与平均码长；(4) 对码流 `000110110` 译码。

答：**(1) 构造**（每次取权值最小的两棵树合并，新树权值为两者之和，放回集合）：

步	合并的两个权	新权	剩余集合
1	3(g) + 5(a)	8	7, 8(d), **8**, 11, 14, 23, 29
2	7(c) + 8(d)	15	**8**, 11, 14, **15**, 23, 29
3	8 + 11(h)	19	14, 15, **19**, 23, 29
4	14(e) + 15	29	19, 23, **29**, 29
5	19 + 23(f)	42	29, 29, **42**
6	29(b) + 29	58	42, **58**
7	42 + 58	100	完成

树形（左分支记 0、右分支记 1）：



```
          0 /  \1    0 /  \1
         19     23(f) 29(b) 29
       0/  \1              0/  \1
      8    11(h)         14(e)  15
    0/ \1                     0/  \1
  3(g) 5(a)                  7(c) 8(d)
```

(2) 编码：

字符	g	a	h	f	b	e	c	d
权	3	5	11	23	29	14	7	8
编码	0000	0001	001	01	10	110	1110	1111
码长	4	4	3	2	2	3	4	4

(3) WPL：既可以逐个算$\sum w_i l_i$，也可以用一条更快的性质——**WPL 等于构造过程中所有新生成结点权值之和**：

$$\text{WPL} = 8 + 15 + 19 + 29 + 42 + 58 + 100 = 271$$

逐个验算：$3{\cdot}4+5{\cdot}4+11{\cdot}3+23{\cdot}2+29{\cdot}2+14{\cdot}3+7{\cdot}4+8{\cdot}4 = 271$ ✓ 平均码长$= 271/100 = 2.71$ 位/字符。对照：8 个字符的定长编码需要 3 位，压缩率约$2.71/3 \approx 90.3\%$。

(4) 译码 `000110110`：从根出发，遇 0 走左、遇 1 走右，**到叶结点就输出一个字符并回到根**。`0001` → `a`；`10` → `b`；`110` → `e`。结果是 **abe**。能这样一位一位走下去而不产生歧义，正是因为 Huffman 码是**前缀码**（字符只在叶结点上）。

注意：权值出现并列时（本题第 2 步的两个 8）选谁都可以，**树形会不同但 WPL 相同**，编码也随之不同。答题时注明自己的取舍规则即可。

5-27. ★ 字符集$\{A, B, C, D\}$的出现频率为$\{50\%, 25\%, 12.5\%, 12.5\%\}$。(1) 构造 Huffman 树，给出各字符的编码与平均码长，并与定长编码比较；(2) 若D的频率**上升**、其余三个字符的相对比例不变，而**编码方式仍用第 (1) 问算出的那一套**，D 的频率升到多少时，这套 Huffman 编码反而不如定长编码省？

答：**(1)** 合并过程：$0.125+0.125 = 0.25$；再$0.25(B)+0.25 = 0.5$；最后$0.5(A) + 0.5 = 1$。

```
        1.0
    0 /    \ 1
  A(.5)     .5
         0 /   \ 1
        B(.25)   .25
              0 /   \ 1
```

```
C(.125) D(.125)
```

字符	A	B	C	D
频率	0.5	0.25	0.125	0.125
编码	`0`	`10`	`110`	`111`
码长	1	2	3	3

平均码长$= 0.5\times 1 + 0.25\times 2 + 0.125\times 3 + 0.125\times 3 = \mathbf{1.75}$ 位。定长编码 4 个字符需 2 位，所以 Huffman 每字符省 0.25 位（约 12.5%）。

(2) D 的频率超过$3/10$ 之后就不如定长。设D 的频率为y，其余三个按原比例（$4:2:1$）缩放，它们原本合计0.875、加权码长$\sum f\cdot l = 1.75 - 0.125\times 3 = 1.375$。这套**固定不变**的码的平均长度是

$$L(y) = \frac{1.375}{0.875}(1-y) + 3y = \frac{11}{7}(1-y) + 3y = \frac{11}{7} + \frac{10}{7}y$$

令$L(y) = 2$（定长的位数）得$y = \frac{3}{10}$。

验算$y = 0.3$：其余按$4:2:1$ 分掉0.7，即$A = 0.4$、$B = 0.2$、$C = 0.1$，$L = 0.4\times 1 + 0.2\times 2 + 0.1\times 3 + 0.3\times 3 = 2.0$ ✓ 恰好等于定长。

这一问考的是一句话：Huffman 树是按一组给定频率算出来的，频率一旦漂移而码表不变，它就不再最优，甚至可能输给定长编码。但别把结论说过头——**按新频率重算**出来的 Huffman 码永远不会差于定长（它是最优前缀码）；会输的是「用旧频率算出来、却拿去编新数据」的那套码。实际的压缩器要么随数据自适应地改码表，要么把码表随文件一起存下来，原因正在这里。

5-28. ★ 给定序列$K = \{53, 17, 78, 9, 45, 65, 87, 23, 31\}$，(1) 用筛选法把它建成**最小堆**，写出每一步；(2) 在结果堆上删除最小元，写出调整后的数组；(3) 回到 (1) 的建堆结果，插入元素 20，写出 SiftUp 调整后的数组。

答：**筛选法**从最后一个分支结点$i = \lfloor n/2 \rfloor - 1$ 开始倒着做 `SiftDown`。本题$n = 9$，故从$i = 3$ 开始。`SiftDown(i)` 的动作是：与两个孩子中**较小的**比较，若比它大就交换并继续下沉，否则停止。

初始（下标 0..8）：`[53, 17, 78, 9, 45, 65, 87, 23, 31]`

步	i	结点	孩子	动作	数组
1	3	9	23, 31	$9 \le 23$, 不动	53 17 78 9 45 65 87 23 31
2	2	78	65, 87	$78 > 65$, 交换	53 17 65 9 45 78 87 23 31
3	1	17	9, 45	$17 > 9$, 交换；再看 17 的孩子 23、31, $17 \le 23$ 停	53 9 65 17 45 78 87 23 31
4	0	53	9, 65	$53 > 9$ 交换 → 53 到下标 1，孩子 17、45, $53 > 17$ 交换 → 53 到下标 3，孩子 23、31, $53 > 23$ 交换 → 53 到下标 7，是叶，停	9 17 65 23 45 78 87 53 31

建堆结果：`[9, 17, 65, 23, 45, 78, 87, 53, 31]` 校验：$9 \le 17, 65$；$17 \le 23, 45$；$65 \le 78, 87$；$23 \le 53, 31$ √

(2) 删除最小元 9：把**末尾元素 31 移到根**、堆长减到 8，再对根 `SiftDown`：

- 根 31，孩子 17、65 →$31 > 17$，交换 → `[17, 31, 65, 23, 45, 78, 87, 53]`
- 31 在下标 1，孩子 23、45 →$31 > 23$，交换 → `[17, 23, 65, 31, 45, 78, 87, 53]`
- 31 在下标 3，只有一个孩子 53 →$31 \le 53$，停。

结果：`[17, 23, 65, 31, 45, 78, 87, 53]`，取出的最小元是 **9**。

(3) 在 (1) 的建堆结果 `[9, 17, 65, 23, 45, 78, 87, 53, 31]` 上插入 20：新元素先放到**末尾**（下标 8 之后的下标 9），再沿祖先路径**向上浮**，直到父结点不大于它、或者升到根为止。

位置	父结点	比较	结果
9	下标$\lfloor(9-1)/2\rfloor=4$，值 45	$20<45$	交换 → `9 17 65 23 20 78 87 53 31 45`
4	下标$\lfloor(4-1)/2\rfloor=1$，值 17	$20>17$	停

结果：`[9, 17, 65, 23, 20, 78, 87, 53, 31, 45]`，只交换了一次。

代价：建堆$O(n)$（见 5-34）；删除最小元与插入都是$O(\log n)$——路径长度不超过树高（层数）减 1，而完全二叉树的高度是$\lceil\log_2(n+1)\rceil$（5-36）。

三**个常见错**：1. `SiftDown` 只跟左孩子比——必须跟**两个孩子中较小的**比；2. 建堆时从$i=0$ 正着做——那样子树还不是堆，筛选的前提不成立，结果是错的；3. **把插入的方向搞反**。记住两句话：**建堆是从最后一个分支结点倒着向下筛**（整体$O(n)$），**插入是从末尾向上浮**（单次$O(\log n)$）。正因为方向不同，逐个 `insert` 建堆要$O(n\log n)$，而筛选法只要$O(n)$（5-16、5-34）。

5-29. ★ 一棵N 个结点的二叉树用两个一维数组存储：`L[i]`、`R[i]` 分别是结点i的左、右孩子编号，为 0 表示空（结点从 1 开始编号，`L[0] = R[0] = 0`，根的编号已知）。(1) 设计算法由 `L`、`R` 生成父指针数组 `T`，使 `T[i]` 为结点i 的父结点编号（根记 0）；(2) 设计算法判断结点U 与V 是否在**同一条从根到叶的路径**上（即其中一个是另一个的祖先）。分析两问的代价。

答：**(1) 基本思想——反着填**。与其对每个结点去「找父亲」（那要把 `L`、`R` 整个搜一遍，$O(N)$ 一次），不如让**每个结点把自己的两个孩子登记一次**。每条边只写一次，一趟就够。

```
BuildParent(L, R, N)
    for i ← 0 to N:  T[i] ← 0              // 根与空位都留 0
    for i ← 1 to N
        if L[i] ≠ 0 then T[L[i]] ← i
        if R[i] ≠ 0 then T[R[i]] ← i
    return T
```

正确性：二叉树里每个非根结点恰好被某一个结点当作左孩子或右孩子登记一次，所以每个 `T[i]` 至多被写一次、也至少被写一次；根从没被登记，保持 0。**代价**：时间$\Theta(N)$，空间$\Theta(N)$（就是 `T` 本身）。

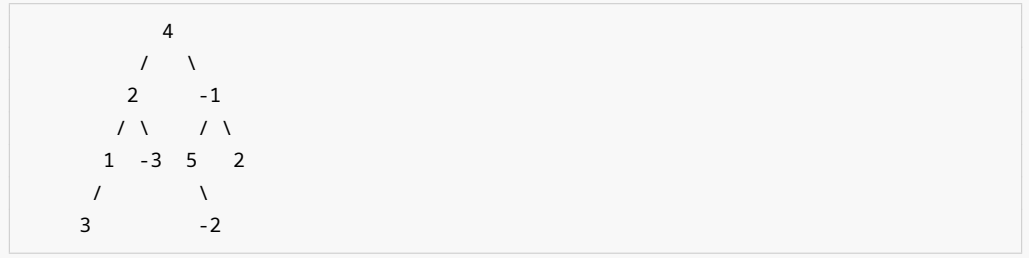
(2) 基本思想——「在同一条根到叶的路径上」等价于「其中一个是另一个的祖先」。有了父指针，只要**从其中一个出发沿父链往上走**，看能不能碰到另一个；两个方向各试一次。

```
IsAncestor(x, y, T)              // 判断 y 是不是 x 的祖先（x ≠ y）
    p ← T[x]
    while p ≠ 0
        if p = y then return true
        p ← T[p]
    return false

SamePath(U, V, T)
    if U = V then return true
    return IsAncestor(U, V, T) or IsAncestor(V, U, T)
```

代价：时间$O(h)$（h 为树高），最坏$O(N)$（树退化成链）；空间$O(1)$。**为什么要翻成「往上走」**：直接做是「从V 出发把整棵子树遍历一遍，看里面有没有U」，那是$O($子树大小$)$、最坏$O(N)$，还要一个栈。**往上走只有一条路，往下走有很多条**——这正是父指针表示法的长处，和 6-23 的并查集 `Find` 是同一个道理（见 6-18 的对照表）。

5-30. 二叉树的结点存整数（可正可负）。从某个结点出发、**只沿孩子方向往下走**、在它自己或它的某个后代处结束，经过的结点序列称为一条**向下路径**（不必从根开始，也不必在叶结束）。给定整数 `target`，设计算法统计结点值之和等于 `target` 的向下路径条数，并分析代价。对下面这棵树、`target = 3`，答案是多少？



答：**5 条**：$4 \to 2 \to -3$，$4 \to -1$，$2 \to 1$，3（单个结点），$5 \to -2$。**解法一：双重递归**——按起点分类：先数「从某个结点出发」的路径，再对所有起点求和。

```
CountFrom(p, need)           // 从 p 出发、和为 need 的向下路径条数
    if p = nullptr then return 0
    c ← 0
    if p->data = need then c ← 1          // 在 p 处结束的那一条
    rest ← need - p->data
    return c + CountFrom(p->left, rest) + CountFrom(p->right, rest)

PathSum(p, target)           // 子树 p 中和为 target 的向下路径数
    if p = nullptr then return 0
    c ← CountFrom(p, target)
```

```
    return c + PathSum(p->left, target) + PathSum(p->right, target)
```

注意：找到一条后**不能停**，也不能在部分和超过 `target` 时剪枝——结点值可以是负数，继续往下还可能再凑出 `target`（例如从 `4` 出发，$4 \to -1$ 和为 3，再往下 $4 \to -1 \to 5 \to -2$ 和为 6，若树里还有值为 -3 的孩子就又凑出 3）。**代价**：每个结点会被它的每个祖先（含自身）各发起的 `CountFrom` 访问一次，共 $\sum_v (\text{深度}_v + 1) = O(nh)$ 时间，退化成链时为 $O(n^2)$；递归栈 $O(h)$。

解法二：前缀和——一趟深度优先周游，维护从根到当前结点路径上的前缀和数组 `pre[0..d+1]`，`pre[0] = 0`。路径 $u \to \cdots \to v$ 的和等于 $\texttt{pre}_v - \texttt{pre}_{\text{u 的父亲}}$，所以在 v 处结束且和为 `target` 的路径条数，就是路径上满足 `pre[d+1] - pre[j] = target`（$0 \le j \le d$）的 j 的个数。

```
Walk(p, d)                  // p 在第 d 层；pre[0..d] 已是根到 p 的父亲的前缀和
    if p = nullptr then return
    pre[d + 1] ← pre[d] + p->data
    for j ← 0 to d
        if pre[d + 1] - pre[j] = target then cnt ← cnt + 1
    Walk(p->left, d + 1);  Walk(p->right, d + 1)
```

调用前令 `pre[0] ← 0`、`cnt ← 0`，执行 `Walk(root, 0)`。代价同样是 $O(nh)$ 时间，但只周游一趟，额外空间 $O(h)$。若用散列表记录「各前缀和出现了几次」（第 10 章），每个结点只需查一次表，期望时间降到 $O(n)$；这时**从一个结点返回前必须把它的前缀和计数减回去**，否则兄弟子树会把不在同一条路径上的前缀算进来。数组版本没有这个问题：返回后 `pre[d+1]` 会被下一个兄弟直接覆盖。

5-31. 二叉搜索树的结点含 `key`、`left`、`right`、`parent` 四个域（关键码互不相同）。**不用栈、不用递归、不在结点上加标记**，设计算法返回第 k 小的关键码所在的结点（k 超过结点数时返回空），并分析时间代价。

答：**基本思想**——BST 的中序序列就是关键码的升序，所以从最小结点出发，沿「中序后继」走 $k-1$ 步即可。父指针让求后继不再需要栈：

- 有右子树：后继是**右子树的最左结点**；
- 没有右子树：沿父指针往上，直到当前结点是其父的**左**孩子，这个父结点就是后继（一路上作为右孩子返回的祖先，都已经在它之前访问过了）。

```
Leftmost(x)
    while x.left ≠ null do x ← x.left
    return x

Successor(x)
```

```
    if x.right ≠ null then return Leftmost(x.right)
    while x.parent ≠ null and x = x.parent.right do x ← x.parent
    return x.parent                    // x 已是最大结点时返回 null

KthSmallest(root, k)
    if root = null then return null
    x ← Leftmost(root)
    repeat k - 1 times
        x ← Successor(x)
        if x = null then return null  // k 超过结点数
    return x
```

代价：$O(h+k)$。走到最左用$O(h)$；连续求$k-1$次后继就是中序遍历的前k步，途经的每条边至多向下、向上各走一次，这些边都落在前k个结点与根到它们的路径上。求第k大对称地从最右结点出发、沿中序前驱走。另一种写法是在结点上加 `visited` 标记来模拟中序——那会**改动树**，下次查询前还得把标记清掉；父指针版只读不写，可以反复查询。

5-32. 求二叉树中两个结点 u、v 的**最近公共祖先**（结点可以是自己的祖先；u、v 都在树中）。(1) 用三叉链表（结点另有 `parent` 域）存储时，给出时间$O(h)$、额外空间$O(1)$的算法；(2) 用二叉链表（只有 `left`、`right`）存储时，给出时间$O(n)$的算法。

答：**(1)** 先求两者的深度，让深的那个沿 `parent` 上移到同一深度，再两个一起上移，直到相遇。

```
Depth(x)
    d ← 0
    while x.parent ≠ null do
        x ← x.parent;  d ← d + 1
    return d

LCA_Parent(u, v)
    du ← Depth(u);  dv ← Depth(v)
    while du > dv do
        u ← u.parent;  du ← du - 1
    while dv > du do
        v ← v.parent;  dv ← dv - 1
    while u ≠ v do
        u ← u.parent;  v ← v.parent
    return u
```

时间$O(h)$，额外空间$O(1)$。这和第 2 章 2-24 求两个单链表的第一个公共结点**是同一道题**：从 u、v 沿父指针到根的两条路径，正是两条共享后缀的单链表，最近公共祖先就是第一个共享结点。

(2) 后序递归，每棵子树向上报告「在我这里找到的 u、v 或它们的最近公共祖先」：

```
LCA_Binary(t, u, v)
    if t = null or t = u or t = v then return t
    L ← LCA_Binary(t.left, u, v)
    R ← LCA_Binary(t.right, u, v)
    if L ≠ null and R ≠ null then return t     // u、v 分居 t 的两侧
    return (L ≠ null ? L : R)                  // 都在同一侧，把那一侧的结果往上交
```

正确性：若 u、v 分居 t 的左右两侧，t 就是最近公共祖先；若 t 本身是 u（或 v），另一个要么在它下面（t 即所求），要么不在——这时由更高层的结点合并两侧结果。这依赖「两者都在树中」的前提；不保证时要另外确认两者都被找到过。**代价**：每个结点访问一次，时间$O(n)$；递归栈$O(h)$。没有父指针，就只能从根往下找，代价从$O(h)$涨到$O(n)$——这正是 5-29 说的「往上走只有一条路」的价值。

五、数学证明

5-33. ★ 证明：非空二叉树中叶结点数n_0与度为 2 的结点数n_2满足$n_0 = n_2+1$。

证明. 设结点总数为n、度为 1 的结点数为n_1，则按度分类有

$$n = n_0 + n_1 + n_2$$

换一个角度数**边**：除根之外每个结点都恰有一条边从它的父结点进入，故边数$e = n - 1$；而每条边都是从某个度为 1 或 2 的结点射出的，故$e = n_1 + 2n_2$。两式联立：

$$n_0 + n_1 + n_2 = n_1 + 2n_2 + 1 \Rightarrow n_0 = n_2 + 1$$

∎

推论（满二叉树定理）：满二叉树中$n_1 = 0$，故**叶结点数 = 分支结点数 + 1**。**推论（空子树数）**：把所有空子树换成叶结点得到扩充二叉树T'，它是满二叉树、且原树的每个结点在T'中都成了分支结点，故T'的叶数 = 原树结点数 + 1，即**一棵n个结点的二叉树恰有$n+1$棵空子树**（链式存储时就是$n+1$个空指针）。

5-34. ★ 证明：用筛选法把n个元素的无序序列建成堆，时间代价为$O(n)$。

证明. 关键观察：**下沉的代价不是树高$\log n$，而是「这个结点离最底层还有多远」**；越靠底层的结点越多，能下沉的距离却越短，两者恰好抵消。

先把堆补成一棵层数相同、**各层全满的二叉树**（深度即最大层号$h =$

$\lfloor \log_2 n \rfloor$，根在第 0 层）。满树每个位置能下沉的距离都不比原堆对应位置短，故在满树上算出的总代价是原堆的上界。各层全满的树第i层恰有2^i个结点，每个最多下沉$h-i$层，于是

$$\text{建堆代价} \le \sum_{i=0}^{h} 2^i (h-i) = \sum_{j=0}^{h} 2^{h-j} \cdot j = 2^h \sum_{j=0}^{h} \frac{j}{2^j},$$

而

$$2^h \sum_{j=0}^{h} \frac{j}{2^j} < 2^h \cdot 2 = 2^{h+1} \le 2n,$$

其中用到$\sum_{j\ge 0} j/2^j = 2$与$2^h \le n$。故建堆为$O(n)$。∎

对照：逐个 `Insert` 建堆是$O(n \log n)$——每次上浮都可能走满一条树高，而**上浮的代价才真正是**$\log n$（越靠底层的结点离根越远）。两种建法的差别正好说明：**同一个结构，调整方向不同，代价可以差一个**$\log$。

5-35. 证明：n 个结点的不同二叉树形态共有$C_n = \frac{1}{n+1} C_{2n}^{n}$ 种。

证明. 设b_n为答案，$b_0 = 1$（空树算一种）。一棵n个结点的非空二叉树由根、左子树、右子树构成；若左子树有i个结点，则右子树有$n-1-i$个（$0 \le i \le n-1$），两者独立，故

$$b_n = \sum_{i=0}^{n-1} b_i \, b_{n-1-i}$$

这与第 3 章出栈序列个数的递推式（3-30）**完全相同**，初值也相同，故$b_n = C_n$。∎

两者之间其实有直接的一一对应：递归定义编码$E(\emptyset) = \epsilon$、$E(T) = I\,E(T_{左})\,O\,E(T_{右})$。首个 I 与其配对的 O 唯一分出左右子树编码，因此合法 I/O 序列与二叉树形态一一对应。记住这个数：$1, 1, 2, 5, 14, 42, 132, \cdots$

5-36. 证明：n 个结点的完全二叉树高度（层数）为$\lceil \log_2(n+1) \rceil$。

证明. 设高度为h（即层号$0..h-1$，共h层）。完全二叉树的前$h-1$层是满的，共$2^{h-1}-1$个结点；第$h-1$层至少 1 个、至多2^{h-1}个。故

$$2^{h-1} - 1 + 1 \le n \le 2^h - 1 \Rightarrow 2^{h-1} \le n < 2^h$$

即$2^{h-1} < n+1 \le 2^h$，两边取以 2 为底的对数得$h-1 < \log_2(n+1) \le h$，于是$h = \lceil \log_2(n+1) \rceil$。∎

用处：这正是堆的插入、删除为什么是$O(\log n)$——一次上浮或下沉最多走$h-1$步。

第 6 章 树

一、填空题

6-1. 树是n（$n \geq 1$）个结点的有限集合。其中有且仅有一个 ___ 结点，其余结点可分为m 个 ___ 的有限集合，每一个又都是一棵树，称为根的 ___。m 棵互不相交的树的集合称为 ___。

> **答**：**根**；**互不相交**；**子树**；**森林**。「互不相交」这四个字是树与图的分界：一旦两棵子树共享结点，就出现了回路或多个前驱，那是图。

6-2. 结点的**度**是指 ___；树的度是指 ___。若一棵树中度为i 的结点有n_i 个（$i = 0, 1, \cdots, m$），则该树的总结点数$n =$ ___，且有$n =$ ___（用度数表示）。

> **答**：该结点的**子树个数（孩子数）**；树中**所有结点度的最大值**；$n = \sum_{i=0}^{m} n_i$；$n = 1 + \sum_{i=1}^{m} i \cdot n_i$。后一条的来历：**除根之外每个结点都恰有一条边指向它**，而边的总数等于所有结点度之和。这是求叶结点数一类填空题的万能工具（见 6-9）。

6-3. 一棵树的嵌套括号表示为 `(A(B(E)(F))(C)(D(G(H))))`。该树共有 ___ 个结点，树的度是 ___，叶结点有 ___ 个，度为 1 的结点有 ___ 个。

> **答**：**8**；**3**；**4**；**2**。
>
> 读法：**紧跟在一个结点后面的每一对括号，就是它的一棵子树**。拆开来是
>
> ```
> A
> ├── B
> │   ├── E
> │   └── F
> ├── C
> └── D
>     └── G
>         └── H
> ```
>
> 各结点的度：A 为 3，B 为 2，D 为 1，G 为 1，其余（E F C H）为 0。故结点数$n = 8$；**树的度**是所有结点度的最大值，为 3；叶结点是 E F C H 共 4 个；度为 1 的是 D G 共 2 个。
>
> **用 6-2 的度数公式自检**，这一步能挡住绝大多数数错：
>
> $$n = 1 + \sum_{i} i \cdot n_i = 1 + (1 \times 2 + 2 \times 1 + 3 \times 1) = 1 + 7 = 8$$
>
> 注意别把「结点的度」和「树的度」混了：前者是某个结点的孩子数，后者是全树的最大值。

6-4. ★ 森林转换成二叉树的规则是：___ 作为二叉树的左子树，___ 作为右子树。转换后，二叉树根结点的**右子树**对应原森林中的 ___。

> **答**：结点的**第一个孩子**（长子）作左子树；结点的**下一个兄弟**作右子树（即「**左子/右兄**」规则）；根的右子树对应原森林中**除第一棵树之外的其余各树**。一个立刻可用的推论：**一棵树（而非森林）转成的二叉树，其根的右子树必为空**。

6-5. ★ 按**先根次序**周游森林，等同于按 ___ 次序周游它对应的二叉树；按**后根次序**周游森林，等同于按 ___ 次序周游它对应的二叉树。二叉树的前序、中序、后序三种深度优先遍历中，不能保证与森林的先根或后根任何一种相同的是 ___ 遍历。

> **答**：**前序**；**中序**；**后序**。例如根 A 只有孩子 B 时，森林的先根为 AB、后根为 BA，对应二叉树后序为 BA；而森林仅由两个单结点树 A、B 组成时，先根和后根均为 AB，对应二叉树后序却为 BA。因而没有上述两种周游中的一种能对所有森林都对应二叉树后序。这两条对应关系是本章最常考的结论，证明见 6-28。

6-6. ★ 完全K 叉树按层次从 0 开始编号，则结点i 的第j 个孩子（$1 \le j \le K$）编号为 ___，其父结点编号为 ___；第i 层最多有 ___ 个结点；高度为h 层且各层全满的K 叉树共有 ___ 个结点。

> **答**：$Ki+j$；$\lfloor (i-1)/K \rfloor$（$i>0$，根无父结点）；K^i；$\frac{K^h-1}{K-1}$（共h 层，层号$0..h-1$）。$K=2$ **时全部退化为二叉树的公式**：孩子$2i+1, 2i+2$，父$\lfloor (i-1)/2 \rfloor$，第i 层2^i 个，h 层共2^h-1 个——可以用这个特例检验自己有没有记错。（若编号从 1 开始，则孩子为$K(i-1)+j+1$，父为$\lceil (i-1)/K \rceil$。）

6-7. 一棵有n 个结点的K 叉树，采用「每个结点固定K 个孩子指针」的链式存储时，其中空指针的个数为 ___。

> **答**：$nK-n+1 = n(K-1)+1$。理由：共nK 个指针域，非空的恰好指向除根之外的每个结点，即$n-1$ 个。$K=2$ 时得$n+1$，正是第 5 章「n 个结点的二叉树有$n+1$ 棵空子树」那条性质。

6-8. 并查集（等价类）用 ___ 表示法存储，`Find` 操作沿 ___ 找到 ___，两个元素属于同一集合当且仅当 ___。两条常用的优化是 ___ 和 ___。

> **答**：**父指针**；**父链向上**；**树根**；**它们的根相同**；**重量权衡合并规则**（元素少的树根挂到元素多的树根下）和**路径压缩**。注意别和「按秩合并」混了：**按秩比的是树高，按重量比的是元素个数**，两者复杂度同阶，但在同一组等价对上长出的树形不同。教材与本课程用的是**重量**。

二、选择题

6-9. 一棵度为 4 的树中，度为 1、2、3、4 的结点分别有 4、2、1、1 个，则叶结点有（ ）个。A. 6　B. 7　C. 8　D. 9

> **答：C**。用$n = 1+\sum i\cdot n_i$：$n = 1+(1\times4+2\times2+3\times1+4\times1) =$

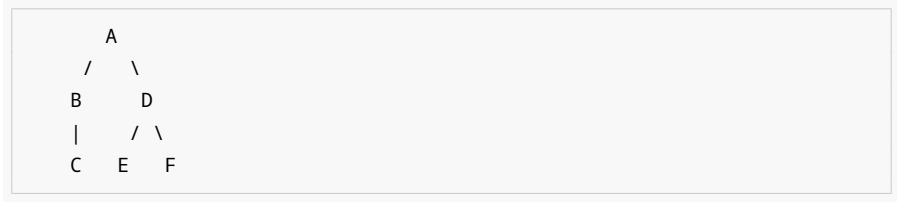
$1+15=16$。又$n=n_0+n_1+n_2+n_3+n_4=n_0+4+2+1+1$，故$n_0=16-8=8$。**这类题一律两步走**：先用度数公式求总结点数，再用分类求和求叶子数。

6-10. ★ 森林F 中有 3 棵树，结点数分别为n_1,n_2,n_3。把F 转换成二叉树B 后，B 的根结点的**右子树**上的结点个数是（　）。A. n_1　B. n_1+n_2　C. n_2+n_3　D. n_3

答：C。B 的根是第一棵树的根，它的**左子树**装第一棵树的其余n_1-1 个结点，**右子树**装剩下的两棵树，共n_2+n_3 个。由此还可得一条常考结论：**森林中树的棵数 = 二叉树中从根出发沿右链能走到的结点数**。

6-11. 已知一棵树的先根序列为 ABCDEF、后根序列为 CBEFDA，则下列说法正确的是（　）。A. C 是 B 的孩子，E、F 都是 D 的孩子 B. A 有三个孩子 C. 该树的度为 3 D. 该树的层次序列为 ABCDEF

答：A。还原过程与二叉树「前序 + 中序」定树同理（依据见 6-28）：先根首字符是根 A，后根末字符也是 A √。A 的第一棵子树在先根里从 B 起、在后根里到 B 止，故为 {B,C}；由先根 BC、后根 CB 知 C 是 B 的孩子。余下 {D,E,F} 是 A 的第二棵子树，先根 DEF、后根 EFD 说明 E、F 都是 D 的孩子。



所以 A 只有两个孩子（B 错）；树的度是 2（C 错）；层次序列是 A B D C E F（D 错）。

6-12. ★ 使用**重量权衡合并规则**的并查集（不含路径压缩），n 个元素任意合并后，树的最大高度的渐进紧确界是（　）。A. $\Theta(1)$　B. $\Theta(\log n)$　C. $\Theta(\sqrt{n})$　D. $\Theta(n)$

答：B。证明见 6-30：每次合并只可能让被挂上去那棵树里的结点深度加 1，而发生这种事时它所在集合的元素个数**至少翻倍**，元素数最多翻$\log_2 n$ 次。等大集合逐层合并可达到$\Theta(\log n)$ 高度；若不用任何优化，最坏就是 D（每次都把大树挂到单元素树下，长成一条链）。

6-13. 对同一片森林，下列两个序列**一般不相同**的是（　）。A. 森林的先根序列 与 对应二叉树的前序序列 B. 森林的后根序列 与 对应二叉树的中序序列 C. 森林的层次序列 与 对应二叉树的层次序列 D. 以上三组都相同

答：C。A、B 是本章的两条基本对应（6-5、6-28）。C 不成立：二叉树的层次遍历是按「离二叉树的根的距离」分层，而转换后**兄弟关系变成了右链**，原本同层的兄弟在二叉树里被拉成了一条向下的链，层号全变了。以 6-20 的森林为例，森林层次序列是 A G B C D H I E F J，

而对应二叉树的层次序列是 A B G C H E D J I F——完全不同。

6-14. 森林F 共有n 个结点，其中m 个是分支结点（至少有一个孩子）。把F 转换为二叉树T（长子作左孩子、下一个兄弟作右孩子）后，T 中左指针为空的结点数和右指针为空的结点数分别是（　）。A. $n-m$，$m+1$　B. $m+1$，$n-m$　C. $n-m$，$n-m$　D. m，$n-m+1$

答：A。**左指针为空** 当且仅当 在F 中没有孩子 当且仅当 是叶结点，共$n-m$ 个。所以「F 的叶结点数等于T 中左指针为空的结点数」。**右指针为空** 当且仅当 是兄弟中的最后一个：每个分支结点的孩子里恰有一个排在最后，共m 个；森林各棵树的根也互为兄弟，最后一棵树的根再算 1 个，合计$m+1$。核对：空指针总数$(n-m)+(m+1)=n+1$，与二叉树的空指针公式（5-33 推论）一致。

6-15. 把森林F 转换成二叉树$B(F)$ 后，若结点u 是结点v 的父结点的父结点，则在原森林中u 与v 可能是下列哪些关系？（多选）（　）A. u 是v 的父结点　B. u 与v 是兄弟　C. v 的父结点是u 的兄弟 D. u 的父结点与v 的父结点是兄弟　E. u 是v 的父结点的父结点

答：A、B、C、E。在$B(F)$ 中往下走一步只有两种：**左**＝到长子，**右**＝到下一个兄弟。从u 走两步到v 共四种走法：

走法	在原森林中v 是	选项
左、左	u 的长子的长子（孙子）	E
左、右	u 的长子的下一个兄弟，即u 的孩子	A
右、左	u 的下一个兄弟的长子（侄子）	C
右、右	u 之后第二个兄弟	B

D 是「堂兄弟」：要从u 先上到父结点、再横到兄弟、再下到孩子，而$B(F)$ 里从u 出发只能下到自己的长子或横到自己之后的兄弟，到不了别的分支下面去。（3000 片随机森林逐对枚举，确实只出现这四种关系。）

三、辨析与简答

6-16. 辨析：「因为森林和二叉树可以一一对应，所以它们是同一种数据结构。」

答：错。一一对应说的是**存储表示可以互相转换**，不等于逻辑结构相同：- 森林中一个结点可以有任意多个孩子，孩子之间是**兄弟关系**；二叉树中每个结点至多两个孩子，且**左右有别**。- 转换后**语义变了**：二叉树里的「右孩子」在森林里是「兄弟」而不是「孩子」，所以在二叉树上求出的「高度」并不是森林的高度。- 森林没有中根遍历，二叉树有中序遍历——**运算集合本身就不同**。

正确的说法是：「**左子/右兄**」**二叉链是森林的一种存储结构**，它让森林能复用二叉树的算法。

6-17. ★ 简答：为什么树（森林）没有「中根遍历」？先根、后根与对应二叉树的前序、中序 为什么会一一对应？

答：**(1)没有中根遍历**：中序的定义是「先左子树、再根、后右子树」，前提是子树恰好分成根的左右两半。树的结点可以有m 棵子树，「根在第几棵子树之后访问」没有自然的选择，所以中根次序无法定义。

(2) 对应关系的来历：设森林F 的第一棵树为T_1、其余为F'。按左子/右兄转换后，T_1 的诸子树构成的森林成了$B(F)$ 的**左子树**，F' 成了**右子树**。- **先根次序**：访问T_1 的根 → 周游T_1 的子树森林 → 周游F'，正是二叉树的「根 → 左 → 右」，即**前序**；- **后根次序**：周游T_1 的子树森林 → 访问T_1 的根 → 周游F'，正是二叉树的「左 → 根 → 右」，即**中序**。

严格证明见 6-28（对结点数作归纳）。

6-18. 简答：比较树的三种链式存储（子结点表、左子/右兄二叉链、父指针表），各自擅长什么运算？

答：

表示法	存什么	擅长	不擅长
子结点表	每个结点带一张孩子链表	**自上而下**：找某结点的全部孩子、按层遍历	找父结点要全表搜索
左子/右兄二叉链	每结点两个指针：长子、下一兄弟	结点大小固定、能直接复用二叉树算法；**深度优先周游**	找第k 个孩子要顺着兄弟链走k 步；找父结点仍不便
父指针表	每结点只存父结点下标	**自下而上**：`Find` 根、判断同集合、**并查集**	找孩子要全表搜索，无法高效自上而下遍历

一句话：**指针指向哪边，哪边的运算就快**。本章小结说「左子/右兄二叉链是采用得最多的表示法」，是因为它结点大小固定、又能复用二叉树的整套算法。

6-19. ★ 并查集的另一种常见存法：数组 `parent[1..n]` 中，非根结点存父结点编号，**根结点存「−（该集合的元素个数）」**。现有 10 个元素：

```
i          1  2  3  4  5  6  7  8  9  10
parent[i]  3  6 -5  6 -1 -4  3  2  7  9
```

(1) 画出它表示的森林，写出各等价类；

(2) 采用**重量权衡合并** + **路径压缩**，依次执行 `Union(10, 8)`、`Union(5, 4)`，写出每一步之后的数组；

(3) 这种存法比 6-23 的「parent 加 weight 两个数组」好在哪里?

答：(1)负数所在的位置就是根：parent[3] = -5、parent[6] = -4、parent[5] = -1，共三棵树。

```
   3          6        5
  / \        / \
 1   7      2   4
     |      |
     9      8
     |
     10
```

等价类为$\{1,3,7,9,10\}$（5 个）、$\{2,4,6,8\}$（4 个）、$\{5\}$（1 个）；负数的绝对值正好是各树的元素个数，$5+4+1=10$ √。

(2)

操作	两次 Find（含路径压缩）	合并	parent[1..10]
Union(10, 8)	Find(10) 走$10\to 9\to 7\to 3$，把 10、9 直接挂到 3；Find(8) 走$8\to 2\to 6$，把 8 直接挂到 6	5 个 > 4 个，6 挂到 3 下，parent[3] $\leftarrow -5+(-4)=-9$	3 6 -9 6 -1 3 3 6 3 3
Union(5, 4)	Find(5) = 5；Find(4) 走$4\to 6\to 3$，把 4 直接挂到 3	1 个 < 9 个，5 挂到 3 下，parent[3] $\leftarrow -10$	3 6 -10 3 3 3 3 6 3 3

最终只剩一棵树：3 的孩子是 1、4、5、6、7、9、10，6 的孩子是 2、8，共 3 层。**两个细节**：第一步压缩 Find(8) 时根还是 6，所以 8 挂到的是 6 而不是 3；第二步 Find(4) 只压缩 4 自己这条路径，2、8 不在路径上，仍挂在 6 下——**路径压缩只动查找路径上的结点**。本题没有出现重量相等的情形；若出现，按 6-23 的约定把值大的根挂到值小的根下。

(3) **一个数组同时回答两个问题**：parent[x] < 0 就是根，不必再约定 0 或 −1 作根的标记；根上的绝对值就是重量，省掉了 weight 数组的n 个整数。合并时一句 parent[r] ← parent[r] + parent[s] 就更新了新根的重量（两个负数相加仍是负数，仍标记着根）。非根结点不需要知道自己子树的大小，所以丢掉这部分信息不影响算法。

四、算法设计与分析

本组题统一使用「左子/右兄」表示：

```
结点 TNode
    data       结点数据
    child      指向第一个孩子（长子），无孩子为 nullptr
    sibling    指向下一个兄弟，无兄弟为 nullptr
```

6-20. ★ 已知森林F由两棵树组成：第一棵的根是 A，A 的孩子依次是 B、C、D，C 的孩子是 E、F；第二棵的根是 G，G 的孩子是 H、I，H 的孩子是 J。(1) 画出F；(2) 写出先根、后根、层次三种序列；(3) 画出F对应的二叉树，并写出它的前序、中序序列，与 (2) 对照。

答：(1) 森林F：

```
      A                G
    / | \              / \
   B  C  D            H   I
     / \               |
    E   F              J
```

(2) 三种序列：- 先根（访问根 → 依次周游各子树 → 再周游其余的树）：A B C E F D G H J I- 后根（先周游各子树 → 访问根 → 再周游其余的树）：B E F C D A J H I G- 层次（按离各自树根的距离分层，同层从左到右）：A G B C D H I E F J

(3) 对应二叉树（左子 = 长子，右兄 = 下一个兄弟）：

```
A
├── 左: B
│       └── 右: C
│               ├── 左: E
│               │       └── 右: F
│               └── 右: D
└── 右: G
        └── 左: H
                ├── 左: J
                └── 右: I
```

逐点说明：A 的左孩子是长子 B、右孩子是下一棵树的根 G；B 无孩子，右孩子是兄弟 C；C 的左孩子是长子 E、右孩子是兄弟 D；E 的右孩子是兄弟 F；G 的左孩子是 H、无右兄弟；H 的左孩子是 J、右孩子是兄弟 I。

- 二叉树**前序**：A B C E F D G H J I = 森林的**先根序列** √
- 二叉树**中序**：B E F C D A J H I G = 森林的**后根序列** √
- 二叉树**层次**：A B G C H E D J I F，与森林的层次序列 A G B C D

H I E F J **不同**（见 6-13 的说明）。

6-21. ★ 在「左子/右兄」表示下，写出森林的**先根、后根**周游算法（递归）和**层次**周游算法（用队列），并分析代价。

答：注意「左子/右兄」里 sibling 链上的结点是**同一层的兄弟**，所以递归要沿 child 下去、沿 sibling 横着走。

```
PreRoot(p)                      // 先根：根 → 各子树 → 其余的树
    while p ≠ nullptr
        访问 p
        PreRoot(p->child)
        p ← p->sibling

PostRoot(p)                     // 后根：各子树 → 根 → 其余的树
    while p ≠ nullptr
        PostRoot(p->child)
        访问 p
        p ← p->sibling

LevelOrder(p)                   // 层次：用队列
    Q ← 空队列
    while p ≠ nullptr           // 先把森林里所有树的根全部入队
        Q.EnQueue(p);  p ← p->sibling
    while not Q.Empty()
        q ← Q.DeQueue()
        访问 q
        c ← q->child            // 把 q 的全部孩子依次入队
        while c ≠ nullptr
            Q.EnQueue(c);  c ← c->sibling
```

代价：三者都是每个结点访问一次，时间$\Theta(n)$。空间：递归版是栈深$O(h)$（h 为树高，退化时$O(n)$）；层次版是队列长度$O(w)$（w 为森林的最大宽度，最坏$O(n)$）。

评分要点：(1) 两个递归里的 `while p ≠ nullptr` 不能少——它负责横着走完兄弟链，漏掉就只周游了第一棵树；(2) 层次遍历必须先把**所有树根**入队，森林不止一棵树；(3) 要说明「访问」的位置差别正是先根与后根的唯一区别。

6-22. ★ 算法填空：多叉树的**非递归后根周游**。设树另提供操作 `Child(p, k)`：返回结点 p 的第k 个孩子（k 从 1 起），不存在时返回 `nullptr`。栈中每条记录是二元组 `(p, k)`，含义是「结点 p 接下来该处理它的第k 个孩子」。

```
PostOrder(root)
    if root = nullptr then return
    S ← 空栈;  S.Push((root, 1))
    while not S.Empty()
```

```
    (p, k) ← S.Pop()
    c ← Child(p, k)
    if ____(①)____
        访问 p
    else
        S.Push(____(②)____)
        S.Push(____(③)____)
```

(1) 补全空缺；(2)② 与 ③ 两句能否交换次序？(3) 设一棵树的根 A 有孩子 B、C、D，C 有孩子 E、F，D 有孩子 G。写出访问序列，以及访问 E 时栈中的记录（自底向上）；(4) 分析代价。

答：(1) ① `c = nullptr`　② `(p, k + 1)`　③ `(c, 1)`

后根次序要求「所有孩子都处理完才访问自己」。记录里的k就是 5-23 末尾说的**标记**：它记住这个结点已经处理到第几个孩子。对照递归写法 `for k ← 1 to p 的度数: PostOrder(Child(p, k)); 然后访问 p`，k 正是循环变量，也就是第 3 章「用栈消除递归」里要保存的**返回点**（3-16）。

(2) **不能交换**。栈后进先出，必须先压 `(p, k+1)`、再压 `(c, 1)`，孩子 c 才会先出栈，它的整棵子树处理完之后才轮到 p 的下一个孩子。交换之后 p 的记录先出栈，会一路跳过所有孩子、先访问 p 本身——第 (3) 问那棵树会最先访问 A，后根次序全乱。

(3) 访问序列 `B E F C G D A`。访问 E 时栈自底向上为 `(A, 3)`、`(C, 2)`：A 下一个该处理第 3 个孩子 D，C 下一个该处理第 2 个孩子 F——栈里恰好是从根到 E 这条路径上的祖先。

(4) 结点 p 共出栈「度数 + 1」次（每个孩子一次，最后一次用来访问自己），总出栈次数为$\sum_p(\text{度数}_p+1)=(n-1)+n=2n-1$。若 `Child(p, k)` 是$O(1)$（如孩子存在数组里），总时间$O(n)$；栈深不超过树的层数$h$，空间$O(h)$。

实现时要留意：若树用本组的「左子/右兄」存储，`Child(p, k)` 得从长子沿 `sibling` 走$k-1$步，每次$O(k)$，总时间退化成$O(\sum_p \text{度数}_p^2)$，一个根带$n-1$个孩子时为$O(n^2)$。改法是记录里**不存序号k，直接存「下一个要处理的孩子」的指针**：初值为 `p->child`，每次推进为 `c->sibling`，又回到$O(n)$。另一条路：森林的后根次序就是对应二叉树的中序（6-5），所以在左子/右兄结构上直接套 5-23 的中序框架（`left` 换成 `child`、`right` 换成 `sibling`）也行。

6-23. ★ 写出并查集的 `Find` 与 `Union`，要求实现**重量权衡合并**与**路径压缩**，并分析代价。

答：用数组 `parent[0..n-1]` 存父结点下标，根的 `parent` 记为-1；另用 `weight[r]` 记录以r为根的集合的**元素个数**。

```
MakeSet(n)
    for i ← 0 to n - 1
        parent[i] ← -1;  weight[i] ← 1

Find(x)                                      // 带路径压缩
    r ← x
    while parent[r] ≠ -1                      // 第一趟：找到根
        r ← parent[r]
    while parent[x] ≠ -1                     // 第二趟：把沿途结点全部直接挂到根上
        next ← parent[x]
        parent[x] ← r
        x ← next
    return r

Union(a, b)                                  // 重量权衡合并
    ra ← Find(a);  rb ← Find(b)
    if ra = rb then return                   // 本来就同一集合，什么也不做
    if weight[ra] < weight[rb] or (weight[ra] = weight[rb] and ra > rb)
        parent[ra] ← rb;  weight[rb] ← weight[rb] + weight[ra]
    else
        parent[rb] ← ra;  weight[ra] ← weight[ra] + weight[rb]
```

两条优化各管什么：- **重量权衡合并**：小集合挂到大集合下，保证树高$O(\log n)$（证明见 6-30）；- **路径压缩**：`Find` 的时候顺手把这条路径上的结点全部直接接到根上，**在根尚未因后续合并而改变时，再查这些结点只需一步**。注意压缩发生在**查找**时，不是合并时。

代价：只用重量权衡合并时，单次操作最坏$O(\log n)$；只用路径压缩时，单次 `Find` 最坏仍可能是$O(n)$（树可以先被合并成一条链），但m次操作摊还下来每次是$O(\log n)$量级；两条同时用时，m 次操作的总代价是$O(m\,\alpha(n))$，α 是反 Ackermann 函数，实际中不超过 4，**这是摊还界，单次操作最坏仍为**$O(\log n)$。初始化另需$\Theta(n)$。

易错点：`Union` 里必须先 `Find` 出两个根再比较——直接对 `a`、`b` 操作是错的；权值并列时要有确定的取舍规则（本课程约定**值大的根挂到值小的根下**），否则画出的树形对不上答案。

6-24. ★ 设有 8 个元素$1..8$，依次读入等价偶对$(1,2)$，$(3,4)$，$(5,6)$，$(7,8)$，$(2,4)$，$(6,8)$，$(4,8)$。采用**重量权衡合并 + 路径压缩**，写出每一步之后的父指针数组，并给出最终的等价类。

答：约定根的父记为0；权值并列时**值大的根挂到值小的根下**。

步	偶对	Find 结果	合并动作	parent[1..8]
0	初始	—	—	0 0 0 0 0 0 0 0
1	(1,2)	1, 2（权 1、1，并列）	2 挂到 1 下，$w_1 = 2$	0 1 0 0 0 0 0 0
2	(3,4)	3, 4（并列）	4 挂到 3 下，$w_3 = 2$	0 1 0 3 0 0 0 0
3	(5,6)	5, 6（并列）	6 挂到 5 下，$w_5 = 2$	0 1 0 3 0 5 0 0
4	(7,8)	7, 8（并列）	8 挂到 7 下，$w_7 = 2$	0 1 0 3 0 5 0 7
5	(2,4)	1, 3（权 2、2，并列）	3 挂到 1 下，$w_1 = 4$	0 1 1 3 0 5 0 7
6	(6,8)	5, 7（并列）	7 挂到 5 下，$w_5 = 4$	0 1 1 3 0 5 5 7
7	(4,8)	1, 5（权 4、4，并列）	5 挂到 1 下，$w_1 = 8$；**路径压缩**把 4 直接挂到 1、8 直接挂到 5	0 1 1 1 1 5 5 5

第 7 步的细节：Find(4) 走$4 \to 3 \to 1$，压缩后 parent[4] ← 1；Find(8) 走$8 \to 7 \to 5$，压缩后 parent[8] ← 5。之后才做合并 parent[5] ← 1。

最终父指针数组：parent[1..8] = 0 1 1 1 1 5 5 5，森林形如

```
          1
    /  /  |  \
   2  3   4   5
            / | \
           6  7  8
```

最终只有一个等价类 $\{1,2,3,4,5,6,7,8\}$，树高只有 3 层（最深路径$6 \to 5 \to 1$）。若不用任何优化、每次都把前一个根挂到后一个根下，这组输入会长成以 8 为根、最深路径$1 \to 2 \to 4 \to 8$ 的树，树高 4 层（parent[1..8] = 2 4 4 8 6 8 8 0），在这么小的数据上还看不出差距；但偶对若按$(1,2),(2,3),\cdots,(7,8)$ 的次序读入，不加优化就会长成$1 \to 2 \to \cdots \to 8$ 的一条链，树高 8 层，之后每次 Find(1) 都要走$O(n)$ 步；而重量权衡合并下这 7 个偶对只会长出一棵以 1 为根、高度 2 层的树 ——这就是两条优化的价值。

6-25. ★ 已知一片森林的**带双标记的先根次序**表示，写出还原成「左子/右兄」链式结构的算法，并对 6-20 那片森林给出它的双标记序列。

答：**基本思想**——先根次序有一条性质：**任何结点的子树都紧跟在它后面**，子树排完才轮到它的下一个兄弟。于是「谁是某个结点的右兄弟」要等它整棵子树扫完才知道，用一把**栈**记着这笔欠账：扫到「有下一个兄弟」的结点就把它压栈；扫到「没有孩子」的结点（说明一棵子树到头了）就弹出一个，把**下一个**建出来的结点接成它的右兄弟。

```
BuildFromDualTag(a[0..n-1])       // 每项有 data、has_child、has_sibling;
↪ 假设标记合法
    if n = 0 then return nullptr
    S ← 空栈;  root ← nullptr
    owner ← nullptr;  side ← ROOT   // 下一个结点应填入的位置
    for i ← 0 to n - 1
        p ← 新结点 (a[i].data);  p->child ← p->sibling ← nullptr
        if side = ROOT then root ← p
        else if side = CHILD then owner->child ← p
        else owner->sibling ← p
        if a[i].has_sibling then S.Push(p)
        if a[i].has_child
            owner ← p;  side ← CHILD
        else if not S.Empty()
            owner ← S.Pop();  side ← SIBLING
        else
            if i ≠ n - 1 then 报错并终止     // 结构已结束但还有多余结点
            return root
    报错并终止                                // 序列结束但仍有孩子或兄弟待填
```

6-20 那片森林的双标记序列（先根次序 `A B C E F D G H J I`，`ltag = 0` 表示**有孩子**，`rtag = 0` 表示**有下一个兄弟**）：

```
先根次序   A  B  C  E  F  D  G  H  J  I
ltag       0  1  0  1  1  1  0  0  1  1      0 = 有孩子
rtag       0  0  0  0  1  1  1  0  1  1      0 = 有下一个兄弟
```

这两行最容易抄反。原书图 6.15 的三行从上到下印的是 `rtag` / `info` / `ltag`——**`ltag` 在最下面**，照着「第一行就是 ltag」抄就会把两行互换。按定义核一遍即可：`B` 是叶结点，`ltag` 必须是 1；`B` 有兄弟 `C`，`rtag` 必须是 0。顺带一句，**先根序列相同的森林可以不止一片**——互换这两行还原出来的森林里，`C` 会变成 `B` 的孩子而不是兄弟，可它的先根序列**恰好还是** `A B C E F D G H J I`。所以这类题自检时要**同时对后根序列**（本题是 `B E F C D A J H I G`）：先根加后根才唯一确定一棵树，只对先根是查不出结构错的。

逐个核对：`A` 有孩子 `B`（ltag 0）、有下一棵树的根 `G` 作兄弟（rtag 0）；`B`

是叶（ltag 1）、有兄弟 C（rtag 0）；C 有孩子 E、有兄弟 D；E 是叶、有兄弟 F；F、D 都是又无孩子又无兄弟（1 1）；G 有孩子 H、无兄弟（0 1）；H 有孩子 J、有兄弟 I（0 0）；J、I 都是 1 1。

代价：时间$\Theta(n)$（每个结点入栈至多一次），辅助栈空间$O(h)$，输出结点本身占$\Theta(n)$ 空间（h 为森林层数）。

这类顺序存储题的通用解法：不必死记四种表示法（带右链先根、带双标记先根、带度数后根、带双标记层次），只要抓住一条—— **它们都是「某种遍历序列 + 刚好够还原结构的附加信息」**。做题时先判断是哪种遍历次序，再想「这点附加信息够不够定出父子和兄弟关系」，现推比背要可靠得多。

6-26. 在「左子/右兄」表示下，写出求森林中结点总数与森林高度的算法。

答：仍是「沿 child 下去、沿 sibling 横着走」，但**两条链的含义不同**：走 child 层号加一，走 sibling 层号不变——这是本题唯一的陷阱。

```
Count(p)                          // 结点总数
    if p = nullptr then return 0
    return 1 + Count(p->child) + Count(p->sibling)

Height(p)                         // 森林高度（层数；空森林为 0）
    if p = nullptr then return 0
    return max(1 + Height(p->child),  Height(p->sibling))
                ↑ 往下走才加一          ↑ 兄弟仍在同一层，不加
```

代价：时间$\Theta(n)$，空间$O(n)$（最坏的递归深度）。**对照易错写法**：把 `Height` 写成 `1 + max(Height(child), Height(sibling))`，那求出来的是**对应二叉树的高度**，不是森林的高度—— 这正是 6-16 里「转换之后语义变了」的具体例子。

6-27. ★ 一棵树的**镜面映射**指把每个结点的子结点次序反转。输入输出树时常把树转成对应的二叉树，并给只有一个子结点的分支结点补一个虚子结点 `$`，形成「**伪满二叉树**」（每个分支结点恰有两个子结点）；再对这棵二叉树作**前序遍历**，内部结点标 0、叶结点标 1，虚结点也输出。现给定这样的前序序列，设计算法输出**原树的镜面映射的广度优先（层次）序列**。以 `x0 p0 $1 q0 s0 $1 t1 r1 $1` 为例，应输出 `x r q p t s`。

答：三步，每一步都是学过的东西。

第一步：按前序递归建「伪满二叉树」。伪满的意思是**每个内部结点恰有两个孩子**，所以读到标 0 的记号就递归读左、再递归读右；读到标 1 的记号就是叶子，不再往下读。`$` 建成空指针。

```
Build(tokens)                         // tokens 是只能顺序取的记号流
    t ← tokens.next()
    if t.ch = '$' then return 空      // 虚结点，它一定标 1，不再往下读
```

```
    建结点 node(t.ch)
    node.left ← node.right ← 空
    if t.flag = 0                    // 内部结点，恰有两个孩子
        node.left  ← Build(tokens)
        node.right ← Build(tokens)
    return node
```

第二步：按「左子/右兄」把二叉树读回成树。不必真的另建一棵——只要记住 `node.left` 是**长子**、`node.right` 是**下一个兄弟**（6-4），于是「结点p 的全部孩子」就是**从 `p.left` 出发的那条右链**。

第三步：层次遍历，孩子从右往左入队。镜面映射是把每个结点的孩子反序，而层次遍历只关心「孩子按什么顺序入队」—— 所以**根本不用真的去反转这棵树**，入队时倒着放就等价了。

```
MirrorLevelOrder(root)
    if root = 空 then return
    Q ← 空队列;  Q.EnQueue(root)
    while not Q.Empty()
        p ← Q.DeQueue()
        输出 p.ch
        C ← 空表
        for c ← p.left;  c ≠ 空;  c ← c.right      // 收集 p 的全部孩子
            C.append(c)
        for j ← |C| - 1 downto 0                   // 倒着入队 = 镜面
            Q.EnQueue(C[j])
```

对例子走一遍：建出的二叉树是 `x.left=p`、`x.right= 空`；`p.left= 空`、`p.right=q`；`q.left=s`、`q.right=r`；`s.left= 空`、`s.right=t`。按左子右兄读回成树：

```
x
├── p
├── q
│   ├── s
│   └── t
└── r
```

层次遍历、孩子倒着入队：输出 `x`，把 `r q p` 入队 → 输出 `r`（无孩子）→ 输出 `q`，把 `t s` 入队 → 输出 `p`（无孩子）→ 输出 `t` → 输出 `s`。结果 **`x r q p t s`** ✓

代价：建树每个记号读一次，$\Theta(N)$；层次遍历每个结点进出队各一次、每条兄弟边走一次，$\Theta(N)$。合计**时间**$\Theta(N)$；空间是树本身的$\Theta(N)$、建树递归栈的$O(N)$ 加队列的$O(w)$（N 为记号数，w 为树的最大宽度）。

三**个容易丢分的地方**：1. `$` 也在输入里，**必须读掉**，否则后面整串错位；2. 判据是「标 0 就读两个孩子、标 1 就不读」，**不要写成「读到 `$` 才停」**；3. **镜面不用真反转**——在入队顺序上做文章，这是本题最值钱的一句话。

五、数学证明

6-28. ★ 证明：按先根次序周游森林F 得到的序列，等于按前序周游F 对应二叉树$B(F)$ 得到的序列；按后根次序周游F 的序列，等于按中序周游$B(F)$ 的序列。

证明. 对森林的结点数n 作归纳。$n=0$ 时两边都是空序列。

设$n>0$，F 的第一棵树为T_1，其根为r，r 的子树构成森林F_1，F 中其余的树构成森林F_2。按左子/右兄的转换规则：$B(F)$ 的根是r，**左子树是**$B(F_1)$，**右子树是**$B(F_2)$。

先根 / 前序：由定义，森林的先根序列$=r\cdot$先根$(F_1)\cdot$先根(F_2)；二叉树的前序序列$=r\cdot$前序$(B(F_1))\cdot$前序$(B(F_2))$。F_1、F_2 的结点数都小于n，由归纳假设两组括号内逐项相等，故整体相等。

后根 / 中序：森林的后根序列$=$ 后根$(F_1)\cdot r\cdot$后根(F_2)；二叉树的中序序列$=$ 中序$(B(F_1))\cdot r\cdot$中序$(B(F_2))$。同样由归纳假设得两者相等。∎

这条结论的用法：凡是「已知森林的先根和后根序列，还原森林」的题，都可以先翻译成「已知二叉树的前序和中序，还原二叉树」——**后者有唯一解**（第 5 章 5-18），所以前者也有唯一解。

6-29. 证明：一棵有n 个结点的K 叉树（每结点固定K 个孩子指针），空指针数为$n(K-1)+1$。

证明. 指针域总数为nK。每个非空指针恰好指向一个结点，且每个结点至多被一个指针指到（它的父结点指来的那一个）；除根之外每个结点都恰好被指到一次，根一次也没有。故非空指针数$=n-1$，空指针数$=nK-(n-1)=n(K-1)+1$。∎

$K=2$ 时得$n+1$，与第 5 章的结论一致；n 个结点的K 叉树里，K **越大，浪费在空指针上的空间比例越高**（$\frac{n(K-1)+1}{nK}\to 1-\frac{1}{K}$），这正是一般的树不用「固定$K$ 个指针」而改用左子/右兄的原因。

6-30. ★ 证明：采用**重量权衡合并规则**（不含路径压缩）的并查集，任何一棵树的高度不超过$\lfloor \log_2 n \rfloor + 1$ 层。

证明. 考察任意一个元素x 的深度。x 的深度只在一种情况下增加 1：x 所在的树被整体挂到另一棵树的根下。按重量权衡规则，这要求**被挂的这棵树的元素个数不超过另一棵**，因此合并之后，x 所在集合的元素个数**至少是原来的 2 倍**。

x 最初所在的集合只有 1 个元素，最终至多n 个，元素个数最多翻

倍$\lfloor \log_2 n \rfloor$ 次，所以x 的深度至多增加$\lfloor \log_2 n \rfloor$，即深度$\le \lfloor \log_2 n \rfloor$、层数$\le \lfloor \log_2 n \rfloor + 1$。∎

推论：`Find` 的最坏代价$O(\log n)$，`Union` 也是$O(\log n)$（它先要做两次 `Find`）。再加上路径压缩，m 次操作的摊还代价降到近乎常数。

6-31. 证明：树中所有结点的度数之和等于结点数减 1，即$\sum_i i \cdot n_i = n - 1$。

证明. 树中的每一条边都可以唯一地记为「某个结点射向它的某个孩子」，因此边数等于所有结点的度数之和$\sum_i i \cdot n_i$。另一方面，树中除根之外的每个结点都恰好有一条边从其父结点射入，且根没有入边，因此边数也等于$n-1$。两者相等。∎

移项即得 6-2 的公式$n = 1 + \sum_i i \cdot n_i$，它是本章填空题的主力工具。

6-32. ★ 满k 叉树指每个结点要么是叶结点、要么恰有k 个孩子。(1) 证明：非空满k 叉树若有L 个叶结点，则内部结点数为$\frac{L-1}{k-1}$；并由此说明L 必须满足「$(k-1)$ 整除$(L-1)$」。(2) 证明：高度为h 层（只有根时$h=1$）的满k 叉树，结点数至少$k(h-1)+1$、至多$\frac{k^h-1}{k-1}$，且两个界都能取到。

(1) 证明. 设内部结点数为n、总结点数$N = n + L$。

数边：每条边都从某个内部结点射向它的一个孩子，而每个内部结点恰有k 个孩子，故边数$E = nk$；另一方面除根之外每个结点恰有一条入边，故$E = N - 1$。两式联立：

$$nk = n + L - 1 \Rightarrow L = n(k-1) + 1 \Rightarrow n = \frac{L-1}{k-1}$$

而n 是结点个数、必为整数，所以$(k-1)$ 必须整除$(L-1)$。∎

归纳版：$n = 0$ 时树只有一个根结点、它本身就是叶，$L = 1 = 0 \cdot (k-1) + 1$ √（$n = 1$ 时根恰有k 个叶孩子，$L = k$，也由下面的归纳步得到）。设结论对m 个内部结点成立，任取一棵有$m+1$ 个内部结点的满k 叉树，取一个深度最大的内部结点，它的k 个孩子必全是叶；删去这k 个叶，它变回叶结点，得到有m 个内部结点的满k 叉树。反过来看就是**把某个叶结点变成内部结点**：减少 1 个叶、新增k 个叶，净增$k-1$ 个，故$L_{m+1} = m(k-1) + 1 + (k-1) = (m+1)(k-1) + 1$ √

(2) 证明. 上界：第i 层（根为第 0 层）至多k^i 个结点，h 层合计至多$\sum_{i=0}^{h-1} k^i = \frac{k^h-1}{k-1}$，各层全满时取到（6-6）。

下界：树有h 层，就有一条从根到第$h-1$ 层某结点的路径，路径上除末端外的$h-1$ 个结点都有孩子，因而都是内部结点，故内部结点数$n \ge h - 1$。由 (1) 的$L = n(k-1) + 1$，总结点数

$$N = n + L = nk + 1 \ge k(h-1) + 1$$

取到下界的树：一根由$h-1$个内部结点连成的「主干」，每个主干结点除了往下接主干，另挂$k-1$片叶，主干末端再挂k片叶。∎

$k=2$时下界为$2h-1$：高h层的满二叉树至少$2h-1$个结点，比一般二叉树的下界h多出将近一倍，因为每往下伸一层，都得同时多带一片叶。**最少结点数不是只算最长那条路径**，这是本问最常见的错法。

一个直接的用处：$k=3$时$(L-1)$必须是 2 的倍数，所以满 3 叉树的叶结点数只能是$1,3,5,7,\cdots$（奇数）。问「有没有恰好 6 个叶结点的满 3 叉树」，答案是**没有**——不必去试着画。

别和 6-29 搞混，两条结论不是一回事：

	说的是什么	结果
本题	**满**k叉树里，n个**内部结点**对应多少**叶结点**	$n(k-1)+1$
6-29	**任意**k叉树里，n个**结点**留下多少**空指针**	$n(k-1)+1$

表达式一模一样，是因为两者算的都是「nk个槽位减掉$n-1$条被占用的边」——一个把剩下的槽位看成叶结点，另一个看成空指针。$k=2$时分别退化成「满二叉树叶数 = 分支结点数 + 1」和「n个结点的二叉树有$n+1$个空指针」（5-33）。

6-33. 设u、v是森林F中两个不同的结点。证明：u是v的祖先，当且仅当在先根序列中u在v之前、且在后根序列中u在v之后。

证明. 必要性：u是v的祖先，则v在以u为根的子树中。先根周游访问u之后才进入u的子树，所以先根序列中u在v前；后根周游要周游完u的全部子树才访问u，所以后根序列中v在u前。

充分性（证逆否命题）：设u不是v的祖先，分两种情形。

1. v是u的祖先：把必要性用在v、u上，先根序列中v在u前，与「u在v前」矛盾。
2. 二者互不为祖先：若它们在同一棵树中，取最近公共祖先w，u、v分别落在w的两个不同孩子c_1、c_2的子树里；若在不同的树中，就取它们各自所在的两棵树。两种情况下，u、v都分属两棵互不相交、有先后次序的兄弟子树（或兄弟树）。先根与后根周游都是**按兄弟次序整棵整棵地**周游子树的，排在前面的那棵子树的全部结点，在两种序列里都排在后面那棵的全部结点之前。所以u若在先根序列中排在v前，在后根序列中也排在v前，两个条件不能同时成立。

两种情形都与条件矛盾，故u是v的祖先。∎

用处：预先记下每个结点在先根、后根序列中的位置，以后判断祖先关系只需比较两次，$O(1)$；对比 5-29 沿父指针往上爬的$O(h)$。把「先根」「后根」换成二叉树的「前序」「后序」，结论与证明同样成立。

模拟卷（120 分钟，100 分）

题目覆盖前面的核心知识点，部分题型有所复现，可当作一次完整自测。答案集中在各组题目之后。

一、填空题（每题 2 分，第 1 题两空各 1 分，共 20 分）

1. 抽象数据类型由数据对象、数据关系和 ____ 三部分组成，其中**不包括** ____。
2. 在长度为 100 的顺序表中删除一个元素，等概率下平均需要移动 ____ 个元素。
3. 容量为 10 的循环队列（牺牲一格），`front = 7`、`rear = 2`，队中有 ____ 个元素。
4. 6 个元素依次入栈，可能的出栈序列共 ____ 种。
5. 模式 `P = "abaabab"` 的未优化 next 数组是 ____。
6. 一棵二叉树有 20 个叶结点、15 个度为 1 的结点，则它共有 ____ 个结点。
7. 有 100 个结点的完全二叉树，叶结点有 ____ 个。
8. 对 6 个字符构造 Huffman 树，树中共有 ____ 个结点。
9. 一片森林有 4 棵树、共 30 个结点，转换成二叉树后，根结点的右子树上有 ____ 个结点（设第一棵树有 9 个结点）。
10. 有n 个结点的 3 叉树（每结点固定 3 个孩子指针），空指针共 ____ 个。

答：1. **基本运算（操作集合）**；**数据的存储结构** 2. **49.5**（$\frac{n-1}{2}=\frac{99}{2}$） 3. **5**（$(2-7+10)\bmod 10=5$） 4. **132**（Catalan：$1,2,5,14,42,132$） 5. **`-1 0 0 1 1 2 3`**（逐位：a → 0；ab → 0；aba → 1；abaa → 1；abaab → 2；abaaba → 3，因为 aba 既是前缀又是后缀） 6. **54**（$n_2=n_0-1=19$，$n=20+15+19=54$） 7. **50**（$\lceil 100/2\rceil=50$） 8. **11**（$2n-1=2\times 6-1$） 9. **21**（$30-9=21$） 10. $2n+1$（$n(K-1)+1,\ K=3$）

二、选择题（每题 2 分，共 16 分）

1. 下列数据结构中，**不属于**线性结构的是（　）。A. 栈　B. 队列　C. 串　D. 二叉树
2. 入栈序列为 `1,2,3,4,5`，下列**不可能**是出栈序列的是（　）。A. `5,4,3,2,1`　B. `2,1,4,5,3`　C. `4,1,5,3,2`　D. `1,3,2,5,4`
3. 在带头结点的单链表中，要在 `p` 所指结点**之前**插入结点 `s`，最省事的做法是（　）。A. 从头找到 `p` 的前驱，再插入，$O(n)$ B. 把 `s` 插到 `p` 之后，再交换两者的数据域，$O(1)$ C. 无法做到 D. 直接 `s->next = p`，$O(1)$
4. 循环队列用「另设计数器」方案时，判满条件是（　）。A. `front == rear`　B. `(rear+1)%M == front`　C. `count == M`　D. `count == M-1`
5. 已知二叉树的中序序列为 `DBEAFC`、后序序列为 `DEBFCA`，则前序序列为（　）。A. `ABDECF`　B. `ABCDEF`　C. `ABDEFC`　D. `ABDCEF`
6. 下列序列中是**最大堆**的是（　）。A. `(90, 70, 80, 60, 50, 30)`　B. `(90, 80, 70, 60, 50, 85)` C. `(90, 70, 80, 85, 50, 30)`　D. `(30, 50, 60, 70, 80, 90)`

7. 森林中有n 个结点、k 棵树，转换成的二叉树中，从根出发沿右链走到底能经过（ ）个结点。A. 1　B. k　C. n　D. $n-k$
8. 使用路径压缩的并查集，路径压缩发生在（ ）。A. `Union` 时　B. `Find` 时　C. 初始化时　D. 每次访问数组时

   答：1. **D**。二叉树是层次（树形）结构，一个结点可以有两个后继。2. **C**。`4,1,...`：先输出 4，说明 1、2、3 都在栈里、3 在最上；它们只能按 `3,2,1` 的相对次序出来，而 C 里是 `1,...,3,2`，违反。用判定法则：取$p_1=4, p_2=1, p_4=3$，有$1<3<4$，正是被禁的模式。3. **B**。这是「前插转后插」的经典技巧：先把 `s` 接在 `p` 之后，再交换 `p` 与 `s` 的数据域，效果等价于在 `p` 前插入，代价$O(1)$。D 是错的：只改 `s->next` 而不改 `p` 的前驱，`s` 根本没挂进链里。4. **C**。有了计数器就能存满M 个，不必牺牲一格。5. **A**。后序末为根 `A`；中序中 `A` 左边 `DBE` 是左子树、右边 `FC` 是右子树。左子树后序 `DEB` → 根 `B`，中序 `DBE` → 左 `D`、右 `E`；右子树后序 `FC` → 根 `C`，中序 `FC` → `F` 是 `C` 的左孩子。前序 = `A BDE CF` = `ABDECF`。6. **A**。逐个验$K_i \geq K_{2i+1}, K_{2i+2}$：A 中$90\geq 70, 80$；$70\geq 60, 50$；$80\geq 30$ √。B 错在$K_2=70<K_5=85$；C 错在$K_1=70<K_3=85$；D 是递增序列，它是**最小堆**（递增序列一定满足父$\leq$ 子），而不是最大堆：$K_0=30<K_1=50$ 就已违反。7. **B**。右链就是「兄弟链」，从根沿右链走到底经过的正是森林中各棵树的根，共k 个。8. **B**。路径压缩是在**查找**沿途顺手做的；合并只改一根指针，无路可压。

三、辨析与简答题（每题 6 分，共 24 分）

1. 辨析：「用大O 表示时间复杂度，说的就是算法在最坏情况下的代价。」
2. 简答：链式队列在**出队使队列变空**时，为什么必须同时修改队尾指针？不改会出什么问题？
3. 简答：KMP算法中，为什么目标串的下标可以自始至终不回退？
4. 辨析：「二叉搜索树的中序序列有序，所以堆的中序序列也有序。」

   答：**1.错**。「最好/最坏/平均」挑的是**输入**，「$O/\Omega/\Theta$」定的是**函数的界**，这是两个正交的维度，可以自由组合：「最好情况是$O(n)$」完全合法。单说「复杂度是$O(n^2)$」只承诺「不会比n^2 更糟」，并没有说清是对哪一类输入而言。严谨的写法是把两个维度都点明，例如「最坏情况下为$\Theta(n^2)$」。

   2. 链式队列的 `front` 和 `rear` 分别指向队头和队尾结点。出队时若摘走的正好是最后一个结点，队列变空，`front` 被置空，但 `rear` 仍指着那个**已经被释放的结点**——它成了一根野指针。下一次入队时若用 `rear->next = 新结点` 去接，就会**写进已释放的内存**（典型表现是 AddressSanitizer 报 `heap-use-after-free`，也可能悄悄损坏堆而在别处崩溃）。正确做法：`dequeue` 发现队列变空时，把 `rear` 一并置空。

3. 因为**失配之前已经匹配上的那一段是已知的**：设在模式的位置i处失配，则 `T` 中与 `P[0..i-1]` 对齐的那一段与之逐字符相同。要让模式重新对上，新的对齐位置必须使模式的某个前缀等于 `P[0..i-1]` 的某个后缀，而「最长的这种前缀」的长度正是 `next[i]`——比它更短的对齐也许可行但更晚，比它更长的对齐必然失败。所以直接把模式右移$i-\texttt{next}[i]$位，目标下标停在原地继续比即可（`next[i]=-1` 时目标下标前进一格、模式从头比），**被跳过的那些起点已经被证明不可能成功**，不必回去试。这样 `j` 只增不减，匹配过程$O(n)$。

4. 错。堆只保证**父$\le$子**（最小堆）这种**局部有序**，兄弟之间毫无关系，更不保证「左子树全部小于根、根小于右子树」。例如最小堆 `[1, 2, 3]`（根 1，左 2，右 3）的中序序列是 `2 1 3`，并不有序。堆的用途是$O(1)$取极值、$O(\log n)$增删，**不支持按关键码检索**；要中序有序得用二叉搜索树。

四、算法设计与分析题（每题 10 分，共 30 分）

1. 设计算法判断一棵二叉树是否为**完全二叉树**。写明基本思想，给出算法与复杂度。

答：**基本思想**——用层次遍历（队列），**把空孩子也入队**。完全二叉树的特征是：按层从左到右扫描时，**所有非空结点在前、所有空位在后**，中间不能夹空。于是只要在队列中第一次遇到空指针之后，再出现任何非空结点，就不是完全二叉树。

```
IsComplete(root)
    if root = nullptr then return true
    Q ← 空队列;  Q.EnQueue(root)
    met_null ← false
    while not Q.Empty()
        p ← Q.DeQueue()
        if p = nullptr
            met_null ← true
        else
            if met_null then return false      // 空位之后又出现结点 → 不是
↪   完全二叉树
            Q.EnQueue(p->left)
            Q.EnQueue(p->right)
    return true
```

用到的队列 ADT：`EnQueue(x)` 入队、`DeQueue()` 出队并返回队头、`Empty()` 判空，均为$O(1)$。

正确性：完全二叉树按层次编号后结点连续占据$0..n-1$，因此层次扫描中空位只能出现在末尾；反之若空位后还有结点，说明编号不连续。

代价：最坏情况（树确为完全二叉树）下每个结点与每个空指针各入队出队一次，时间$\Theta(n)$，提前返回时更少，故一般为$O(n)$；队列包含待检查的空位，最大长度与树的最大宽度同阶，空间$O(n)$。

2. 给定一个长为n 的字符串S，设计$O(n)$ 算法求S 的**最长回文前缀**（即最长的、本身是回文的前缀）。写明思想、算法与代价。

答：**基本思想**——把问题转化成特征向量问题。令R 为S 的逆串，构造新串

$$U = S + \# + R$$

其中 # 是不出现在S 中的分隔符。则U **的最长公共真前后缀长度**f **就是**S **的最长回文前缀长度**：U 的前缀来自S 的前缀，U 的后缀来自R 的后缀（即S 前缀的逆），两者相等正说明这个前缀等于自己的逆，即回文。分隔符 # 保证匹配段不会跨过中线（否则长度可能超过n）。

```
LongestPalindromicPrefix(S, n)
    R ← reverse(S)
    U ← S + '#' + R                          // 长度 2n + 1
    用 GetNext 求 U 的未优化特征向量，并多算一项 f = next[2n+1]
    return S[0 .. f-1]                       // 长度为 f 的前缀
```

必须写清的两点：(1) 用的是**未优化**的 next——优化版的数值不再是「最长公共真前后缀长度」；(2) 分隔符不可省，且要保证它不在字母表中。

代价：构造U 为$\Theta(n)$，求特征向量$\Theta(n)$，合计$\Theta(n)$ 时间、$\Theta(n)$ 空间。

例：$S =$ `"abacd"`，$U =$ `"abacd#dcaba"`，$f = 3$，最长回文前缀是`"aba"` √。

3. 某班有n 名学生，现给出若干条「x 与y 在同一个小组」的信息。设计数据结构与算法，支持：(1) 加入一条信息；(2) 查询两名学生是否同组；(3) 统计小组个数。要求每次操作接近$O(1)$，并说明依据。

答：**数据结构**——用**父指针表示的并查集**：数组 `parent[0..n-1]`（根记-1）、`weight[0..n-1]`（以该点为根的集合的元素个数），另用一个整数 `groups` 记录当前小组数。

ADT：

```
MakeSet(n)      功能：n 个学生各成一组；groups ← n
Find(x)         入口：学生编号；出口：所在小组的代表（根），沿途做路径压缩
Union(x, y)     入口：两个编号；功能：合并两个小组（已同组则什么也不做）
SameGroup(x,y)  入口：两个编号；出口：两人是否同组
Count()         出口：当前小组个数
```

```
Union(x, y)
    rx ← Find(x);  ry ← Find(y)
    if rx = ry then return                          // 已同组，小组数不变
    if weight[rx] < weight[ry] then 交换 rx, ry   // 重量权衡：小的挂到大的下
    parent[ry] ← rx
    weight[rx] ← weight[rx] + weight[ry]
    groups ← groups - 1                          // 每成功合并一次，小组数减一

SameGroup(x, y)   return Find(x) = Find(y)
Count()           return groups
```

三**个操作的代价**：- (1)加入信息 = 一次 `Union` = 两次 `Find` 加常数工作；- (2) 查询 = 两次 `Find`；- (3) 统计 = 直接返回 `groups`，$O(1)$——**不要每次去扫一遍数组数根**，那是$O(n)$。

依据：只用重量权衡合并时树高$O(\log n)$（证明见 6-30），故 `Find` 为$O(\log n)$；再加上路径压缩，m 次操作的总代价为$O(m\,\alpha(n))$，α 是反 Ackermann 函数，在现实规模下增长极慢，因此**单次操作的摊还代价接近常数**，单次最坏仍为$O(\log n)$。初始化 `parent`、`weight` 与 `groups` 另需$\Theta(n)$ 时间，空间$\Theta(n)$。

易错点：`groups` 必须在**确实发生了合并**时才减一；对已经同组的一对再 `Union`，小组数不能变。

五、证明题（10 分）

设一棵非空二叉树有n 个结点。证明：它的**扩充二叉树**（把每棵空子树替换为一个外部结点后得到的树）恰有$n+1$ 个外部结点，并由此说明：n 个结点的二叉树用二叉链表存储时，有$n+1$ 个空指针域。

证明. 记原树的结点为**内部结点**，新添的为**外部结点**。

第一步：扩充二叉树是满二叉树。原树的每个结点原先有 0、1 或 2 棵空子树，扩充后每棵空子树都换成了一个外部结点，因此每个内部结点恰有 2 个孩子；外部结点没有孩子。故扩充二叉树中不存在度为 1 的结点。

第二步：数叶结点。设扩充二叉树的外部结点数为m。外部结点即叶结点（$n_0 = m$），内部结点即度为 2 的结点（$n_2 = n$）。由二叉树的性质$n_0 = n_2 + 1$（证明见 5-33）得

$$m = n + 1$$

第三步：与空指针的对应。二叉链表中，每个结点有两个指针域，共$2n$ 个；非空指针恰好指向除根之外的每个结点，共$n-1$ 个；故空指针域有$2n-(n-1) = n+1$ 个。而每一个空指针正对应扩充时添上

的一个外部结点，两者一一对应，数目同为$n+1$。∎

这条结论的意义：**二叉链表里有一半以上的指针是空的**（$n+1$ 个空、$n-1$ 个非空），空指针占的空间与结点本身同阶。线索（穿线）二叉树就是把这些空指针利用起来存遍历前驱后继，从而不用栈也能完成中序遍历。

考前速查表

一页纸能带走的东西。**公式后面括号里是本篇的题号**，忘了推导就翻回去看。

渐进分析

事项	结论
大O定义	存在正常数c, n_0，使一切$n \ge n_0$都有$f(n) \le c\,g(n)$（1-4）
常见阶	$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < 2^n < n!$（1-5）
$T(n) = T(n/2) + c$	$\Theta(\log n)$（1-23）
$T(n) = 2T(n/2) + c$	$\Theta(n)$
$T(n) = 2T(n/2) + n$	$\Theta(n\log n)$（1-26）
$T(n) = T(n-1) + n$	$\Theta(n^2)$
$\log_2(n!)$	$\Theta(n\log n)$（1-28）
调和级数$1 + \frac{1}{2} + \cdots + \frac{1}{n}$	$\Theta(\log n)$；故$\sum_{i\le N} N/i = \Theta(N\log N)$（1-22）
$T(n) = 2T(n-1) + c$	$\Theta(2^n)$；注意$4^{n/2} = 2^n$，同阶要明写（1-8）
外层倍增、内层跑到i	$1 + 2 + 4 + \cdots + 2^k < 2n$，是$\Theta(n)$不是$\Theta(n\log n)$（1-14）

线性表

事项	结论
顺序表插入 / 删除平均移动	$n/2$ ／$(n-1)/2$（2-2、2-25）
链表省空间的条件	装填程度$< \frac{s}{s+p}$（2-5、2-27）
单链表插入口诀	**先接新结点的后继，再改前驱的指针**（2-8）
双链表在 `p` 后插 `s`	`p->next` **最后改**；或先存进 `s->next`，之后都从 `s->next` 走（2-12）
双链表删除 `p`	`p->prev->next = p->next;` `p->next->prev = p->prev;`（2-4）
双链表交换相邻两结点	别背六句，写成「先摘再插」；或直接换数据域（2-22）

事项	结论
有序链表二分	可以折半比较，但循链取中点使总时间仍为$O(n)$（2-26）
丢了头指针还能走遍全表	循环链表；双链表即使不循环也行，先沿 `prev` 退回表首（2-13）
两链表的第一个公共结点	长表先走长度差，再同步走、**比地址**；$O(m+n)$ 时间、$O(1)$ 空间（2-24）

栈与队列

事项	结论
出栈序列种数	Catalan $C_n = \frac{1}{n+1}C_{2n}^{n}$：$1, 2, 5, 14, 42, 132$（3-3、3-30）
合法出栈序列判据	不存在$i<j<k$使$p_j < p_k < p_i$（3-8、3-31）
合法**操作**序列（I/O）判据	I、O 总数相等，且每个前缀里 I 不少于 O（3-33）
两个栈做队列 / 两个队列做栈	前者出队摊还$O(1)$；后者总有一端每次$O(n)$（3-26 那版是出栈），**摊不掉**（3-12、3-26）
顺序栈与链式栈	操作都是$O(1)$；顺序栈有空槽、可能上溢，链式栈每元素多一个指针、每次一次分配（3-14）
循环队列（牺牲一格）	空 `front==rear`；满 `(rear+1)%M==front`；长 `(rear-front+M)%M`（3-2、3-32）
循环队列（计数器）	空 `count==0`；满 `count==M`，可存满（3-15）
消递归要存什么	参数、局部变量、**返回点编号**（3-5、3-23）
尾递归	可直接改循环，空间降到$O(1)$（3-16）
后缀求值	**先弹的是右操作数**（3-22）
只进不出的双端队列	n 个元素可得2^{n-1} 种序列（3-7）
嵌套解压 `k[s]`	数字栈 + 串栈：`[` 保存外层现场，`]` 取回、重复后拼接（3-27）
最小栈	辅助栈存历史最小，压入条件用 $\leqslant$，重复的最小值每份都记（3-28）
直方图最大矩形	单调栈 + 高度 0 的哨兵；出栈时以它为高、新栈顶为左界结算，$O(n)$（3-29）

字符串

事项	结论
next 定义（未优化）	`next[i]` = `P[0..i-1]` 的最长公共真前后缀长度，`next[0] = -1`（4-4）
next 优化	`P[i]==P[k]` 时借用 `next[k]`，于是会出现-1（4-14）
优化 next 逐位算	借用的 `next[k]` 是**已优化**的值（4-17）
失配后右移位数	$i -$ `next`$[i]$，**目标下标不回退**（4-10、4-13）
KMP 代价	匹配$O(n)$ + 预处理$O(m)$（4-25、4-27）
最小循环节	$p = n - f$（f 为**整串**的最长公共真前后缀长度）；至少重复两次当且仅当$p < n$ 且p 整除n（4-6、4-26）
循环移位判定	B 是A 的循环移位，当且仅当两串等长且B 是$A + A$ 的子串（4-22）
列出全部不同子串	按长度分组找齐，再逐位比较排字典序（4-17）
平方子串AA	按半长L 扫描，$S[j] = S[j + L]$ 连续成立L 次即找到，$O(n^2)$（4-23）
最长回文前缀	对 `S + '#' + reverse(S)` 求 next（模拟卷四·2）
内存/串区间重叠	`memcpy`/`strcpy` 未定义；`memmove` 先比地址再定拷贝方向（4-16）
回文最少分割	$f[i] = \min\{f[j] + 1\}$，$O(n^2)$；贪心切最长回文前缀会错（4-7）
每个前缀的出现次数	未优化 next 算到$j = n$，`cnt` 自长向短沿 next 累加，$O(n)$（4-24）

二叉树

事项	结论
第i 层 / 深度k（根在第 0 层）	至多2^i ／$2^{k+1} - 1$ 个结点（5-2）
完全二叉树高度	$\lceil \log_2(n + 1) \rceil$ 层（5-36）
顺序存储下标	父$\lfloor (i - 1)/2 \rfloor$（$i > 0$），孩子$2i + 1$、$2i + 2$（5-4）
最后一个分支结点	$\lfloor n/2 \rfloor - 1$；叶子$\lceil n/2 \rceil$ 个（5-5）

事项	结论
叶子与度 2 的关系	$n_0 = n_2 + 1$（5-3、5-33）
空指针数	$n + 1$（5-33 推论、模拟卷五）
形态数	Catalan（5-1、5-35）
定树	前序 + 中序 √，后序 + 中序 √，前序 + 后序 ✗（5-12、5-18）
三个序列都残缺	以中序切左右、前后序认根，每段缺的标识由结点集合反推（5-19）
建堆	筛选法$O(n)$，逐个插入$O(n \log n)$（5-16、5-34）
堆的增删	$O(\log n)$；查任意值$O(n)$（5-20）
Huffman 树	n 个叶 $\to 2n-1$ 个结点、无度 1 结点；WPL = 各次合并新权之和（5-7、5-26）
BST 删除	用中序前驱（左子树最右者）顶替，**先安置它的左子树**（5-14、5-24）
带约束的 BST 计数	先写无约束的 Catalan 递推，再收窄求和范围（5-25）
固定的 Huffman 码遇上频率漂移	可能反输给定长编码；重算出来的才始终不差于定长（5-27）
向下路径和计数	按起点双重递归，或前缀和一趟周游，$O(nh)$；结点可负时不能剪枝（5-30）
扩充二叉树的前序	单个序列就能唯一确定原树；外部结点$n+1$ 个（5-9）
2-3 树的结点数	先由$2^h \le$ 叶数$\le 3^h$ 定高，再自底向上逐层取$\lceil L/3 \rceil .. \lfloor L/2 \rfloor$（5-10）
父指针求第k 小	从最左结点起走$k-1$ 次中序后继，$O(h+k)$，不用栈也不加标记（5-31）
最近公共祖先	三叉链：对齐深度后同步上移，$O(h)$；二叉链：后序递归，$O(n)$（5-32）

树与森林

事项	结论
度数公式	$n = 1 + \sum_i i \cdot n_i$（6-2、6-31）
森林转二叉树	长子作左、兄弟作右；**一棵树转出的二叉树根无右子树**（6-4）
遍历对应	先根 = 前序，后根 = 中序，**层次不对应**（6-5、6-13、6-28）

事项	结论
右链长度	从二叉树根沿右链走到底的结点数 = 森林中树的棵数（6-10）
K 叉树编号（根 0）	孩子$Ki+1..Ki+K$，父$\lfloor (i-1)/K \rfloor$（$i>0$），第i 层至多K^i 个（6-6）
K 叉树空指针	$n(K-1)+1$（6-7、6-29）
满K 叉树叶结点数	$n(K-1)+1$（n 为**内部结点**数）；与上一行同源不同义（6-32）
高h 层的满K 叉树结点数	至少$K(h-1)+1$，至多$\frac{K^h-1}{K-1}$（6-32）
父指针的长处	**往上走只有一条路**：判祖先/后代、并查集（5-29、6-23）
求镜面映射的层次序列	不用真反转树，**入队时孩子从右往左**（6-27）
并查集优化	重量权衡合并（比**元素个数**）→ 树高$O(\log n)$；路径压缩在 `Find` 时做（6-8、6-23、6-30）
并查集根存— 元素个数	`parent[x] < 0` 即根，绝对值即重量，省掉 `weight` 数组（6-19）
顺序存储的通法	「某种遍历序列 + 刚好够还原结构的附加信息」，现推比背可靠（6-25）
多叉树非递归后根	栈存（结点, 下一个孩子序号），先压回自己再压孩子（6-22）
嵌套括号表示法	紧跟结点的每一对括号就是它的一棵子树（6-3）
$B(F)$ 的空指针	左空 = F 的叶结点数；右空 = 分支结点数 + 1（6-14）
$B(F)$ 中的「祖父」	原森林中是父子、兄弟、叔侄或祖孙，不会是堂兄弟（6-15）
祖先判定	u 是v 的祖先 ⇔ 先根中u 在前且后根中u 在后（6-33）

答题格式提醒

1. **先声明约定**：下标从 0 还是 1、根在第 0 层还是第 1 层、高度算层数还是边数。
2. **设计题先写 ADT**：用到栈、队列、并查集时，先列出函数名、入口参数、出口参数、功能一句话。

3. **算法之后一定跟三件事**：基本思想（一两句）、正确性依据（不变式或归纳）、时空代价。
4. **复杂度要说清是哪种情况**：「最坏情况下$O(n^2)$」比光写$O(n^2)$ 稳。
5. **画图题把中间步骤留在卷面上**：建堆、Huffman、并查集这三类题，过程分往往比结果分多。